`ComplexGitSync` 6.2.0

Documentation

Nicolas Flipo

October 13, 2026

Contents

1 Getting Started **7**
- 1.1 What `ComplexGitSync` is for . . . 7
- 1.2 The project lifecycle . . . 7
- 1.3 Public and private repositories . . . 8
- 1.4 Memory and persistence . . . 9
- 1.5 Installation . . . 9
- 1.6 Standalone or nested . . . 9
- 1.7 A first project . . . 10
- 1.8 Day-to-day work . . . 11
- 1.9 Getting help . . . 11
- 1.10 Next steps . . . 12

2 Direct Python Object API **13**
- 2.1 Programmatic Project Configuration . . . 13
- 2.2 From Empty Registry to READY via `.gts` . . . 13
- 2.3 Object-Level Tag Propagation . . . 14
- 2.4 READY Methods in the Client API . . . 14
- 2.5 Landing a Branch: `merge` and `refresh_private` . . . 18
- 2.6 Targeting a Single Path: `add` and `remove` . . . 19
- 2.7 `.gitignore` Lifecycle Sync and Commit Identity . . . 19
- 2.8 Forcing a Clone Protocol . . . 20
- 2.9 Repository Discovery: `discover_repos` . . . 20
- 2.10 Submodule Migration: `import_submodules` . . . 21
- 2.11 Adopting a Submodule Tree: `init_from_submodules` . . . 21
- 2.12 Looking at a memory: `memory_status`, `memory_list`, `memory_show`, `memory_explore` 22
  - 2.12.1 Starting a memory's history over: `memory_reboot` . . . 24
  - 2.12.2 Keeping a branch's memory when its project branch is merged: `memory_merge` 24
  - 2.12.3 Recording one piece of agent work: `self_history_add` . . . 25
  - 2.12.4 Adopting self-history: `self_history_adopt` . . . 26
  - 2.12.5 Reading self-history: `memory_self_history` . . . 26
- 2.13 Machine-readable output: `status_json` and `verify_json` . . . 26
- 2.14 Environment observation and drift . . . 27
- 2.15 A memory as a repository: `memory_init`, `memory_clone`, `memory_push` . . . 27
  - 2.15.1 A developer's tree with no memory: `memory_setup` . . . 28
- 2.16 Ledger Integrity: `verify` . . . 28
- 2.17 Diagnosing and Repairing a Git Situation: `autofix` . . . 29

3 Architecture Overview **31**
- 3.1 Three-Tier Architecture . . . 31
- 3.2 Architectural Positioning . . . 31
- 3.3 Tier 1 — Core Data . . . 32

3.4 Tier 2 — Actions . . . 33
3.5 Tier 3 — Client / API . . . 35
3.6 Module Layout by Tier . . . 36
3.7 Document Hierarchy . . . 37
3.8 Registry Model . . . 37

4 Writing ċgs Files: The Complete Authoring Guide 41
4.0.1 What is a ċgs File? . . . 41
4.0.2 Minimal ċgs: The Simplified Form . . . 41
4.0.3 Normalization: What the Simplified Form Expands To . . . 42
4.0.4 Inline Table Syntax: Overriding Defaults . . . 43
4.0.5 Advanced Table Syntax: Legacy Form . . . 44
4.0.6 Repository Placement and Path Resolution . . . 44
4.0.7 Nested Configuration Discovery . . . 45
4.0.8 Access Protocols and Custom Providers . . . 46
4.0.9 Branches: default_branch and fallback_branch . . . 46
4.0.10 Private Repositories . . . 47
4.0.11 Complete Reference: All Recognized Fields . . . 47
4.0.12 Practical Examples . . . 48
4.0.13 Authoring with CLI Tools . . . 49
4.0.14 Validation and Common Errors . . . 50
4.0.15 Templates and Starting Points . . . 50
4.0.16 Best Practices . . . 51
4.0.17 Internal Round-Trip Guarantee . . . 51

5 User Guide 53
5.1 CLI Overview . . . 53
5.1.1 Prerequisites . . . 53
5.2 Lifecycle Contract . . . 54
5.3 Commands . . . 55
5.3.1 `initialise` . . . 55
5.3.2 `bootstrap` . . . 56
5.3.3 `discover` . . . 57
5.3.4 `pull` . . . 58
5.3.5 `pull -force` . . . 59
5.3.6 `autofix` . . . 59
5.3.7 `checkout` . . . 60
5.3.8 `branch create` and `branch list` . . . 61
5.3.9 `fetch` . . . 61
5.3.10 `branch close` . . . 61
5.3.11 `branch check` . . . 62
5.3.12 `branch delete` . . . 62
5.3.13 `add` . . . 63
5.3.14 `rm` . . . 63
5.3.15 `commit` . . . 63
5.3.16 `push` . . . 64
5.3.17 `tag` . . . 64
5.3.18 `release`: `freeze`, `list`, `load` . . . 64
5.3.19 `submodules report` and `submodules import` . . . 66
5.3.20 `submodules init` . . . 68
5.3.21 `memory` . . . 68
5.3.22 `self-history` . . . 72

5.3.23 `verify check` and `verify repair` . . . 73
5.3.24 `env show` and `env check` . . . 75
5.3.25 `view-tree` . . . 75
5.3.26 What `cgitsync` promises a script . . . 76
5.3.27 Where a command works: the default workspace . . . 77
5.3.28 `status` . . . 77
5.3.29 `merge` . . . 79
5.3.30 `pull -private` . . . 81
5.4 Document Formats . . . 81
5.4.1 Local Authoring Spec (`.cgs`) . . . 81
5.4.2 Branches in a `.cgs` . . . 83
5.4.3 Repository Placement and Nested Discovery . . . 86
5.4.4 Git Tree State Snapshot (`.gts`) . . . 87
5.4.5 Local Git Register and Sync Ledger (`.lgr`) . . . 88
5.5 Worked Examples: From `CGSil1` to `cawaqsviz` . . . 88
5.5.1 Level 1 — `CGSil1`: the minimal synthetic topology . . . 89
5.5.2 Level 2 — `cawaqs`: the same minimal style, at scale . . . 90
5.5.3 Level 3 — `cawaqsviz`: three ways to the same `.cgs` . . . 90
5.6 Configuration and Environment . . . 92
5.7 Error Reference . . . 92
5.8 Troubleshooting . . . 93

Chapter 1

Getting Started

This chapter takes a new user from installation to a first persisted State of a project. It follows the structure of the project's `README.md`, with the detail needed to act on each part. Every command and option is described in full in the User Guide.

1.1 What `ComplexGitSync` is for

Scientific projects are becoming more complex: beyond source code, they rely on documentation, tutorials, data, continuous integration and deployment, and the persistence of their results — each often held in a Git repository of its own. `ComplexGitSync` manages such a project as a distributed, git-native operating space, in the form of a **GitTree**: a tree of nested Git repositories that is synchronised, versioned and restored as one unit.

A project can couple **public and private repositories**, so that intellectual property is preserved even when the project is developed collaboratively. Private repositories can also carry the configuration of **agentic assistance** for different parts of the project. Every state the project goes through is recorded, which gives the project **persistence and reproducibility**: any recorded state can be rebuilt, exactly, on another machine.

1.2 The project lifecycle

Figure 1.1 shows the five steps every project goes through. Each is described below with the vocabulary the rest of this manual uses.

1. Describe. The project's topology is written once in a `.cgs` specification, a short TOML file: which repositories the project is made of, where each one sits in the tree, and which are private. A `.cgs` names branches, so it always gives the latest work. A `.gts` snapshot recorded earlier can be used in its place: it names exact commits, so it gives the project exactly as it was.

```
project = "my-project"
repos = [
  "github:me/my-project",          # the root repository
  "github:me/my-project-docs",     # a project repository
  { repository = "github:me/.myRules",
    private = true, writable = true },   # a private one
]
```

Each identifier has the form `provider:owner/name`; the providers are `github`, `gitlab`, `codeberg` and `custom`. Branch, protocol and path all have defaults, and the file `examples/template.cgs` is a starting point. Before anything is cloned, `validate` checks a `.cgs` and `view-tree` draws the tree it describes:

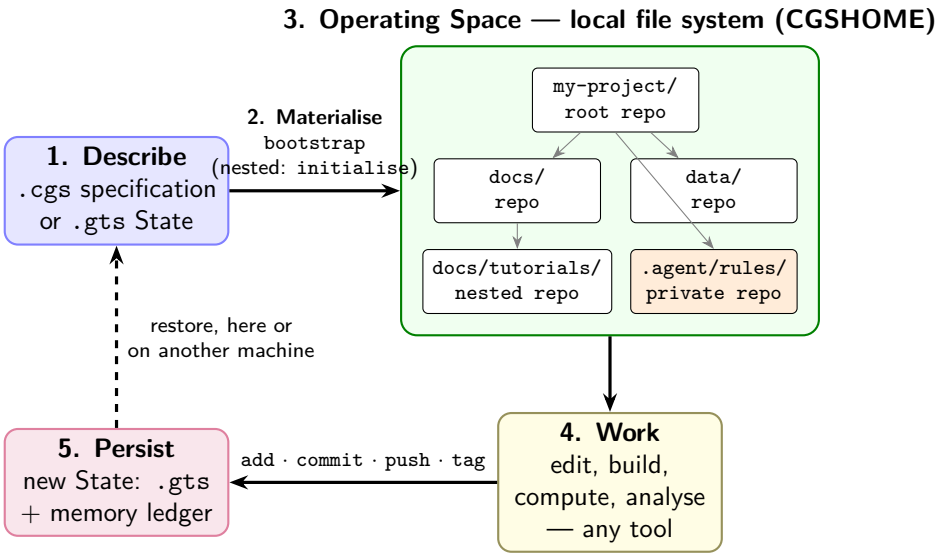


Figure 1.1: The lifecycle of a project: a description is materialised as a local file system, the Operating Space; the project is worked on with any tool; its new state is persisted, and can be restored from there.

```
$ pixi run cgitsync validate my-project.cgs
$ pixi run cgitsync view-tree my-project.cgs
```

2. Materialise. `bootstrap`, the generic command for any kind of project, clones every repository the description names and places it at its path in the tree. `initialise` is its nested counterpart, run from inside the project (Section 1.6).

3. The Operating Space. The result is an ordinary directory tree on the local disk, called the project's `CGSHOME`. Some of its sub-directories are Git repositories of their own, public or private, nested at the paths the description gave them. ComplexGitSync keeps its own records in `CGSHOME/.cgitsync/`: the States, the ledger, and a log of every command it ran.

4. Work. The project files are used with any tool, exactly as in any other directory: editors, compilers, notebooks, simulation codes, plain `git` inside one repository. ComplexGitSync does not intervene.

5. Persist. `cgitsync` runs each Git operation across the whole tree, in the right order, and checks every repository before changing any of them. Each command that changes the tree records the resulting **State**: a `.gts` snapshot holding the exact commit of every repository, entered in the project's **memory**, a hash-chained ledger that can be verified. A State can be given to `bootstrap` (or, in the nested mode, to `initialise`) to restore it — which closes the cycle.

1.3 Public and private repositories

A GitTree holds two kinds of repositories.

Project repositories are the work itself: code, documents, public data. They all follow the project's branch: `cgitsync checkout feature` moves every one of them to `feature`.

Private repositories are how the project is run: pipelines, agent instructions, internal rules, private data. They are declared with `private = true`, are often shared between several projects, and keep their own branch rather than following the project's.

A private repository is read-only unless its entry adds `writable = true`. A writable one is written to only on request: commands act on the project repositories by default, and on the writable private ones with `-private`. ComplexGitSync never writes to a read-only one.

```
$ pixi run cgitsync commit "update the analysis"            # project
    repositories
$ pixi run cgitsync commit --private "tighten a rule"       # writable
    private ones
```

`tutorials/05_private_repos.md` works through a complete example.

1.4 Memory and persistence

Every State, and every operation that produced one, is entered in the project's memory, together with the tool versions and the environment it ran in. A project with no private repository keeps its memory on the local disk only. A project with private repositories is expected to keep it in a private repository of its own, so that its history outlives any single machine; `cgitsync` offers to set this up the first time it records something:

```
$ pixi run cgitsync memory setup       # create and adopt the memory
    repository
$ pixi run cgitsync memory list        # every recorded State, newest
    first
$ pixi run cgitsync memory as-of 2026-09-01  # the project on that date
$ pixi run cgitsync verify check       # does the recorded history verify
    ?
```

Once adopted, the memory is sent with every `push`, `tag` and `release freeze`. `tutorials/06_memory.md` covers it in full.

1.5 Installation

Tool	Why it is needed
Git	Clones and synchronises every repository.
Pixi	Installs the locked Python environment and runs `cgitsync`; Python itself comes with it.
`gh`, `glab` or `tea`	Optional: creates repositories on GitHub, GitLab or Codeberg (`repo create`, `memory setup`).

Git authentication must also work for every remote the project uses.

```
$ git clone https://github.com/flipoyo/ComplexGitSync.git
$ cd ComplexGitSync
$ pixi install
$ pixi run cgitsync --help
```

`cgitsync` is a Pixi task, not a globally installed program: every command is typed as `pixi run cgitsync ...` from inside the ComplexGitSync clone.

1.6 Standalone or nested

The ComplexGitSync clone can manage a project from outside it or from inside it. The choice is made once, by the command used to materialise the project.

Standalone (recommended). ComplexGitSync stays where it was cloned, and `bootstrap` clones the whole project tree, root included, into a new workspace of its own, by default `$HOME/.cgs/<project name>-<timestamp>`:

```
$ pixi run cgitsync bootstrap my-project.cgs
$ export CGSHOME=/home/me/.cgs/my-project-<timestamp>   # printed by
   bootstrap
$ pixi run cgitsync status
```

Since commands are run from the ComplexGitSync clone, the `CGSHOME` variable tells them which workspace to act on. Every command prints the workspace it picked and why — from `$CGSHOME`, or from the current directory — and warns when that is not where you are standing — the usual sign of an export left over from an earlier project.

Nested. The project's root repository is cloned first, and ComplexGitSync is cloned inside it; `initialise` then builds the rest of the tree around them, with no export needed. This suits projects that ship the tool along with their own content, such as hydrological digital twins, where whoever obtains the project obtains the means to synchronise it:

```
$ git clone https://gitlab.com/CGS_test/CGSil1.git
$ cd CGSil1
$ git clone https://github.com/flipoyo/ComplexGitSync.git
$ cd ComplexGitSync
$ pixi install
$ pixi run cgitsync initialise ../CGSil1.cgs
```

Each command refuses the other's job by name before touching the disk: `initialise` run from outside a checked-out root names `bootstrap`, and `bootstrap` into a directory that already holds a checkout names `initialise`. `status` reports which case is in force, as `use_case=standalone` or as `use_case=nested`.

1.7 A first project

Choose the starting point that matches the project; each has a tutorial that walks through it end to end.

To discover the tool on a sample tree. The small public tree `CGSil1` has nothing of your own at risk. `tutorials/01_first_multi_repo_workspace.md` installs it with `bootstrap`, and shows the same tree installed nested with `initialise` so the difference is concrete; `tutorials/02_working_with_a_tree.md` then runs the whole lifecycle on it.

The project already provides a `.cgs`. This is the usual case, for any kind of project: use `bootstrap`, as in Section 1.6. `tutorials/03_onboarding_a_real_build_tree.md` does this for a real 19-repository project.

The project is on disk but has no `.cgs`. `discover` scans a directory for Git repositories and drafts the `.cgs` from what is checked out. It writes nothing until `-write` is given, and a relative file name is written inside the scanned directory; review the draft before using it.

```
$ pixi run cgitsync discover ~/work/my-project --write my-project.cgs
$ pixi run cgitsync validate ~/work/my-project/my-project.cgs
```

The project uses git submodules. `submodules init` adopts the checkout in one command — discover, write the `.cgs`, initialise, then turn each submodule into a plain nested repository — and `-dry-run` shows the plan first. It leaves the conversion staged, not committed. `tutorials/04_adopting_a_real_project.md` walks through it.

```
$ pixi run cgitsync submodules init ~/work/my-project --dry-run
$ pixi run cgitsync submodules init ~/work/my-project
```

1.8 Day-to-day work

Once the tree is materialised, each Git operation is run once, for the whole tree. `tutorials/02_working_with_a_tree.md` works through these commands on the sample tree, including going back to a release from its State:

```
$ pixi run cgitsync status                  # where every repository
    stands
$ pixi run cgitsync view-tree               # the tree, drawn
$ pixi run cgitsync pull                    # bring every repository up
    to date
$ pixi run cgitsync checkout my-feature     # move the project to a
    branch
  ... work on the files ...
$ pixi run cgitsync add
$ pixi run cgitsync commit "what changed"
$ pixi run cgitsync push
$ pixi run cgitsync release freeze "first release"   # add, commit, pull
    , push, tag, record
$ pixi run cgitsync release list                     # every release,
    whoever made it
$ pixi run cgitsync release load 1                   # back to a release
    , detached
```

`status` prints one row per repository — its branch, whether its files are `clean` or `dirty`, and whether it is `synced`, `ahead` or `behind` its remote — and a summary line naming the project's branch. `pull` visits the tree from the root down; `add`, `commit` and `push` from the leaves up, so a parent never records a child that is not yet committed. `add`, `commit`, `push` and `release freeze` accept `-dry-run` to show what they would do.

Commands that could lose work refuse instead. `merge` checks every repository before merging any, so a conflict anywhere leaves the whole tree untouched; `pull -force` refuses while a commit exists that no remote holds, and sets uncommitted work aside with `git stash` rather than discarding it.

1.9 Getting help

```
$ pixi run cgitsync --help              # every command, grouped by
    purpose
$ pixi run cgitsync <command> --help    # one command: purpose, options,
    examples
$ pixi run cgitsync help --all          # every command and option on one
     page
```

`-h` is accepted wherever `-help` is. When a command fails, it prints one line saying why, and its full log is kept in `CGSHOME/.cgitsync/logs/`; `cgitsync autofix` reads the last one and diagnoses the situation.

1.10 Next steps

The six tutorials under `tutorials/` work through the cases above, from the simplest to the most advanced. The User Guide is the reference for every command, option, document format and error.

Chapter 2

Direct Python Object API

This chapter shows how to manipulate `ComplexGitSync` objects directly (without the CLI wrapper), including loading a `.gts` snapshot, reconstructing the runtime registry, mirroring identity into `GitTree`/`GitRepo`, and propagating a tag target.

For normal Multi-repo Sync usage, prefer the CLI lifecycle documented in the README and Getting Started guide. The reference test topology is CGSil1 (`https://gitlab.com/CGS_test/CGSil1`): `initialise -> pull -> checkout/add/commit/push -> freeze_release -> list_releases -> load_release`.

The lower-level object/API lifecycle remains: `load(.cgs) -> .gts LOADED -> expand(.cgs|.gts) -> .gts PENDING -> validate(.cgs|.gts) -> .gts READY -> pull(.cgs|.gts) -> checkout(.gts) -> add -> git(tree,"commit",msg) -> git(tree,"push") -> git(tree,"tag",name) -> freeze_relea`

2.1 Programmatic Project Configuration

Project configuration does not require the CLI or an existing `.cgs` file. The public facade accepts authoring values non-interactively and delegates normalization, static validation, and optional serialization to `cgs_format.py`. It performs no Git or network operations.

```
from ComplexGitSync import ComplexGitSyncClient

client = ComplexGitSyncClient()
document = client.configure(
    "GX4G",
    ["codeberg:GX4G/GX4G"],
    output_path="GX4G.cgs",  # optional
)
reference_tree = document.to_git_tree()
```

`configure` is the `.cgs` writer and has no CLI command of its own. `discover -write`, `submodules init`, and direct `initialise -project/-repo` authoring are the adapters over it.

2.2 From Empty Registry to READY via `.gts`

```
from pathlib import Path

from ComplexGitSync import GitRepo, GitTree, RefKind, TreeLifecycleState
from ComplexGitSync.orchestre import GtsDocument
from ComplexGitSync.registry import RegistryTranslator
from ComplexGitSync.git_tree import WorkingGitTree

# 1) Empty registry lifecycle
```

```
empty_registry = WorkingGitTree()
assert empty_registry.recompute_tree_state() == TreeLifecycleState.
    UNLOADED

# 2) Load the CGSil1 runtime snapshot (.gts) and rebuild the dependency
    registry
cgshome = Path("../..")
gts_doc = GtsDocument.from_toml(cgshome / ".cgitsync/state/CGSil1.gts")
registry = RegistryTranslator.from_gts_document(gts_doc)
assert registry.lifecycle_state == TreeLifecycleState.READY

# 3) Rebuild GitTree/GitRepo identity objects from discovered registry
    entries
tree = GitTree()
for entry in registry.values():
    tree.add_repo(
        GitRepo(
            project_owner_name=entry.project_owner_name or "owner",
            project_name=entry.project_name or entry.name,
            gitprovider=entry.gitprovider,
            access_protocol=entry.access_protocol,
            commit_sha=entry.commit_sha,
        )
    )
```

2.3 Object-Level Tag Propagation

```
# Set the same tag target across all registry entries (parent -> leaves)
tree.propagate_tag(registry, "v1.2.3")
for entry in registry.values():
    assert entry.target_ref_kind == RefKind.TAG
    assert entry.target_ref_name == "v1.2.3"
```

This object-level flow is the same state model used by the CLI and `ComplexGitSyncClient`, but exposed directly for advanced automation and tests.

2.4 READY Methods in the Client API

The same lifecycle model is exposed by `ComplexGitSyncClient`. In practice, the workflow is:

- `load(...)` is the step-1 name for direct source loading.
- `expand(...)` is the canonical step-2 name; it runs nested `.cgs` discovery from parents to leaves (recursive).
- `validate(...)` is the canonical step-3 name; it accepts both `.cgs` and `.gts` sources.
- `initialise(...)` bootstraps a tree and must finish in `READY`; `bootstrap(...)` is the standalone entry point that clones the root too; `pull(...)` resynchronises an existing tree.
- `git(tree, command, *args)` is the unified step-5 interface for `"commit"`, `"push"`, and `"tag"` operations. Each follows leaf-first ordering.

- `freeze_release(commit_message, *, force_tag=None, output_gts=None, stage_all=True, force_access_protocol=None)` is the one release procedure, `release freeze` on the command line. It tags `<project-name>-<suffix>`: `force_tag`, else the version in the root's `pixi.toml`, else the next number. The root's tag is annotated: its subject carries the freeze message, and its body is the project-scope State as TOML. `next_release_tag(force_tag=None)` returns that tag, the version and where the suffix came from, without changing anything. `freeze(...)` and `freeze_state(...)` are internal steps that emit the next persisted `.gts` state and have no CLI command of their own.
- `list_releases()` returns one dict per release of the project, newest first: `tag`, `version`, `recorded_at`, `by`, `state`, `made_with`, `source` (`ledger`, `tag`, `ledger+tag` or `tag only`) and `loadable`. It reads the ledger and the root repository's tags, so a release another user made is listed too.
- `load_release(name, *, workspace=None, cgs_path=None)` puts every repository back at the release's recorded commit, detached, cloning any the tree lacks, and refuses a tree with uncommitted changes; afterwards `last_release_load` (a `ReleaseLoad`: the tag, the branch the tree was on, the repositories cloned, the ones left alone) says what it did. With `workspace` it builds a new workspace from the release's State, as `bootstrap` does. `name` is the whole tag or what follows the project's name. Each row of `list_releases()` also carries the freeze `message`.

```
from pathlib import Path

from ComplexGitSync import ComplexGitSyncClient

client = ComplexGitSyncClient()
cgspath = Path("../..")
cgshome = cgspath / "CGSil1"

# 1. load the CGSil1 test-case .cgs -> .gts LOADED
client.load("../CGSil1.cgs")

# 2. expand(.cgs/.gts) -> .gts PENDING
print(client.expand("../CGSil1.cgs"))

# 3. validate(.cgs/.gts) -> .gts READY
client.validate("../CGSil1.cgs")

# 4. clone(.cgs/.gts)
# Same default as the CLI: output_path defaults to CGSPATH=../..
client.initialise("../CGSil1.cgs")

# Equivalent explicit form:
client.initialise("../CGSil1.cgs", output_path=cgspath)

# initialise refuses, changing nothing, when a destination holds work
    that
# exists nowhere else; there is no clean-up call. Commit and push it, or
     move
# the directory aside yourself.

# 5. READY methods - unified git interface
registry = client.get_dependency_registry()
# Pull uses ROOT -> PARENT -> LEAF; every repository in the tree is a
    plain
```

```
# independent clone (there are no gitlinks) and receives its own git
    pull.
# A .gts source must be an actual snapshot path (find the current one
    via
# RuntimeStateStore.latest_snapshot_for(), the same lookup the CLI uses
    when
# a command's snapshot argument is omitted) -- unlike a .cgs source,
    there is
# no fixed, predictable filename to hard-code here.
client.pull("../CGSil1.cgs")
# pull() fetches every branch of each remote, so checkout() can join a
    branch
# somebody else pushed: it starts the local branch from the remote-
    tracking
# ref when one exists, and only otherwise from the current HEAD.
client.checkout("feature/my-branch")
# Mutations use LEAF -> PARENT -> ROOT.
client.add()
client.git(registry, "commit", "feat: update CGSil1 CGS#1")
client.git(registry, "push")
client.git(registry, "tag", "v1.2.3")

# 6. release: add, commit, pull, push, tag CGSil1-<version or number>,
    record the State
client.freeze_release("release 2026.05")
client.freeze_release("a beta", force_tag="beta")  # tagged CGSil1-beta

# Every release of the project, whoever made it, and back to one later:
for release in client.list_releases():
    print(release["tag"], release["source"])
client.load_release("beta")
```

Every one of these that reaches a remote folds the memory first. When this tree mounts and has adopted a memory (`.cgitsync/.memory`), `push()`, `tag()`, and `freeze_release()` each call `memory_push` on it before doing their own work — regardless of `private`, since the memory is this project's own record of itself, not a configuration repository scoped by that flag. `client.last_memory_fold` holds what the most recent one of these folded: the `memory_push` result dict, or `None` when no memory is mounted or the fold could not proceed. A fold that fails — the memory declared but not yet adopted, or unreachable for any other reason — is never raised: it is turned into a `warnings.warn(...)` call instead (catchable with `pytest.warns`/`warnings.catch_warnings`, the same as `checkout`'s own build-change warning), so a Python caller sees it exactly as a CLI user would, and the write it was folding for still completes. `client.memory_declared()` answers whether a memory is mounted at all, with no side effect, for a caller that wants to know in advance.

`list_branches(private=False).` Returns one `RepoBranches` per repository, parent-first: its `name`, the `current` branch (`None` when detached) and the sorted local `branches`. Read-only; a repository not yet cloned is left out. `private=True` narrows it to the writable configuration repositories, as for `branch()`.

```
for repo in client.list_branches():
    print(repo.name, repo.current, repo.branches)
```

project_branches(private=False). Returns one ProjectBranch per branch of the project — the root's local and origin branches, a closed/ branch under its original name with closed set. Each carries local, on_origin, current, following (repositories holding the branch they would be on under it), missing and uncloned. Reads refs only: no fetch, no State written. private narrows the coverage, not the branches listed.

fetch(private=False). Runs git fetch -prune origin in every cloned repository in scope and returns one RepoOutcome per repository (also on client.last_write_outcomes): acted when it fetched, otherwise detail says why not. Moves no branch and writes no State.

tree_branch_label(). Returns the branch the project is on, as status prints cgitsync_branch: a branch name, detached or unknown.

close_branch(branch_name, private=False). Closes a project branch across the full tree, leaf-first, locally and on the remote, never deleting anything. Like delete_branch, it raises GitSyncError in a user's tree, one that holds no private repository, before touching any repository. Each repository closes the branch it follows under branch_name (<project>_<branch> in a private/local repository; a private/distant one is skipped). A branch that exists only on the remote is closed there. Requires READY. Raises GitSyncError before touching any repository when branch_name is the project's own default branch, or when any repository in scope is currently on the branch it would close — the error names which. client.last_write_outcomes holds one RepoOutcome per repository visited: acted is False for a repository with no such branch or a private/distant one, never an error. private=True selects the writable configuration repositories instead of the project's own, the same as branch() (create).

```
client.close_branch("memory-dev")
for outcome in client.last_write_outcomes:
    print(outcome.name, outcome.acted, outcome.detail)
```

Before it renames anything, close_branch keeps every commit the branch alone holds on the project's ancestors branch (<project>_ancestors in a private/local repository) with a merge that only adds a commit, records each move as a Relocation in the ledger entry it writes, and refuses unless the ledger still verifies. The closed branch can then be deleted by any tool without losing a commit. client.last_ancestry holds what each repository's copy of the branch held, and client.last_kept_outcomes what keeping it did.

branch_ancestry(branch_name, private=False) answers, without writing anything but remote-tracking refs, what deleting a branch would lose: one RepoAncestry per repository, leaf-first, whose verdict is "safe", "recorded", "needs ancestor", "absent" or "skipped", with the commits only it reaches (exclusive) and the memory files only it holds (content). delete_branch(branch_name, private=False) deletes a *closed* branch on origin and locally, leaf-first, once everything it alone holds is kept and recorded; a move the close already recorded is reused, never written twice, and any failing step raises GitSyncError with nothing deleted. preserved_branches() names every project branch the ledger records as kept on ancestors, deleted ones included.

```
for answer in client.branch_ancestry("closed/feature-x"):
    print(answer.name, answer.verdict, answer.detail)
client.delete_branch("feature-x")
print(client.preserved_branches())
```

memory_as_of(cgshome, moment, branch=None) and memory_list(cgshome, branch=None) read another chapter of the memory from Git when branch is given — its branch, its closed name, or the copy ancestors keeps once it is deleted — and say which in read_from. memory_explore

does the same for a chapter not checked out here, and `memory_show` finds a State that is not on disk in any of them.

2.5 Landing a Branch: `merge` and `refresh_private`

`merge(project_branch, private=False, ff_only=False, no_ff=False)` merges a branch across the tree, leaf-first, and returns one `(repo_name, merged_ref)` pair per repository a merge moved.

The argument is always the **project's** branch name. Each repository resolves what that means for itself, so a private/local configuration repository merges `<its default_branch>_<project_branch>` rather than a branch of the project's name, which it does not have. `merge_plan(project_branch, private=False)` returns one `(repo_name, source_ref, status, conflicting_paths)` row per repository without merging anything — what the CLI's `-dry-run` prints. `status` is `"merge"`, `"already-on-it"`, `"no-branch"` or `"conflicts"`, and the last one carries the files that block it.

Every repository in scope is checked before any is merged, so a conflict anywhere leaves the whole tree untouched. The check names the conflicting files, and counts binary files: a tracked PDF changed on both sides conflicts just as a source file does.

`merge_resolve(project_branch, private=False, ff_only=False, no_ff=False)` is the opt-in alternative. It merges one repository at a time, stops at the first that conflicts, and returns a `ResolveOutcome` naming what it merged, where it stopped, the files that stopped it, and the repositories it never reached. It gives up the all-or-nothing guarantee on purpose: that is the only way to reach the conflicted worktree a merge tool needs. `ResolveOutcome.stopped_status` is `"conflicts"`, `"unrelated"` (no common commit; nothing was attempted) or `None` when the run did not stop; `stopped_source` names the branch the stopped repository was merging, and `binary_paths` the conflicting files that are not text. `merge_resolve_all(...)` repeats it: each text conflict the merge tool resolves is committed and added to `merged`, which gathers every pass, and it stops on anything else, with `hand_command` set when no merge tool could be opened. It always ends.

`open_merge_tool(repo_name)` then opens one repository's conflicts in a merge tool. It returns `None` once the tool has run, or the command to run by hand when no tool is available — a missing editor is a normal outcome, not an error. A `merge.tool` you configured always wins, and your Git configuration is never written.

`refresh_private()` is the other direction, and what `pull -private` runs: it fetches and merges each private/local repository's base branch into the derived branch it is on, so a settings branch does not drift behind the project's while a feature branch is open.

```
client.merge("multi-branch")                    # the project's own repos
client.merge("multi-branch", private=True)      # ComplexGitSync_multi-
    branch
                                                #   -> ComplexGitSync
client.merge("multi-branch", all_writable=True)   # both halves, one
    pass

# Merging into a branch that is not checked out, in one call.
client.merge_into_plan("multi-branch", "main")   # what it would do, per
     repo
client.merge_into("multi-branch", "main")        # checkout and merge,
    together
client.build_installed_from("main")              # the version that
    branch carries
client.refresh_private()                        # the other way round
```

all_writable: the API half of -all. add, commit, push, merge, merge_resolve and merge_plan each take all_writable=False alongside private=False. Setting it runs the call across this project's own repositories *and* its writable configuration ones in a single pass. The two are mutually exclusive and passing both raises; leaving both unset keeps today's behaviour exactly. Read-only configuration repositories are never written to under any combination — the parameter maps to RepoScope.WRITABLE, which is narrower than RepoScope.ALL.

2.6 Targeting a Single Path: add and remove

add(paths=None) and remove(paths) accept one or more filesystem paths (each relative to CGSHOME, or absolute) instead of operating on the whole tree:

```
# Stage only these two files, each in the repo that owns it -- every
# other repo in the tree is untouched.
client.add(paths=["notes.md", "deps/leaf/src/main.py"])

# Remove a single tracked file: deleted from disk, removal staged.
client.remove(["deps/leaf/obsolete.txt"])
```

Both resolve each path to its owning repo through the same helper, resolve_repo_for_path(tree, path) (git_tree.py) — the deepest (most specific) repo whose absolute_path contains the resolved path wins, so a nested child is correctly preferred over its parent. A path outside every repo in the tree raises GitSyncError immediately, before anything is staged or removed; for remove, a path that resolves to a directory or does not exist raises the same way rather than partially applying. add() with no paths keeps today's exact behaviour: every repo staged in full, leaf-first.

remove is a plain git rm (deletes from disk, stages the removal) — unrelated to GitRunner.rm_cached (index-only, used internally by import_submodules to drop a gitlink while keeping the child's working tree); the two are not interchangeable.

2.7 .gitignore Lifecycle Sync and Commit Identity

initialise_cgs, initialise_cgs_document, initialise_gts (the nested install from a snapshot, each repository at its recorded commit; initialise dispatches to it for a .gts), restart, and pull (for .cgs sources) all accept the same three optional keyword arguments, mirroring the CLI's -commit-gitignore, -git-user-name, and -git-user-email flags:

```
from ComplexGitSync import ComplexGitSyncClient, MasterConfig

client = ComplexGitSyncClient()
client.initialise_cgs(
    "../CGSil1.cgs",
    output_path=cgspath,
    commit_gitignore=True,        # stage/commit/push each changed .
        gitignore
    git_user_name="cgitsync-bot",
    git_user_email="bot@example.com",
)

# The identity above is persisted to CGSHOME/.cgitsync/master.toml, so a
# later call on the same workspace picks it up without repeating it:
client.initialise_cgs("../CGSil1.cgs", output_path=cgspath,
   commit_gitignore=True)
assert MasterConfig.resolve_identity(cgshome, client.git_runner) == (
    "cgitsync-bot",
```

```
    "bot@example.com",
)
```

All three keyword arguments default to `False`/`None`: by default, every repository with children (root, or any nested repository that itself has further nested children) is safely pulled and has its `.gitignore` updated, but nothing is staged, committed, or pushed, and the commit identity — were a commit ever made — would be whatever `git config user.name`/`user.email` already resolves to locally. `MasterConfig` (`master.py`) is the class behind the identity override: `configure()` sets it in memory for the current process, `persist(cgshome, ...)` writes it to `CGSHOME/.cgitsync/master.toml` (and also updates the in-memory value), `load(cgshome)` reads a previously persisted override back in, and `resolve_identity(repo_path, git_runner)` returns the `(user_name, user_email)` pair — or `(None, None)` to defer entirely to local git config. This file is workspace-local configuration, not part of the `.cgs`/`.gts` project spec, and is preserved never touched by `initialise`.

2.8 Forcing a Clone Protocol

`initialise_cgs`, `initialise_cgs_document`, `bootstrap`, and `clone_cgs` additionally accept `force_access_protocol`, mirroring the CLI's `-force-protocol` flag on `initialise`, and `bootstrap` (`restart`/`pull` do not accept it):

```
# Every repo this run clones -- root siblings and any child discovered
# later from a nested .cgs in a different repo -- is cloned over https,
# regardless of what its own .cgs entry says. No .cgs file is read
# differently or written.
client.initialise_cgs(
    "../CGSil1.cgs",
    output_path=cgspath,
    force_access_protocol="https",
)
```

`"ssh"` or `"https"`; `None` (the default) leaves every entry's own `.cgs`-declared `access_protocol` untouched. The override lives only in memory for the duration of the call – it is applied at the single point every clone's remote URL is built (`ComplexGitSyncClient._build_remote_url`), so it reaches root siblings and nested-discovered children alike without `cgs_format.py` parsing, or any `.cgs` file on disk, ever being touched differently.

2.9 Repository Discovery: `discover_repos`

`discover_repos(root_dir=None, *, max_depth=5, output=None)` scans a directory for git repositories already checked out on disk and drafts a `.cgs` from what it finds — the entry point for adopting a project that exists but has no specification yet:

```
from ComplexGitSync.orchestre import ComplexGitSyncClient

client = ComplexGitSyncClient()

# Read-only scan - nothing is cloned, fetched, or modified
report = client.discover_repos("/path/to/checkout")
for repo in report.repos:
    print(repo.relative_path, repo.identifier, repo.branch)

for warning in report.warnings:
    print("unresolved:", warning)
```

```
# Write the draft out: a relative output lands inside the scanned
# directory, so this writes /path/to/checkout/draft.cgs
report = client.discover_repos("/path/to/checkout", output="draft.cgs")
print(report.written_to)
```

The method returns a `DiscoverReport` dataclass with fields `root` (the scanned directory), `repos` (a tuple of `DiscoveredRepo`), `cgs_entries` (authoring-form repo tables ready for `configure()`), `warnings`, `project_name`, and `written_to` (the path the draft was written to, or `None`). Each `DiscoveredRepo` carries `relative_path`, `absolute_path`, `remote_url`, `identifier`, `branch`, and `has_cgs`.

Repository addresses are resolved through the existing `parse_repo_id()` grammar; a repository with no `origin` or an unrecognised provider is reported in `warnings` and omitted from `cgs_entries` rather than guessed at. Passing `output` raises `GitSyncError` when nothing resolvable was found, so an empty draft is never written.

2.10 Submodule Migration: `import_submodules`

`import_submodules(repo_root, *, apply=False, recursive=False)` converts git submodules in a local repository to plain ComplexGitSync nested clones. It is what the CLI's `submodules report` (`apply=False`) and `submodules import` (`apply=True`) run:

```
from ComplexGitSync.orchestre import ComplexGitSyncClient

client = ComplexGitSyncClient()

# Dry run - inspect without changing anything
report = client.import_submodules("/path/to/repo")
for sub in report.submodules:
    print(sub.name, sub.path, sub.url)

# Apply - convert every level, root first
report = client.import_submodules(
    "/path/to/repo",
    apply=True,
    recursive=True,
)
print(report.converted)   # tuple of converted submodule names
```

The method returns an `ImportSubmodulesReport` dataclass with fields `submodules` (all entries found), `applied` (whether the conversion ran), `converted` (names of converted submodules), and `scan_root` (the repository the call was pointed at, which every reported path can be expressed from). When `apply=False` (the default), no files are modified. When `apply=True`, each submodule is removed from the index (`git rm -cached`), its stanza is removed from `.gitmodules`, and the parent's `.gitignore` is updated to ignore the child path. The staged changes should then be reviewed and committed manually.

`recursive=True` also converts any submodule that has its own checked-out `.gitmodules`, at any depth. That walk follows the submodule graph declared by `repo_root`'s own `.gitmodules`, so a repository whose holder has already been converted is no longer reachable — which is why the conversion of a whole tree happens once, and last (see `init_from_submodules` below).

2.11 Adopting a Submodule Tree: `init_from_submodules`

`init_from_submodules(repo_root, *, cgs_path=None, max_depth=5, dry_run=False, force=False, force_access_protocol=None)` performs the whole adoption of a submodule-based checkout, in the one order that works. It is what the CLI's `submodules init` runs:

```
from ComplexGitSync.orchestre import ComplexGitSyncClient

client = ComplexGitSyncClient()

# Report the plan without writing, cloning, or converting anything
plan = client.init_from_submodules("/work/project", dry_run=True)

# Adopt: discover -> write .cgs -> initialise -> convert every submodule
report = client.init_from_submodules("/work/project")
print(report.cgs_path)                      # the .cgs used
print(report.tree.is_ready())               # True
print(report.import_report.converted)       # converted submodule names
```

Each step delegates to the method that already owns it (`discover_repos`, `initialise_cgs`, `import_submodules`); the value this method adds is the ordering and the preconditions. `initialise_cgs` adopts the root in place but deletes and re-clones every other repository from its remote, where the submodules are still declared — so converting first is undone for every non-root repository, and a repair pass cannot reach any deeper level once the root's `.gitmodules` is gone.

The method returns an `InitFromSubmodulesReport` with fields `root` (which is also `CGSHOME`), `discover` (the `DiscoverReport` from step 1), `cgs_path`, `cgs_written`, `import_report`, `tree`, and `dry_run`. Nothing is committed: the conversion touches every repository that held a submodule, so `branch`, `checkout`, `add`, and `commit` stay explicit, separate calls. A `GitSyncError` is raised before anything is written when the discovery resolved nothing, when the project name does not match the directory name (`CGSHOME` is `<parent>/<project-name>`), or when `repo_root` has no `.gitmodules` to convert (`force=True` overrides the last one).

2.12 Looking at a memory: `memory_status`, `memory_list`, `memory_show`, `memory_explore`

The four read-only questions the CLI's `memory` group asks, each returning plain data rather than text:

```
from ComplexGitSync.orchestre import ComplexGitSyncClient

client = ComplexGitSyncClient()

status = client.memory_status("/path/to/CGSHOME")
status["states"], status["entries"]      # how much is remembered
status["verification"]  # verified / no-history / legacy / corrupt /
    time-inconsistent
status["latest_toolchain"]["git"]        # what produced the newest
    record

for row in client.memory_list("/path/to/CGSHOME"):
    row["state"], row["recorded_at"], row["commands"]

state = client.memory_show("/path/to/CGSHOME", "2acdc98b")   # hash or
    prefix
state["repos"], state["entries"]
state["tree"]                            # this State's own topology,
    like view_tree()
state["environments"]                    # [{"id": "env(<hash>)", "path
    ": ..., "record": ...}]
for entry in state["entries"]:
    for commit in entry["commits"]:
```

```
        commit["message"], commit["repository"], commit["published"]
state["published"]                          # remote and ref each commit
    went to

answer = client.memory_as_of("/path/to/CGSHOME", "2026-09-30T17:00")   #
     UTC unless an offset
answer["entry"]                             # last entry recorded at or
    before it, or None
answer["entry"]["seq"], answer["entry"]["state"]   # the hash
    memory_show takes
answer["reliable"], answer["history"]       # False + "time-inconsistent"
    when a clock ran backwards
answer["findings"]                          # why, as `verify` would say

environment = client.memory_show_environment("/path/to/CGSHOME", "env(<
    hash>)")
environment["record"]["machine"], environment["record"]["tools"]

answer = client.memory_explore("/path/to/CGSHOME")
answer["branch"]                            # the memory branch checked out
     here, or None
for commit in answer["commits"]:            # published commits, newest
    push first
    commit["repository"], commit["branch"], commit["sha"], commit["
        message"]

timeline = client.memory_explore("/path/to/CGSHOME", timeline=True)
for entry in timeline["entries"]:           # every ledger entry, in order
    entry["seq"], entry["command"], entry["state"], entry["commits"],
        entry["published"]
    # entry["state"] is what memory_show(cgshome, entry["state"]) takes
```

Commit messages come back whole. Shortening a long one to a single line is the caller's decision, which is why the CLI has `-full` and this method does not: a caller reading the memory from Python wants the message that was written, not the one that fitted.

`memory_show` raises `GitSyncError` when the prefix matches no State, or more than one.

`memory_explore`'s default view answers what a colleague pulling this branch would see — commits with no publication row are left out, because that is `memory_show`'s question, not this one. `branch=` asks for a memory branch other than the one checked out on this disk; today only the checked-out one can be read this way, and a mismatch raises `GitSyncError` naming `memory_clone(branch=...)` rather than answering for the wrong branch.

The commands that give a memory a repository of its own have the same mirror. `add_memory_repo_cgs` is the one the owner named; the CLI spells it `memory mount`.

```
client.repo_create("github:you/.memory", private=True)
#   {"created": "created" | "exists" | "unavailable", "command": "gh
    repo create ..."}

client.add_memory_repo_cgs("project.cgs")        # one entry, comments
    kept
client.memory_adopt("/path/to/CGSHOME")          # .cgitsync/.memory,
    fresh and empty
client.memory_push("/path/to/CGSHOME")           # folds .cgitsync's
    pending content in first
client.memory_branch("/path/to/CGSHOME", "main") # the branch a merge
    will need

# Only for a memory mounted before this command existed, directly at
```

```
# .cgitsync instead of .cgitsync/.memory:
client.memory_migrate("/path/to/CGSHOME", "project.cgs")
```

`repo_create` never raises for a repository that already exists — that is `"exists"`, an ordinary answer — and never for a provider tool that is absent or signed out, which is `"unavailable"` carrying the command to run. It raises only for a provider no tool is known for, and for a tool that failed for some other reason. `memory_list` includes a State that entries name but that is not on disk, with `path` set to `None` — `verify` reports it as `MISSING_STATE`, and this is how a caller finds out which one.

2.12.1 Starting a memory's history over: `memory_reboot`

```
answer = client.memory_reboot("/path/to/CGSHOME")
answer["folded"]          # pending records folded in before archiving
answer["archived_from"]   # the branch's name before the reboot
answer["archived_to"]     # "<branch>.archived-<YYYYMMDD>", real, on
    origin
answer["exported"]        # .cgitsync/.memory/.cgs/<project>-v<N>.cgs
answer["branch"]          # the original name, fresh: .cgs/ kept, else
                           # cleared, one State committed and pushed

# A fresh start from the very first adopt, instead of inheriting
    whatever
# the fallback branch already holds:
client.memory_adopt("/path/to/CGSHOME", remote=remote, branch=branch,
    reboot=True)
```

Nothing is ever force-pushed or deleted: the old branch is pushed to origin under its archived name *before* the old name is removed from origin, so the commits it holds are reachable under some name on the remote for the whole operation, never only locally. The fresh branch's own genesis commit is pushed too, immediately, rather than left for the next `memory_push` — the old name is already gone from origin by then, so a bootstrap on another machine in between would otherwise find no branch under this project's name at all. `memory_reboot` raises `GitSyncError` when the mount is not a repository yet, and when today's archived name already exists — a second reboot the same day needs a name of its own rather than colliding with the first.

2.12.2 Keeping a branch's memory when its project branch is merged: `memory_merge`

```
answer = client.memory_merge("/path/to/CGSHOME", "feature-x", into="main
    ", keep="ours")
answer["status"]   # "merge" | "fast-forward" | "already-merged"
answer["detail"]   # "kept the memory of MyProject; MyProject_feature-x
    is kept as history"
answer["source"], answer["target"]   # the memory branches, derived from
     the project branches
answer["commit"]   # the merge commit, or None when nothing was due
answer["pushed"]   # False for a defaulted memory or one with no remote
```

`keep` is `"ours"` (keep `into`'s memory) or `"theirs"` (keep the source's), Git's meaning, and is required: any other value raises `GitSyncError` before anything is touched. `into` defaults to the project branch checked out. A memory is never merged file by file; the commit takes one side's files whole and records the other as its second parent. A missing source or target memory branch raises `GitSyncError` naming it (and, for the target, `memory_branch`); a push the remote would reject raises one that says the merge is already done locally. Nothing is forced.

2.12.3 Recording one piece of agent work: self_history_add

```
from ComplexGitSync.memory.self_history import AgentInfo,
    ConformityCriterion, ConformityScore

worker = AgentInfo(role="Dev", vendor="vendor-name", model="model-name")
orchestrator = AgentInfo(role="Orchestration", vendor="vendor-name",
    model="model-name")
conformity = ConformityScore(
    spec_respect=ConformityCriterion(score=33, basis="measured",
        reasoning="lint and test both pass"),
    gating=ConformityCriterion(score=33, basis="measured", reasoning="
        nothing private pushed"),
    quality=ConformityCriterion(score=30, basis="asserted", reasoning="a
         reasonable first pass"),
)

path = client.self_history_add(
    "/path/to/CGSHOME",
    ticket="AgentReport",
    goal="Implement WP1: the record format and the add command.",
    action="Wrote memory/self_history.py and self_history_add.",
    worker=worker,
    orchestrator=orchestrator,
    conformity=conformity,
    lint_passed=True,
    tests_passed=True,
)
```

Writes one record to `.cgitsync/.self-history/<hash>.toml`, content-addressed the same way a State or an Environment record is. `contract` (the current signed `AgentContractRecord`'s own hash) and `checks.status_errors` (this workspace's `errors=` count) are filled in from what the client can itself observe; every other field, including `lint_passed`/`tests_passed`, is the caller's own account — `self_history_add` cannot run `pixi run lint` or `pixi run test` itself, since `subprocess` stays confined to `git_runner.py`, and that module runs Git, not Pixi.

`AgentInfo.role` is validated against `.localSpec/AGENT.md`'s six-role roster; `ConformityCriterion.basis` against `"measured"`/`"asserted"` only. Both raise `ValueError` on anything else, the same way `goal`/ `action` raise for more than three lines.

`ConformityScore` is out of 100: `spec_respect` 0–33, `gating` 0–33, `quality` 0–34, the total their plain sum. A score outside its range raises `ValueError`. `ConformityScale` (in `memory.conformity_scale`) holds those maxima and renders a score with them. The optional `ConformityScore.explanation` is the orchestrator's short account of how the total came about; it is left out of the stored record when empty.

`state_before`/`state_after` are **verified against the ledger**, not merely shape-checked — deliberately a ledger question, not a `.cgs`/discovery one: a `.gts` is a static snapshot, so whether a citation names something real is answered the same way `verify` already answers it, via `MISSING_STATE`/ `STATE_DIGEST_MISMATCH` — never by re-parsing or re-deriving something from a transformed document. Concretely, each citation, if given, must name a State some ledger entry actually recorded, on disk, still hashing to its own name; one that fails any of the three raises `GitSyncError` rather than being recorded as though it were fact. `state_after` is additionally **observed**, not merely validated, when omitted: the ledger's own most recent entry at the moment `self_history_add` runs.

`memory_push` folds this pending directory into `.cgitsync/.memory/.self-history/` and pushes it — *before* folding and pushing `.memory` itself, once self-history has been adopted (below). A tree that never adopted it sees no change: the fold is a no-op the moment `.self-history`'s

own mount has no `.git`.

2.12.4 Adopting self-history: `self_history_adopt`

```
result = client.self_history_adopt("/path/to/CGSHOME", owner="you")
result["mount"]   # .cgitsync/.memory/.self-history
result["branch"]  # .memory's own branch, unless a different one was
    given
result["remote"]  # the SSH URL self-history was adopted from
```

`self_history_adopt` is the *one place* this fact is ever decided: it writes `config-memory.cgs` the first time, for a brand-new project or as the explicit retrofit for a `.memory` adopted *before* self-history existed (this project's own, among others). Every precondition here raises `GitSyncError` instead of returning quietly — `.memory` not yet adopted, self-history already adopted, no owner resolvable, or the remote not reachable.

There is no separate opt-in flag beyond that one decision: `memory_adopt` follows it automatically and silently afterward, whenever `.memory`'s own already-fetched content already carries `config-memory.cgs` — typically because a project's own memory branch inherited it from a fallback base branch that already had it, the same mechanism that already lets a new project's memory share history with another. This is deliberately *not* a check against `self.registry`: a `.gts` is a static snapshot of an already-discovered tree, and `nested_config` (the `.cgs`-side declaration that makes `.self-history` a discoverable child of `.memory`) is `.cgs`-only information a `.gts`-loaded registry never carries in the first place — asking the registry would be the wrong layer, not a gap a runtime probe should paper over. Reachability is checked too, but only *after* that question, to decide whether an already-declared repository can actually be fetched right now.

`memory_clone` brings self-history back too, reading which repository to clone from `config-memory.cgs` itself rather than re-deriving it, the moment that file says self-history was adopted upstream. `memory_reboot` leaves `.self-history` entirely alone by construction: it only ever touches `.memory`'s own mount.

2.12.5 Reading self-history: `memory_self_history`

```
records = client.memory_self_history("/path/to/CGSHOME")
records[0]["ticket"]        # the short ticket name
records[0]["conformity"]    # {"spec_respect": {...}, "gating": {...}, "
    quality": {...}}
```

Every record this workspace holds, folded and pending, oldest first — what the CLI's `self-history list` prints — each as the same dict `SelfHistoryRecord.to_dict()` produces – consultation only. An empty list means no self-history has been written here yet, not an error.

2.13 Machine-readable output: `status_json` and `verify_json`

Two methods return the same answers as `status()` and `verify()`, serialised as one JSON object each. They are what the CLI's `-json` flag prints, so a Python caller and a shell pipeline read exactly the same fields.

```
import json
from ComplexGitSync.orchestre import ComplexGitSyncClient

client = ComplexGitSyncClient()
client.load_gts("/path/to/snapshot.gts")
```

```
payload = json.loads(client.status_json())
payload["cgitsync_branch"]          # the branch the tree is on
payload["profile"]                  # "user" or "dev" (client.registry.
    profile)
payload["summary"]["repos"]         # how many repositories it holds
payload["repositories"][0]["sync"]  # per repository, as the table shows
     it

chain = json.loads(client.verify_json("/path/to/CGSHOME"))
chain["status"]                     # "clean" or "findings"
```

`verify_json` keeps the report it built on `client.last_verify_report`, so a caller needing both the object and the verdict does not verify the chain twice — which under `repair=True` would mean repairing twice.

The shape of both objects lives in `json_render.py`, defined once for every command rather than per command, and carries a `schema_version`. The promise attached to it is **additive only**: new fields may appear; existing fields do not change meaning and do not vanish.

2.14 Environment observation and drift

The client exposes the same read-only Environment operations as the CLI:

```
from ComplexGitSync.cgs_format import CgsDocument
from ComplexGitSync.orchestre import ComplexGitSyncClient

client = ComplexGitSyncClient()
client.load_gts("/path/to/CGSHOME/.cgitsync/state/<hash>.gts")

observed = client.environment()
print(observed.architecture, observed.pixi_platform, observed.digest())

requirements = CgsDocument.from_toml("/path/to/project.cgs")
drift = client.check_environment(requirements)
if not drift.matches:
    print("missing", drift.missing)
    print("older", drift.older)
print("observed but undeclared", drift.undeclared)
```

`environment()` returns a secret-free, content-addressable `TreeEnvironment`. Credentials are booleans only; manifests are tree-relative paths and digests, never copied contents. When a source `.cgs` is available, `check_environment()` may be called without an argument. It returns data and installs nothing.

2.15 A memory as a repository: `memory_init`, `memory_clone`, `memory_push`

```
from ComplexGitSync.orchestre import ComplexGitSyncClient

client = ComplexGitSyncClient()
client.load_gts("/path/to/CGSHOME/.cgitsync/state/<hash>.gts")

proposal = client.memory_init("/path/to/CGSHOME")
proposal["entry"]        # the .cgs entry that mounts the memory
proposal["branch"]       # this project's branch of the memory
    repository
```

```
proposal["create_with"]  # the command that creates it -- never run for
    you

client.memory_push("/path/to/CGSHOME")                          # commit +
    push
client.memory_clone("/other/workspace", owner="flipoyo")     # onto a new
     machine
```

`memory_clone` needs no loaded project — that is the case it exists for — so give it an `owner` and a `branch` when nothing is loaded, or a `remote` address directly for a memory that does not live where the convention says. It raises `GitSyncError` rather than writing over a memory already present, or when the branch it wants has never been pushed.

2.15.1 A developer's tree with no memory: `memory_setup`

A tree holding any private repository is a developer's (`client.registry.profile` is `TreeProfile.DEV`); one holding none is a user's. A developer's memory is meant to be synced, so when the `.cgs` declares none these three methods offer one. None of them prompts: asking is the CLI's job.

```
proposal = client.memory_setup_proposal()   # None for a user tree, or a
     declared memory
proposal["owner"]         # guessed: most frequent private owner, else
    the root's, else None
proposal["create_with"]  # "gh repo create you/.memory --private"
proposal["warning"]       # the plain-words warning, naming the fix

result = client.memory_setup(provider="github", owner="you", name=".
    memory")
result["done"]            # ["create", "declare", "adopt"] on success
result["failed"]          # None, or the step that stopped it

client.memory_setup_decline()   # asked once: remembered in .cgitsync/
```

`memory_setup` runs `repo_create`, `add_memory_repo_cgs` and `memory_adopt` in that order and stops at the first that fails; it raises `GitSyncError` only when there is nothing to set up or no owner is known. A command that records a State in such a tree still records it, sets `client.memory_setup_due`, and emits a `MemorySetupWarning` (`orchestre.memory_setup`) — a warning, never a refusal.

Calling `memory_push` by hand, as above, is for settling the memory on its own. *READY Methods in the Client API*, earlier in this chapter, covers the more common case: `push`, `tag`, and `freeze_release` each call it automatically, first, once a memory is mounted and adopted.

2.16 Ledger Integrity: `verify`

`verify(cgshome, *, repair=False)` checks the hash-chained `.cgitsync/lgr` ledger under `cgshome` for tamper-evidence. The CLI's `verify check` calls it with `repair=False`, and `verify repair` with `repair=True`:

```
from ComplexGitSync.orchestre import ComplexGitSyncClient

client = ComplexGitSyncClient()
report = client.verify("/path/to/CGSHOME")

if report.is_clean:
    print("ledger is clean")
else:
```

```
    for seq, finding, detail in report.findings:
        print(seq, finding.name, detail)

# Repair a stale HEAD cache (never rewrites or deletes an entry)
client.verify("/path/to/CGSHOME", repair=True)
```

Returns an `integrity.VerificationReport`. Read `report.state` first: it is one of four `HistoryState` members — `VERIFIED`, `NO_HISTORY`, `LEGACY`, `CORRUPT` — and `report.is_verified` is `True` only for the first.

`report.is_clean` is a narrower question: it means "no findings were recorded", which an empty ledger also satisfies. Use it to ask whether anything is *wrong*; use `state` to ask what was actually established. `findings` holds `(seq, Finding, detail)` tuples.

Every command that writes a State now appends an entry to `.cgitsync/lgr/`, so a workspace used since that landed answers `VERIFIED`. A workspace whose only history predates it answers `LEGACY`: the single-file register is still read, and nothing writes it any more. `Finding` enumerates `BROKEN_LINK`, `BAD_ENTRY_HASH`, `SEQ_GAP`, `SEQ_DUPLICATE`, and `HEAD_STALE` for what this method actually checks (chain linkage and the `HEAD` cache); its remaining three members (`MISSING_STATE`, `ORPHAN_STATE`, `STATE_DIGEST_MISMATCH`) are reserved for a future store-level verification pass that cross-references entries against the actual `state/<hash>.gts` files on disk.

With `repair=True`, a stale `HEAD` cache is corrected in place. Ledger entries are never rewritten or deleted by this method, repair or not — a broken chain is reported, not silently healed.

2.17 Diagnosing and Repairing a Git Situation: `autofix`

`autofix(*, error=None, repo_name=None)` reads the error the tree's last command actually failed with, and runs whichever registered repair recognises it — the same command `cgitsync autofix` wraps (§5.3.6 of the user guide):

```
from ComplexGitSync.orchestre import ComplexGitSyncClient

client = ComplexGitSyncClient()
client.load("/path/to/CGSHOME/snapshot.gts")

# error=None reads .cgitsync/logs/*.log's most recent failing command
outcome = client.autofix()

# Or hand it an error directly -- useful from a script that already
# caught one, or in a test
outcome = client.autofix(
    error="fatal: Not possible to fast-forward, aborting.",
    repo_name=".memory",
)

print(outcome.repaired, outcome.detail)
```

Returns an `autofix.RepairOutcome`: `repaired` (bool) and `detail` (str, one line naming what changed or why nothing did). `repaired=False` is not a failure — it also covers the case where the situation turned out not to need this repair at all (e.g. a merge that applied cleanly on its own).

Raises `autofix.NoMatchingRepairError` (mapped to the CLI's `EXIT_REFUSED`, never a traceback, when reached through `cgitsync autofix`) when no failing command is found to read, or when nothing registered recognises the error — `autofix` refuses rather than guessing. It can also raise the ordinary `errors.GitSyncError` a repair itself raises, for example when the two sides of a divergence are not disjoint enough to reconcile safely without a person's judgement.

`autofix` is deliberately not one growing class: each situation it recognises lives in its own `ComplexGitSync.autofix.repair_*` module (today, `repair_divergent_user` — a private repository whose branch diverged because two machines wrote to it independently, where the content itself, like `.memory`'s hash-chained ledger, carries an ordering invariant a plain `merge` cannot see). Adding a new situation this method can handle means adding a new module and registering it, never editing an existing one to grow another branch.

Chapter 3

Architecture Overview

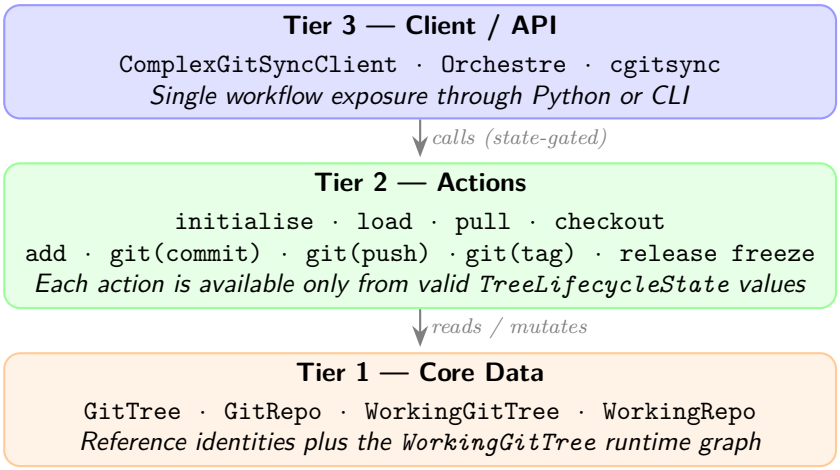
This chapter describes how `ComplexGitSync` is structured around three explicit tiers and explains the interaction between its classes. Related domain classes are grouped in modules according to the responsibility boundaries below.

3.1 Three-Tier Architecture

`ComplexGitSync` is organised into three tiers. Dependencies only flow **downward**: the Client calls Actions; Actions read and mutate Core Data; Core Data has no upward dependency.

Figure 3.1 shows the overall layout.

Figure 3.1: Three-tier architecture: Client/API exposes state-gated methods; Actions operate on Core Data; Core Data separates reference identities from the `WorkingGitTree` parent–leaf runtime graph.



3.2 Architectural Positioning

`ComplexGitSync` is positioned as a deterministic synchronisation layer *on top of Git*, not as a replacement for Git itself. It combines two distinct graph scopes:

- **Git DAG**: per-repository commit history and native remote semantics.
- **GitTree DAG**: workspace-level dependency graph coordinating parent/root/leaf propagation and ordering.

The system therefore differs from:

- plain Git usage (single repository scope),
- monorepo layouts (single storage boundary),
- submodule-only management (linking primitive without lifecycle orchestration),
- generic distributed sync engines (not constrained by Git-native ‘.gts‘/‘.lgr‘ deterministic contracts).

Figure 3.2 summarizes this positioning.

Figure 3.2: Architectural positioning matrix across Git scope and workspace coordination guarantees.

Approach	Primary scope	What it does not guarantee
Git (single repo)	Commit DAG per repository	Multi-repository propagation and coordinated lifecycle gating
Monorepo	One repository, one commit DAG	Independent repository identities and per-repo remote ownership
Submodule management	Parent repo references child revisions	Deterministic tree-wide mutation workflow and snapshot contracts
ComplexGitSync	Git DAG + GitTree DAG	Not a credential broker or remote policy authority

`release freeze` materializes deterministic checkpoints as `.gts` snapshots (schema + SHA-256 hash), and carries the project-scope State in the root repository's annotated tag, so `release list` and `release load` find a release from any workspace; `.lgr` assigns stable local ids and stores append-only ledger events for replayable local evolution.

Cross-declared nested repositories may initially resemble a local tangle at declaration time; runtime expansion normalizes this into a DAG via `fix_circularities`.

3.3 Tier 1 — Core Data

The reference model is centred on **`GitTree`**, which owns canonical **`GitRepo`** identities. Its runtime counterpart, **`WorkingGitTree`**, adds the parent–leaf graph of mutable **`WorkingRepo`** nodes. A project consists of one *root* repository that may nest *parent* repositories and terminal *leaf* repositories.

Figure 3.3 shows a representative project tree.

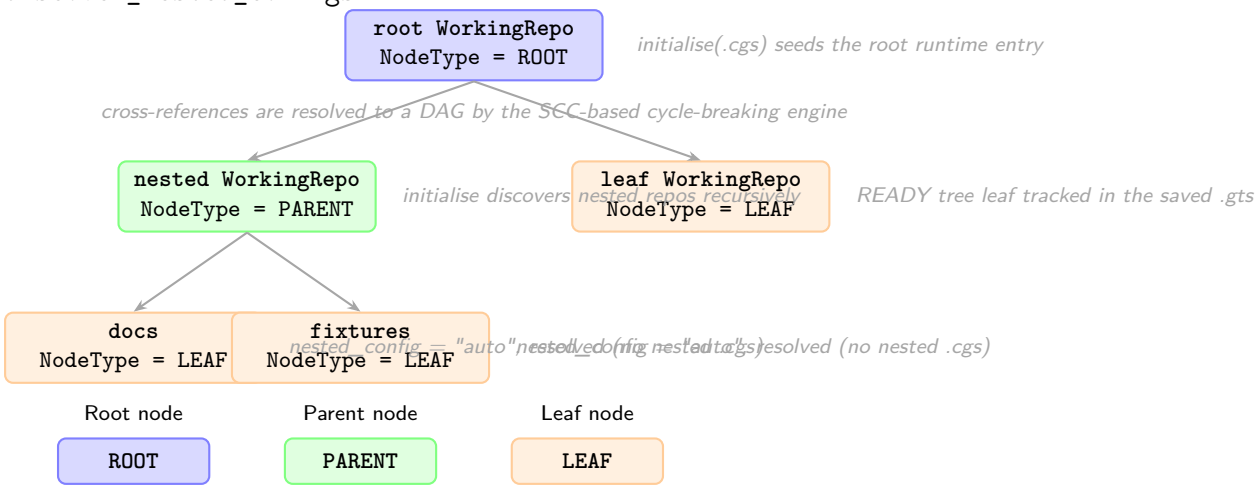
Registry entries in `WorkingGitTree` are indexed by a colon-separated repo ID derived from their position in the tree (e.g. `root:deps/child-repo:docs`). Cross-referenced parents therefore create a tangle-like declaration layer, but the expanded runtime graph remains a DAG after the cycle-breaking engine has run.

Cycle Breaking Engine

`fix_circularities` is a two-phase engine that converts a potentially cyclic dependency graph into a valid DAG:

1. **SCC detection (Tarjan's algorithm).** A directed dependency graph is built from the registry (edges: parent absolute path → child absolute path). `find_strongly_connected_components` identifies groups of paths that form a cycle. For each non-trivial SCC, an *Anchor* path is selected using three heuristics: (1) most external incoming edges, (2) closest to project root, (3) smallest SHA-256 hash. Back-edge entries — registry entries resolving to the anchor's path but parented inside a non-anchor SCC node — are flagged `is_external_reference = True` and removed.

Figure 3.3: `WorkingGitTree` is a parent–leaf graph of `WorkingRepo` nodes. Root and parent nodes own nested `.cgs` files discovered during `initialise`, `bootstrap` or `discover_nested_configs`.



2. **Hash-compatibility deduplication.** Residual path-duplicate entries are removed when their synchronisation state is compatible with the canonical entry.

The clone engine uses a **sync stack** (set of absolute paths already entered into the clone pipeline) to exclude entries whose path is already being processed, providing defence-in-depth against any residual back-references that escape the registry cleanup phase.

`topological_sort(registry)` returns entries in parent-first order (Kahn's BFS algorithm) for safe sequential clone/pull operations. Runtime `pull` uses the same three-level direction: `ROOT -> PARENT -> LEAF`; every repository — root, parent, and leaf alike — is a plain independent clone and is pulled in that same parent-first order, rather than through any submodule linkage.

Figure 3.6 shows the class overview across all three tiers, `GitTree`/`GitRepo` among them; the full current module-by-module Ring-level dependency graph lives in `docs/DevGuide/architecture.md`.

3.4 Tier 2 — Actions

Actions are operations that read or mutate the Core tier. **Every action is gated by the current `TreeLifecycleState`**: actions that require a `READY` tree will refuse to execute and log a gating-refusal event if the tree is not ready.

Figure 3.7 shows the full lifecycle state machine. The public CLI presents this lifecycle through `initialise`, `pull`, and the READY-state sync commands; lower-level load/expand/-validate stages are internal steps of those workflows.

Action	Minimum state	Produces
`initialise(.cgs)`	none	clone tree + **READY**
`initialise(.gts)`	none	loaded/restored **READY**
`pull(.cgs|.gts)`	READY	resynchronised **READY**
`checkout(.gts)`	READY	**READY**
`add`	**READY only**	READY
`commit <msg>`	**READY only**	READY
`push`	**READY only**	READY + updated hash
`tag <name>`	**READY only**	READY + updated tag
`release freeze`	**READY only**	next `.gts` id
`release load`	**READY only**	READY, detached at the release

For `.cgs` refs, repositories can be declared by branch or by tag. Format validation is static and does not resolve either reference remotely. The runtime layer selects the declared target (tag takes precedence when both are present) and checks its availability through `GitRunner`.

The authoring boundary is explicitly `PARSE -> NORMALIZE -> VALIDATE`. TOML parsing produces authoring data; normalization expands `provider:owner/repository` identifiers and fills `main` branch defaults, `ssh`, automatic nested discovery, and relative paths. Validation receives only the complete canonical `CgsDocument`. Authoring syntax is therefore not the internal representation.

File and CLI authoring converge at this boundary. The CLI collects `-project` and repeatable `-repo` values, then calls the non-interactive `ComplexGitSyncClient.configure()` Python API. Python callers can use the same method directly. The facade delegates to `CgsDocument`; neither it nor the CLI owns repository grammar or normalization. Loading an equivalent `.cgs` file and using either definition path produce semantically identical canonical documents. `discover -write` runs the same offline pipeline and serializes the result without entering Git runtime.

The boundary is also responsible for the reverse projection. `CgsDocument.to_git_tree()` creates the reference model and `CgsDocument.from_git_tree()` converts a reference or working tree back to canonical configuration. `GitTree.to_cgs()` remains only as a convenience delegate; it does not assemble repository tables or format TOML. The minimal serializer then removes deterministically reconstructible defaults. After the generated TOML is parsed and normalized again, its canonical representation must equal the one before serialization. Byte equality is not part of this contract.

Repository placement is parent-relative. At the top level the parent path is the project root; during nested discovery it is the repository whose `.cgs` is being expanded. Thus `../sibling` resolves to an existing sibling directory. Discovery compares normalized absolute paths and keeps an existing registry entry instead of inserting a duplicate.

Placement and traversal are independent. `nested_config = auto` searches the repository root for exactly one `*.cgs` — finding zero resolves the repository as a normal leaf, finding more than one raises immediately — `disabled` stops discovery through that reference, and a relative `.cgs` path selects an exact file inside the repository, failing if that named file does not exist. Explicit paths may not escape the repository root.

The textual `provider:owner/repository` grammar belongs exclusively to `cgs_format.py`. Its `parse_repo_id()` function owns syntax validation and splitting. Normalization uses that function, and CLI repository arguments must reuse it rather than introduce another parser.

The complete `.cgs` format pipeline is deterministic and offline. Parsing, normalization, validation, and serialization do not probe repositories or execute Git. Remote existence and branch/tag availability are resolved only after the validated document enters the runtime layer, through explicit `GitRunner` operations.

Provider behavior remains in `git_repo.py`: the `GitProvider` enum, canonical provider set, known-host map, and `RepoAddress` URL composer operate on already-separated identity fields. GitHub, GitLab, and Codeberg map respectively to `github.com`, `gitlab.com`, and `codeberg.org`. `custom` has no known-host entry and requires an explicit `gitprovider_url`; no fallback host is guessed.

Before any mutating workflow step (`commit`, `push`, `tag`, or `release freeze`), the action layer now runs a workspace preflight validator. It checks dirty worktrees, detached HEADs, missing remotes, branch divergence, unresolved merges, and stale recorded `commit_sha` values. Diagnostics are severity-based: warnings report actionable-but-allowed states, while blocking errors stop the operation. `tag` still requires a clean worktree and a non-existing tag.

Formal `.gts` snapshot specification (T31)

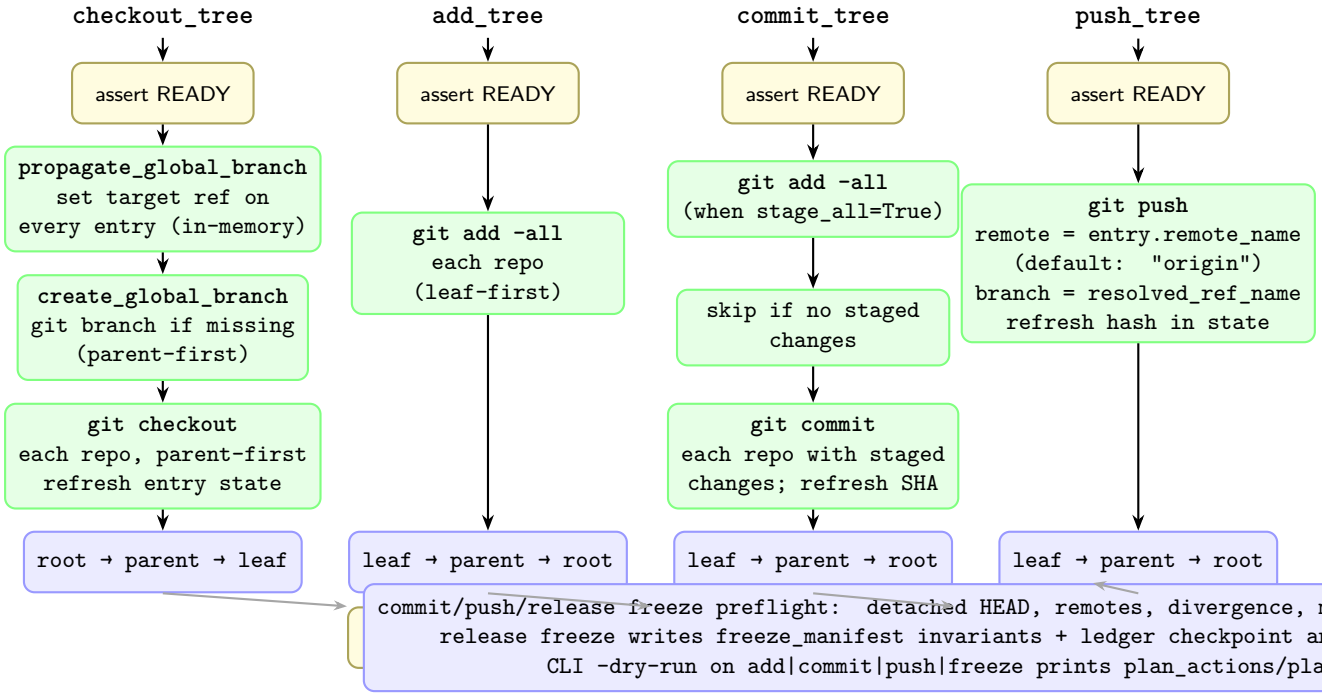
`.gts` now carries explicit schema and deterministic hash metadata in the `[document]` table: `schema_version = "1.1"`, `hash_algorithm = "sha256"`, and `snapshot_hash = "<64-hex>"`.

The canonical digest is computed from a deterministic payload made of `project`, `tree_state`, and sorted `repo_state` records, excluding volatile generation metadata (`generated_at` and `command_origin`). Non-root entries require `parent_absolute_path`; READY repositories require `commit_sha`. This makes `.gts` a stable checkpoint primitive for repeatable workspace state comparison.

Each `repo_state` record also carries the repository's provider identity: `gitprovider`, `group_name`, and `gitprovider_url` (mirroring the same fields a `.cgs` repo entry carries — §5 "Local Authoring Spec"), plus `access_protocol`. `gitprovider/group_name/ gitprovider_url` are part of the canonical hashed payload: they say *which repository this is*, so two snapshots pointing at different hosts are not the same state. `access_protocol` is deliberately excluded from the hash — it records only how the repository was last reached (ssh or https), not what it is, and a tree cloned entirely over ssh must hash identically to the same tree cloned over https. A `.gts` written before this schema addition has no `gitprovider` field at all; every command that would otherwise derive a clone URL from it (only the missing-repository restore path — `push`/`pull` instead convert the repository's actual existing remote in place, never rebuilding it from identity) refuses with an explicit error instead of silently defaulting to GitHub.

Figure 3.4 shows the internal sequence of the four tree-wide synchronisation actions implemented in the `operations/` package.

Figure 3.4: Synchronisation operations: `pull`/`checkout_tree` process `ROOT -> PARENT -> LEAF`; `add_tree`, `commit_tree`, and `push_tree` iterate `LEAF -> PARENT -> ROOT` so that inner repos are synchronized before their parents.



3.5 Tier 3 — Client / API

`ComplexGitSyncClient` is the single public facade. It:

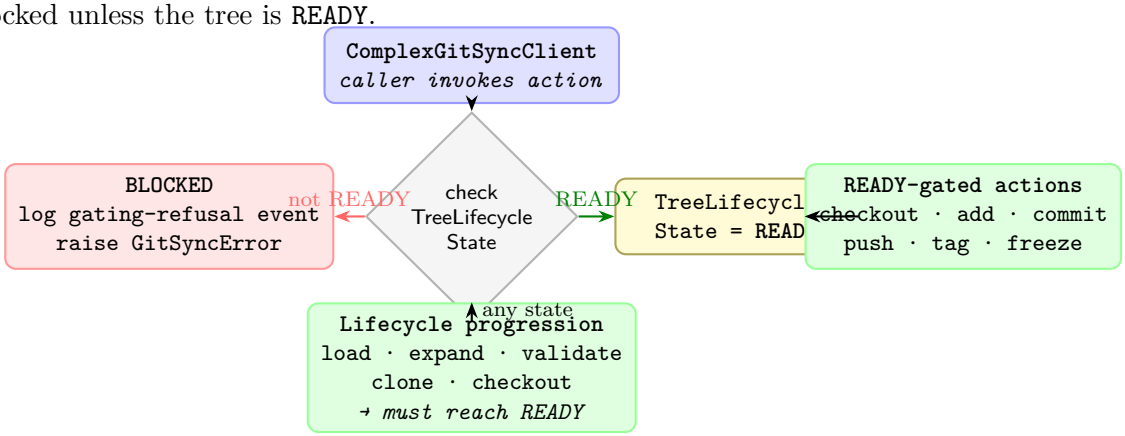
- holds references to `Orchestre`, `GitRunner`, `RuntimeStateStore`, and the live `WorkingGitTree`;
- computes the current `TreeLifecycleState` on demand and gates every action against it;

- emits structured log events for every state transition, action start/end, fallback decision, and `.gts` write/load.

`Orchestre` is the coordination layer between the Client and the tree models. The CLI (`cli.py`) collects terminal arguments or interactive prompt values, then delegates to the reusable Client API. The Client delegates format semantics to `cgs_format.py`; runtime operations remain separate.

Figure 3.5 shows how the Client gates action access based on the live tree state.

Figure 3.5: Client state-gating: the Client reads `TreeLifecycleState` before delegating to an action. READY-gated actions (`checkout`, `add`, `commit`, `push`, `tag`, `release freeze`) are blocked unless the tree is `READY`.



3.6 Module Layout by Tier

Listing 3.1: Source modules grouped by architectural tier

```
src/ComplexGitSync/
  # --- Tier 1: Core Data (centred on GitTree / GitRepo) ---
  git_repo.py                    GitRepo, WorkingRepo, RepoAddress,
                                 RepoNode, enums
  git_tree.py                    GitTree, WorkingGitTree,
                                 ProjectTreeState, tree utilities,
                                 sync_gitignore (.gitignore maintenance)

  # --- Tier 2: Actions ---
  operations/                    BranchOperation, CommitOperation,
     MergeOperation,
                                 PushOperation, RestartOperation,
                                    RemovalOperation,
                                 Preflight, RepoOutcome
  orchestre/                     discovery and render collaborators
                                 (DiscoveryCommands, Reporting)

  # --- Tier 3: Client / API ---
  orchestre/                     ComplexGitSyncClient (facade) and the
     collaborators
                                 it delegates to; Orchestre (the tree
                                    holder)
  cli.py                         build_parser / main
  orchestre/                     RuntimeStateStore, CommandRunLogger
  master.py                      MasterConfig (workspace-local Git
     identity
```

```
                                for ComplexGitSync-authored commits)

  # --- Document layer (cross-cutting) ---
  config_document.py            ConfigDocument (format-neutral base)
  cgs_format.py                 CgsDocument (.cgs parse/normalize/
     validate/serialize)
  gts_document.py               GtsDocument (runtime state snapshot)
  errors.py                     exception hierarchy
```

3.7 Document Hierarchy

Figure 3.8 shows the document class hierarchy.

All persistent documents share the `ConfigDocument` base class, which provides format-agnostic serialisation (`to_toml`, `to_json`, `to_yaml`) and factory methods (`from_toml`, `from_json`, `from_yaml`). Subclasses add document-specific validation and convenience properties. The base is defined in `config_document.py`; `cgs_format.py` owns the complete `.cgs`-specific boundary, while the `orchestre/` package consumes validated documents and retains runtime/orchestration responsibilities.

- **CgsDocument** (`.cgs`) — the authoring spec; describes the static project topology. *Never* a runtime snapshot.
- **GtsDocument** (`.gts`) — a generated snapshot capturing the exact state of the tree including commit SHAs and absolute paths. *Never* hand-edited.
- **Local Git Register** (`.lgr`) — the project-local register that assigns one stable local id to each generated `.gts` and tracks the current snapshot pointer.

3.8 Registry Model

Figure 3.9 shows the registry model.

`WorkingGitTree` is the authoritative in-memory graph. It maps *repo IDs* (colon-separated path strings such as `root:deps/child-repo`) to `WorkingRepo` instances. `WorkingRepo` holds both the static identity declared in `.cgs` and the dynamic state updated during operations.

The `RegistryTranslator` class in `registry.py` translates document objects into a live working tree:

```
# Load from authoring spec
registry = RegistryTranslator.from_cgs_document(document, config_path)

# Load from snapshot
registry = RegistryTranslator.from_gts_document(snapshot_document)

# Serialise to snapshot
gts_doc = RegistryTranslator.to_gts_document(
    registry, command_origin="clone", source_cgs_path=path
)
```

Figure 3.6 is deliberately kept at Tier granularity rather than diagramming all of `src/ComplexGitSync/`'s current modules — see `docs/DevGuide/architecture.md` for the dev-facing companion that carries the full per-module Ring-level breakdown and dependency graph this chapter does not attempt to reproduce.

Figure 3.6: Class overview by tier. Headline classes for each of the three tiers, plus the cross-cutting Document layer and the `cli/` adapter package that the Tier model never described. Every module in `src/ComplexGitSync/` is named here, either as a class box or in a tier's module list; see `docs/DevGuide/architecture.md` for the full current module-by-module Ring-level dependency graph.

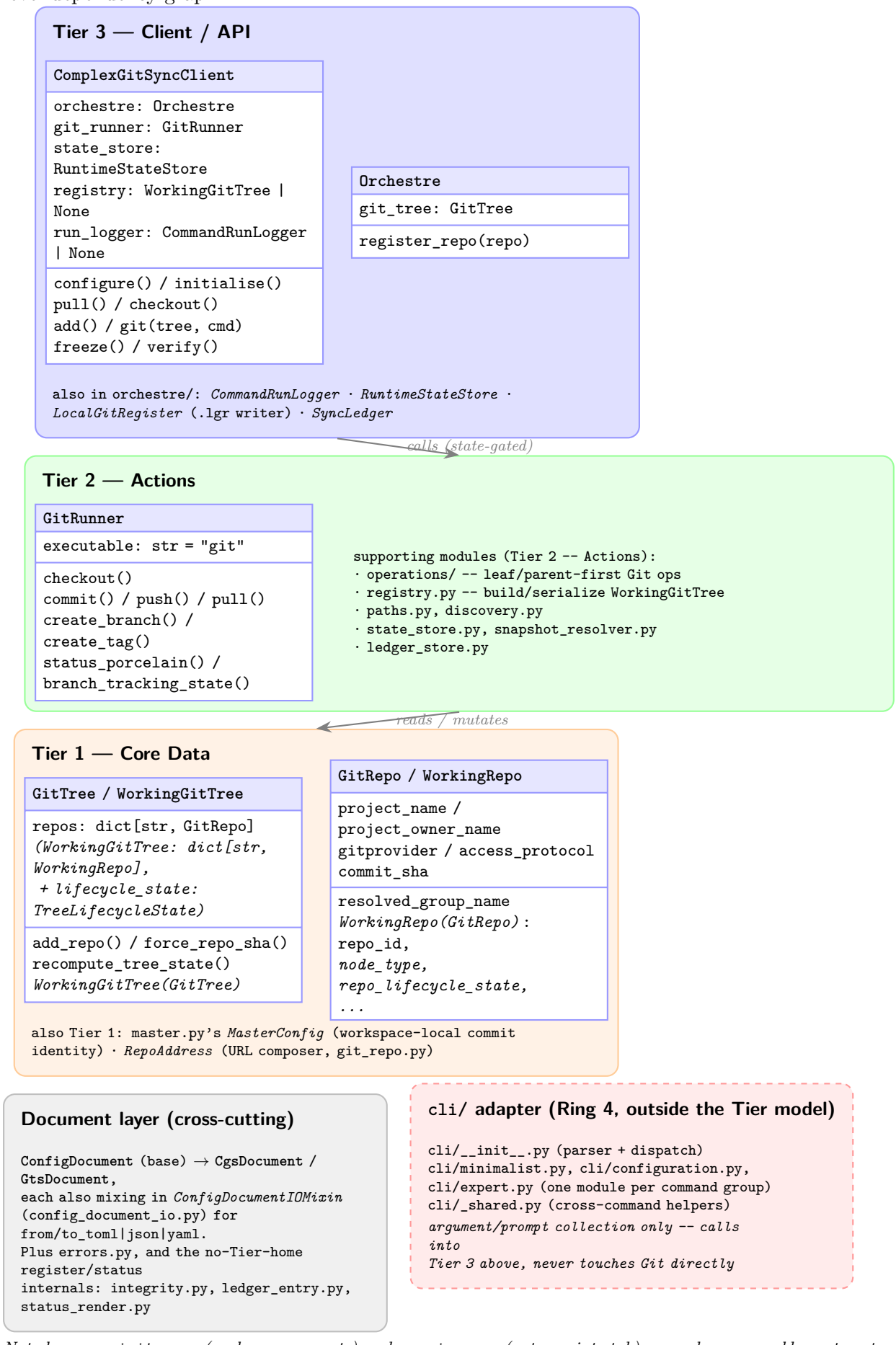


Figure 3.7: GitTree lifecycle state machine. The simplified user-facing lifecycle is `initialise(.cgs)` → clone → `READY`; `initialise(.gts)` → restore → `READY`; `pull(.cgs|.gts)` → resync → `READY`. Synchronized Git operations (`checkout`, `add`, `git("commit")`, `git("push")`, `git("tag")`, `release freeze`) are gated on `READY`.

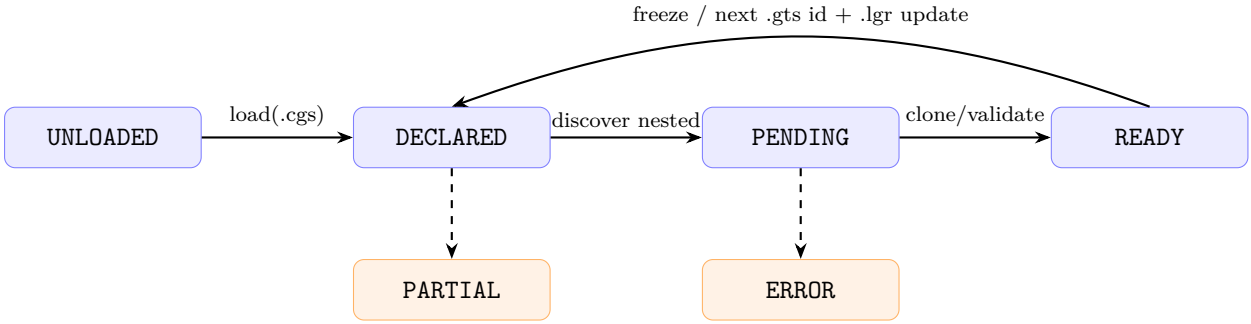


Figure 3.8: Document class hierarchy. `ConfigDocument` (`config_document.py`) is the abstract, I/O-free base; `ConfigDocumentIOMixin` (`config_document_io.py`) supplies the file-based `from/to_toml|json|yaml` methods. Each concrete document type combines both via ordinary multiple inheritance.

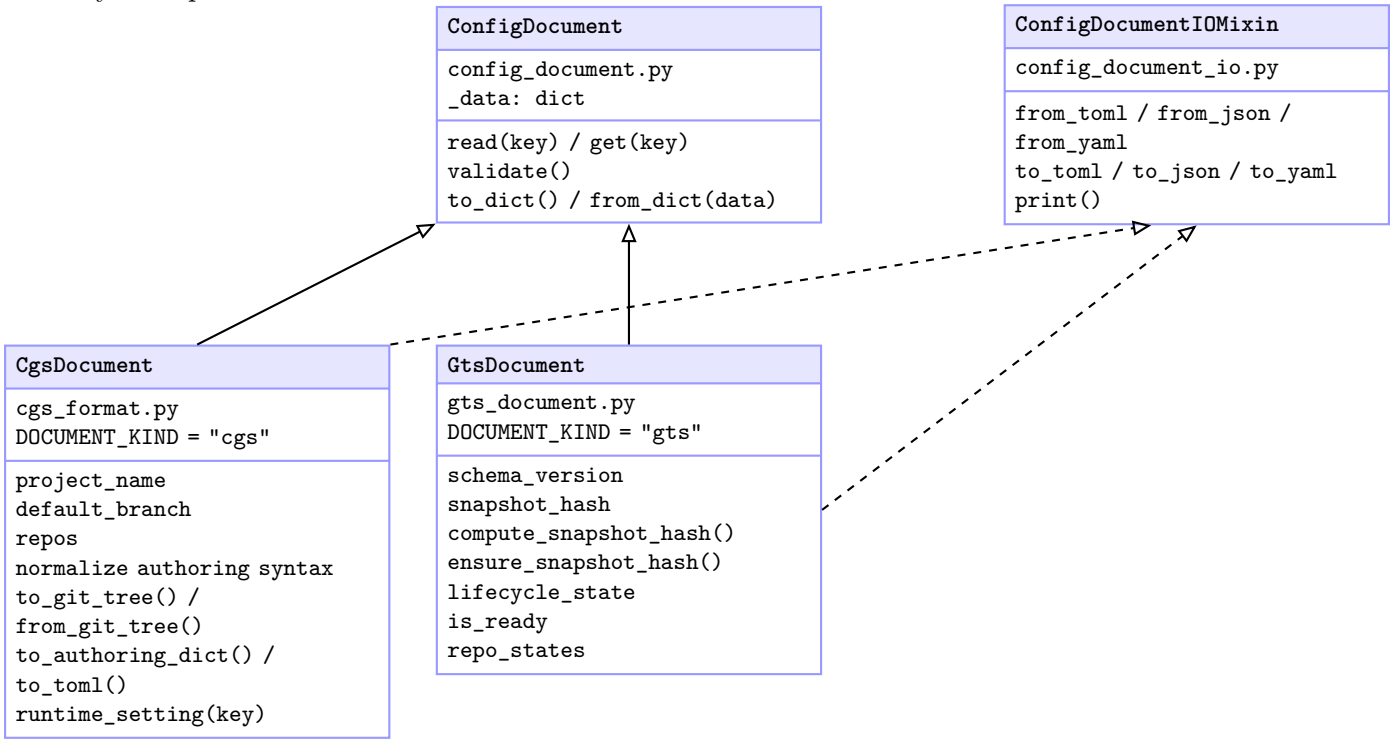
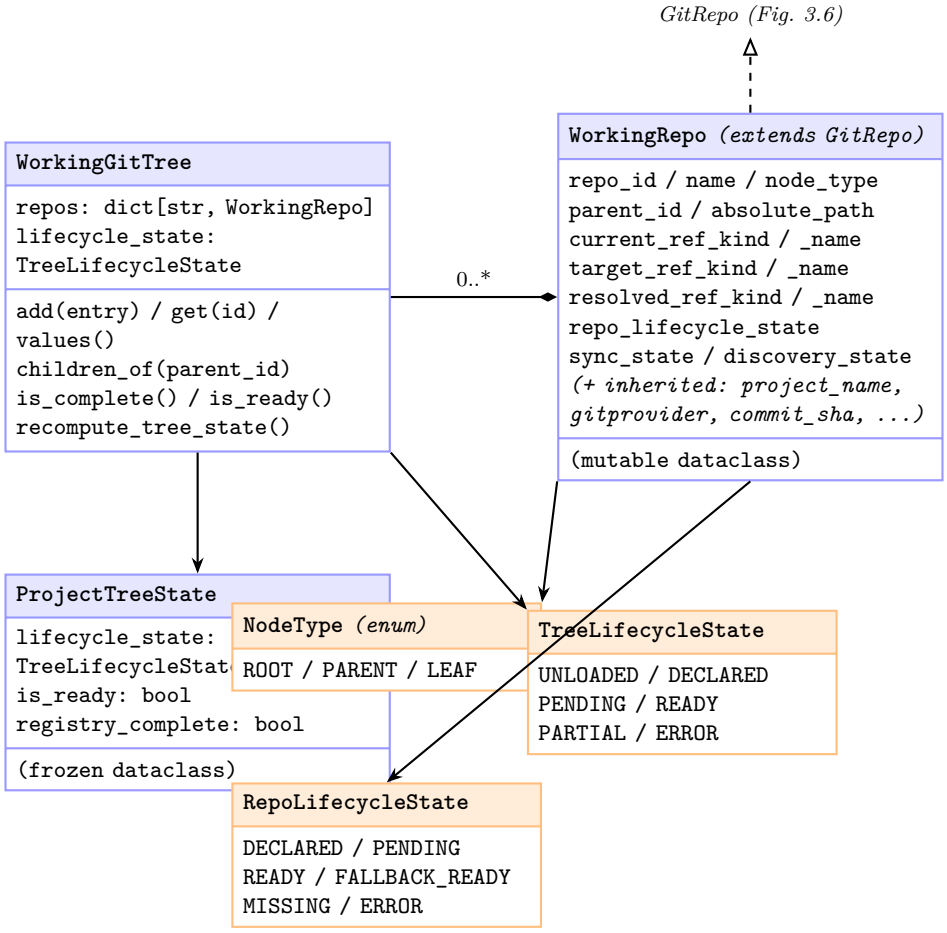


Figure 3.9: Registry model. `WorkingGitTree` is the authoritative in-memory graph; `WorkingRepo` *extends* `GitRepo` (Figure 3.6) rather than duplicating its identity fields, adding only the mutable tree-position and synchronisation state below.



Chapter 4

Writing ċgs Files: The Complete Authoring Guide

This section is the definitive reference for authoring `.cgs` (ComplexGitSync specification) files. It covers both the simplified shorthand syntax suitable for most use cases and the full advanced syntax for complex topologies, with practical guidelines, complete examples, and common pitfalls.

4.0.1 What is a ċgs File?

A `.cgs` file is a **TOML-formatted** specification that describes a tree of Git repositories to be managed as a single logical unit. It is the only hand-authored input in ComplexGitSync; all other documents (`.gts`, `.lgr`) are generated automatically at runtime.

Key properties:

- **Offline and deterministic:** Parsing, normalization, and validation require no network access. The same file always produces the same internal representation.
- **Never modified at runtime:** `.cgs` files are read-only during execution. Snapshots (`.gts`) capture workspace state; they do not update the specification.
- **Two equivalent forms:** A concise shorthand syntax for simple cases, and a verbose inline-table syntax for advanced overrides. Both normalize to the same canonical internal document.
- **Validation is strict:** Unknown fields, malformed identifiers, duplicates, and missing required keys are rejected before any Git operations begin.

4.0.2 Minimal ċgs: The Simplified Form

For the vast majority of projects, a `.cgs` file needs only two top-level keys: `project` (a string) and `repos` (a list of repository identifiers).

Basic Structure

```
project = "my-project"

repos = [
    "github:owner/repo1",
    "gitlab:group/repo2",
    "codeberg:user/repo3",
]
```

Repository Identifier Syntax

Each repository identifier follows the pattern:

```
provider:owner/repository
```

provider One of the supported Git providers: `github`, `gitlab`, `codeberg`, or `custom`.

owner The owner (user or organization/group) on the provider.

repository The repository name itself.

Nested namespace paths (e.g., GitLab subgroups) are supported. The final path segment is the repository name; all earlier segments join into `project_owner_name`:

```
repos = [
    "gitlab:my-group/my-subgroup/my-repo",  % 3-level namespace
    "github:my-org/my-repo",                % 2-level namespace
]
```

Valid providers and their default hosts:

Provider	Host	Remotes Generated
`github`	`github.com`	SSH and HTTPS
`gitlab`	`gitlab.com`	SSH and HTTPS
`codeberg`	`codeberg.org`	SSH and HTTPS
`custom`	explicit URL required	SSH and HTTPS

4.0.3 Normalization: What the Simplified Form Expands To

The simplified shorthand is automatically **normalized** into a complete canonical document. Every omitted field receives a deterministic default value.

Default Values Applied During Normalization

Field	Default	Applied To
`format_version`	`"1.0"`	document table
`default_branch`	`"main"`	project and each repo
`fallback_branch`	`"main"`	each repo
`access_protocol`	`"ssh"`	each repo
`nested_config`	`"auto"`	each repo
`relative_path`	repo name	each repo

Normalization Example

The following simplified input:

```
project = "demo"

repos = [
    "github:acme/core",
    "gitlab:acme/utils",
]
```

Normalizes to this canonical form (conceptually):

```
[document]
format_version = "1.0"

[project]
name = "demo"
default_branch = "main"

[[repos]]
gitprovider = "github"
project_owner_name = "acme"
project_name = "core"
repo_name = "core"
default_branch = "main"
fallback_branch = "main"
access_protocol = "ssh"
nested_config = "auto"
relative_path = "core"

[[repos]]
gitprovider = "gitlab"
project_owner_name = "acme"
project_name = "utils"
repo_name = "utils"
default_branch = "main"
fallback_branch = "main"
access_protocol = "ssh"
nested_config = "auto"
relative_path = "utils"
```

Important: The project's `default_branch` propagates to all repos unless explicitly overridden per-repo.

4.0.4 Inline Table Syntax: Overriding Defaults

When a repository needs non-default values for any field, use an **inline table** instead of a plain string. Only the fields that differ from defaults need to be specified.

Common Override Scenarios

```
project = "demo"

repos = [
    % Non-default branch
    { repository = "github:acme/core", default_branch = "develop" },

    % Different default and fallback branches
    { repository = "gitlab:acme/libs",
      default_branch = "next",
      fallback_branch = "main" },

    % Custom placement path
    { repository = "github:acme/docs",
      relative_path = "documentation" },

    % HTTPS access instead of SSH
    { repository = "codeberg:org/tool",
      access_protocol = "https" },
```

```
    % Disable nested config discovery
    { repository = "github:acme/legacy",
      nested_config = "disabled" },

    % Custom provider with explicit URL
    { repository = "custom:team/internal",
      gitprovider_url = "https://git.internal.company.com" },
]
```

Combined Overrides

Multiple overrides can be combined in a single inline table:

```
repos = [
    { repository = "github:acme/web",
      relative_path = "apps/web",
      default_branch = "production",
      fallback_branch = "staging",
      access_protocol = "https",
      nested_config = "disabled" },
]
```

4.0.5 Advanced Table Syntax: Legacy Form

The legacy advanced form uses explicit TOML tables. Fully supported but not recommended for new files.

```
[project]
name = "demo"
default_branch = "main"

[[repos]]
repository = "github:acme/core"

[[repos]]
repository = "gitlab:acme/utils"
default_branch = "develop"
relative_path = "lib/utils"
```

When to use advanced tables:

- Project-wide settings in the `[project]` table
- When you prefer visual separation between entries
- For compatibility with old ComplexGitSync versions

Note: The `[project]` table can also be inline: `project = { name = "demo", default_branch = "main" }`.

4.0.6 Repository Placement and Path Resolution

relative_path: Where Each Repository Lives

The `relative_path` field determines where a repository is cloned relative to its parent. Default: the repository name.

```
% Default: repo cloned at <parent>/repo_name
repos = ["github:acme/core"]

% Explicit: repo cloned at <parent>/apps/core
repos = [
    { repository = "github:acme/core", relative_path = "apps/core" }
]
```

Project Root Auto-Mount Rule

If exactly one repository name matches the project name, its `relative_path` is automatically set to `"."`:

```
project = "CGSil1"

repos = [
    "gitlab:CGS_test/CGSil1",  % Auto-mounted at "."
    "gitlab:CGS_test/CGSil2",  % At "./CGSil2"
    "github:flipoyo/CGSih1",   % At "./CGSih1"
]
```

Equivalent to:

```
project = "CGSil1"
repos = [
    { repository = "gitlab:CGS_test/CGSil1", relative_path = "." },
    { repository = "gitlab:CGS_test/CGSil2", relative_path = "CGSil2" },
    { repository = "github:flipoyo/CGSih1", relative_path = "CGSih1" },
]
```

Cross-Repository References

A repository can reference another repository in the same tree using relative paths:

```
% In CGSil2.cgs (nested under CGSil2/):
repos = [
    { repository = "github:flipoyo/CGSih1",
      relative_path = "../CGSih1" }
]
```

Here, `../CGSih1` resolves from `<CGSHOME>/CGSil2/` to `<CGSHOME>/CGSih1`, referencing the sibling declared elsewhere. No `nested_config` override is needed: discovery's absolute-path dedup guard already skips re-registering `CGSih1` once it is known under its canonical entry, before `"auto"` ever gets a chance to reopen its `.cgs`.

4.0.7 Nested Configuration Discovery

The `nested_config` field controls whether ComplexGitSync looks for additional `.cgs` files inside each repository.

`auto` (default) Loads the sole root-level `*.cgs` file if one exists; finding none is a normal leaf, not an error; multiple matches are rejected.

`disabled` Does not inspect the repository for nested config.

a relative path A path ending in `.cgs` loads that exact file; if the file doesn't exist, this is treated as an error, since an explicit path asserts that a specific file must be there.

Example: Multi-Level Nested Topology

```
project = "parent-project"

repos = [
    "github:org/parent",
    { repository = "github:org/child-a", nested_config = "auto" },
    { repository = "github:org/child-b", nested_config = "disabled" },
    { repository = "github:org/child-c", nested_config = "config/prod.
       cgs" },
]
```

4.0.8 Access Protocols and Custom Providers

SSH vs HTTPS

Default: SSH. To use HTTPS:

```
repos = [
    { repository = "github:owner/repo", access_protocol = "https" },
]
```

Generated remotes:

- SSH: `git@github.com:owner/repo.git`
- HTTPS: `https://github.com/owner/repo.git`

Custom Git Providers

For providers not in the built-in list, use `custom` with explicit URL:

```
repos = [
    { repository = "custom:team/project",
      gitprovider_url = "https://git.internal.mycompany.com" },
]
```

Important: `gitprovider_url` is **required** for `custom`.

```
% INCORRECT - will fail validation
repos = ["custom:team/project"]

% CORRECT
repos = [
    { repository = "custom:team/project",
      gitprovider_url = "https://git.internal.company.com" }
]
```

4.0.9 Branches: default_branch and fallback_branch

Branch Resolution Order

When cloning or pulling, the target is the repository's `default_branch`, else the project's `default_branch`, else `main`. Only when that target is missing on the remote is the repository's `fallback_branch` cloned instead; it defaults to `main` (a private, writable repository: its own `default_branch`).

Practical Branch Patterns

```
% All repos use "main" (default)
project = "my-project"
repos = ["github:owner/repo1", "github:owner/repo2"]

% All repos default to "develop"
project = { name = "my-project", default_branch = "develop" }
repos = ["github:owner/repo1", "github:owner/repo2"]

% Per-repo overrides
project = "my-project"
repos = [
    "github:owner/core",                    % Uses default "main"
    { repository = "github:owner/feature", default_branch = "dev" },
    { repository = "github:owner/stable",
      default_branch = "release", fallback_branch = "main" },
]
```

4.0.10 Private Repositories

A tree holds two kinds of repository. **Project repos** are the work itself and follow the project's branch. **Private repos** are how the project is run — pipelines, agent instructions, house rules — and are shared with the author's other projects, so they must not follow it: creating a project's feature branch inside a repository other projects also mount pollutes them all.

Setting `private = true` keeps a repository off the project's branch. Adding `writable = true` makes it **private/local** — this project's own settings, on a branch named after the project, which `cgitsync` may write to. Without it the repository is **private/distant**: read-only, because it belongs to someone else.

```
repos = [
    "github:flipoyo/ComplexGitSync",
    { repository = "github:flipoyo/.agentSpec", default_branch = "main",
      nested_config = "auto", private = true },
    { repository = "github:flipoyo/.localSpec", default_branch = "
       MyProject",
      fallback_branch = "main", private = true, writable = true },
]
```

With that in place, `cgitsync branch create feature-x` creates `feature-x` in the project's own repositories and in neither mount; `cgitsync checkout` leaves both on their declared branch; and `cgitsync pull` still updates them, on that branch rather than the tree's.

Two limits are worth knowing. Pinning governs *branch* propagation only: restoring a frozen release with `checkout <tag> -ref-kind tag` puts every repository back on the commit the snapshot records, private or not, so a release stays reproducible. And a repository declared in a *nested* ċgs is private by that file, which lives in the repository that owns it — pinning it means publishing that change there, not in the parent project.

4.0.11 Complete Reference: All Recognized Fields

Top-Level Fields

Field	Type	Required	Default
`project`	string or table	yes	—
`repos`	list	yes	—
`document`	table	no	`{format_version = "1.0"}`

Project Table Fields

Field	Type	Required	Default
`name`	string	yes	—
`default_branch`	string	no	`"main"`

Repository Fields

Field	Type	Required	Default
`repository`	string	yes*	—
`gitprovider`	string	no	from `repository`
`project_owner_name`	string	no	from `repository`
`project_name`	string	no	from `repository`
`repo_name`	string	no	from `repository`
`default_branch`	string	no	from project or `"main"`
`fallback_branch`	string	no	`"main"`
`access_protocol`	string	no	`"ssh"`
`relative_path`	string	no	repo name (or "." if matches project)
`nested_config`	string	no	`"auto"`
`private`	boolean	no	`false`
`gitprovider_url`	string	no**	—

* Either `repository` (shorthand) or the combination of `gitprovider`, `project_owner_name`, `project_name` is required.

** `gitprovider_url` is required when `gitprovider = "custom"`.

4.0.12 Practical Examples

Example 1: Simple Monorepo-Style Project

```
project = "my-app"

repos = [
    "github:myorg/my-app",    % Root (auto-mounted at ".")
    "github:myorg/core",      % At ./core
    "github:myorg/ui",        % At ./ui
    "github:myorg/api",       % At ./api
]
```

Example 2: Mixed Providers with Custom Branches

```
project = "cross-provider"

repos = [
    "github:myorg/main-repo",
    { repository = "gitlab:mygroup/data", default_branch = "data-main"
       },
    { repository = "codeberg:user/tools", access_protocol = "https" },
]
```

Example 3: Complex Topology with Nested Configs

```
project = "big-system"

repos = [
    "github:org/root",
    { repository = "github:org/frontend",
      relative_path = "apps/frontend", nested_config = "auto" },
    { repository = "github:org/database",
      relative_path = "infra/db", nested_config = "disabled" },
    { repository = "github:thirdparty/lib",
      relative_path = "external/thirdparty-lib", nested_config = "
         disabled" },
]
```

Example 4: Real-World (cawaqsviz)

Based on `examples/cawaqsviz.cgs`:

```
project = { name = "CaWaQS-Viz", default_branch = "main" }

repos = [
    { repository = "gitlab:cawaqs/gviz/cawaqsviz",
      relative_path = ".", fallback_branch = "main" },
    { repository = "github:flipoyo/HydrologicalTwinAlphaSeries",
      relative_path = "external/HydrologicalTwinAlphaSeries",
      fallback_branch = "main" },
    { repository = "github:flipoyo/hydrological_twin",
      relative_path = "external/HydrologicalTwinAlphaSeries/docs/
         hydrological_twin",
      fallback_branch = "main" },
    { repository = "github:flipoyo/user_guide_CaWaQS-Viz",
      relative_path = "docs/CWV_user_guide",
      fallback_branch = "main" },
]
```

Four repositories on two levels. The third entry's path lies inside the second's, which is how a `.cgs` says that one repository holds another: every path is written from the project root, so nesting shows up as one path lying inside another, and the innermost holder wins. `HydrologicalTwinAlphaSeries` is therefore read back as a `PARENT` with `hydrological_twin` as its child.

No repository here has a `.cgs` of its own, but no `nested_config` override is written for that: the default `"auto"` already resolves a repository with zero nested `*.cgs` matches cleanly.

4.0.13 Authoring with CLI Tools

You do not need to write `.cgs` files by hand.

discover: Scan an Existing Checkout

```
$ pixi run cgitsync discover ~/work/myproject --write draft.cgs
$ pixi run cgitsync validate ~/work/myproject/draft.cgs
```

initialise: Direct CLI Authoring

```
$ pixi run cgitsync initialise \
    --project myproject \
    --repo github:owner/repo1 \
    --repo gitlab:group/repo2
```

Both use the same offline pipeline as direct TOML parsing.

4.0.14 Validation and Common Errors

Always validate before use:

```
$ pixi run cgitsync validate myproject.cgs
```

Validation Checks

- Presence of required keys (`project`, `repos`)
- Valid repository identifier syntax
- Known provider names or custom with URL
- `custom` providers have `gitprovider_url`
- Valid `nested_config` values
- Valid `access_protocol` values
- No duplicate repository declarations
- Non-empty strings for all text fields

Common Errors and Fixes

`project is required` Add `project = "name"`.

`repos is required` Add `repos = [...]`.

`expected 'provider:owner/repository'` Fix identifier syntax.

`gitprovider_url is required for custom provider` Add `gitprovider_url = "https://..."`.

`duplicate repository` Remove one or make `relative_path` distinct.

`nested_config must be auto, disabled, or a path` Use valid value. Paths must end in `.cgs`.

4.0.15 Templates and Starting Points

Template Files

`examples/template.cgs` Copy for normal authoring.

`examples/normalized_template.cgs` Reference for canonical form.

Quick Start Checklist

1. Copy `examples/template.cgs` to `<your-project>.cgs`
2. Edit `project` name
3. Add repositories to `repos` list
4. Override defaults only when necessary
5. Validate: `pixi run cgitsync validate <file>.cgs`
6. Initialize: `pixi run cgitsync initialise <file>.cgs`

4.0.16 Best Practices

- **Start simple:** Use plain strings until you need overrides.
- **Use auto-mount:** Name one repo after project for automatic root.
- **Validate early and often:** After every edit.
- **Keep comments:** Document non-default values.
- **Avoid advanced tables:** Inline tables are more maintainable.
- **Use `nested_config = "disabled"` judiciously:** The default `"auto"` already resolves cleanly whether or not a repo has its own `.cgs` — including cross-references and cycle back-references, which the discovery dedup guard already prevents from re-registering. Reserve `"disabled"` for the rarer case where a repo does carry a `.cgs` you explicitly want ComplexGitSync to never open.
- **Prefer SSH:** Works well for most providers.
- **Be explicit with custom:** Always include `gitprovider_url`.

4.0.17 Internal Round-Trip Guarantee

ComplexGitSync guarantees a semantic round-trip:

$$\texttt{.cgs} \rightarrow \texttt{CgsDocument} \rightarrow \texttt{GitTree} \rightarrow \texttt{CgsDocument} \rightarrow \texttt{.cgs}$$

Parsing and normalizing a generated `.cgs` reproduces the canonical document. Whitespace and comments need not match byte-for-byte, but semantics must.

Chapter 5

User Guide

This chapter is the authoritative reference for every user-facing command, configuration option, document format, and error condition exposed by `ComplexGitSync`.

5.1 CLI Overview

The single entry-point is the `cgitsync` command.

`cgitsync` is a Pixi task, not a globally installed executable. This project is developed and run exclusively through Pixi (`pixi install` once, per checkout) — there is no `pip install` that puts a bare `cgitsync` on `$PATH`. Every invocation shown in this chapter as `pixi run cgitsync ...` must be typed exactly that way; a bare `cgitsync ...` will not be found by the shell.

```
$ pixi run cgitsync <command> [options]
```

Running `cgitsync` without a command prints a help summary and exits with code 0.

5.1.1 Prerequisites

Three tools are needed before the first command: Git clones this checkout and the repositories in a tree; Pixi installs the locked Python environment and runs `cgitsync`; and repository creation needs the provider's own command line tool — `gh` for GitHub, `glab` for GitLab, or `tea` for Codeberg. Install only the provider tool you use.

The complete list depends on the tree and the work being done:

Dependency	When it is needed	Where its version or requirement comes from
Git	Every clone, inspection and synchronisation.	Its observed version is recorded with the State.
Pixi	Install this project's environment and run `cgitsync`.	`pixi.lock` pins the project environment; the observed Pixi version is recorded separately.
Python	Run `ComplexGitSync` inside the Pixi environment.	`pixi.toml` selects the interpreter; the observed interpreter is recorded.
`gh`, `glab`, or `tea`	`repo create`, according to the repository provider.	The provider tool reports its version and whether it is authenticated.
SSH client and agent	Repositories whose remote uses SSH.	Presence is observed. Keys, user names, and key paths are never recorded.
DVC or Git LFS	A tree that uses the corresponding data backend.	The tree declares its use; the observed tool version is recorded when used.
Compilers and system libraries	Native builds or software installed outside Pixi.	The tree declares these in its `.cgs [environment]` table.
`latexmk` and TeX	Developers rebuilding the manuals under `docs/`.	A developer requirement, not an end-user runtime dependency.

The machine must also match a platform supported by the tree's lock files. `pixi run cgitsync env show` reports the operating system, architecture, Pixi platform, libc, tool versions, credential facts, and manifests it observes. `pixi run cgitsync env check` compares that observation with the tree's declared requirements. Neither command installs anything.

`ComplexGitSync` stores no credentials of its own and has no private-repository authentication story: every git operation — `clone`, `pull`, `push`, and the remote lookups behind `checkout`'s branch/tag fallback — runs against whatever the ambient environment already has cached (`ssh-agent`, an HTTPS credential helper, a stored personal-access-token). When that ambient credential is missing or stale, the operation fails immediately with a normal, catchable error rather than sitting silently on a terminal or GUI credential prompt that will never arrive — every git subprocess `ComplexGitSync` runs sets `GIT_TERMINAL_PROMPT=0` and a no-op `GIT_ASKPASS` for exactly this reason. Fix the ambient credential (switch the remote to `ssh`, refresh the token) and re-run the command; nothing needs to be undone first.

5.2 Lifecycle Contract

This guide follows the simplified canonical lifecycle:

1. `initialise(.cgs)` → clone the dependencies of a checked-out root → `READY` *(nested install)*
   `initialise(.gts)` → the same, each repository at the commit the snapshot recorded → `READY`
   `bootstrap(.cgs|.gts)` → clone the whole tree, root included → `READY` *(standalone install)*
2. `pull(.cgs|.gts)` → resync an existing tree
3. `checkout(.gts)` / `add` / `commit` / `push` / `tag` → git operations on the tree

4. `release freeze` → commit, tag `<project>-<version>`, record the State; `release list` and `release load` find it and go back to it

The project-local `.lgr` register and sync ledger are both active (T24, T32). Every synchronisation operation is automatically recorded as an immutable DAG event in the `[[ledger]]` section of the `.lgr` file.

5.3 Commands

Every command answers `-help` with what it does, its options and examples. A group such as `cgitsync memory -help` lists each subcommand with its options, and `cgitsync help -all` prints every command and option on one page, ready for `grep`. `help` is a command of its own: `cgitsync help memory explore` is the same as `cgitsync memory explore -help`. The help is generated from the parser the command actually runs, so it does not drift from it.

Every command follows one grammar. A subcommand is a plain word (`branch close`, `verify repair`); anything that starts with `-` is an option and never a command of its own; a single-dash `-x` is only ever the short form of a `-option`. An old spelling such as `close-branch` is not run: `cgitsync` names its new form (`'close-branch' is now 'branch close'.`) and stops.

The CLI exposes three command groups once a `GitTree` is `READY`. The minimalist level is `initialise`, `bootstrap`, `release`, `status`, and `view-tree`. The expert level exposes the individual operations: `branch create`, `branch list`, `branch close`, `branch check`, `branch delete`, `checkout`, `validate`, `pull`, `pull -force`, `fetch`, `autofix`, `add`, `commit`, `push`, `tag`, `submodules report`, `submodules import`, `submodules init`, `verify check` and `verify repair`, plus `env show` and `env check`. The configuration level, `discover` and `repo`, authors a `.cgs` specification without requiring existing state. There is no command that deletes a workspace for you: to start a directory over, move it aside yourself. Redundant inspection aliases such as `print`, `view-operation`, and `validate-topology` are not part of the public CLI surface.

5.3.1 `initialise`

```
$ pixi run cgitsync initialise <spec.cgs> [--output-path DIR] [--commit-
    gitignore]
     [--git-user-name NAME] [--git-user-email EMAIL]
     [--force-protocol {ssh,https}]
$ pixi run cgitsync initialise <snapshot.gts>
$ pixi run cgitsync initialise --project NAME --repo PROVIDER:OWNER/REPO
     [--repo ...] [--output-path DIR]
```

The *nested install*. `initialise` builds the dependencies of a project whose root is already checked out at `CGSHOME`, with the running `cgitsync` inside it. It refuses — before touching the disk, naming `bootstrap` — when `CGSHOME` is not a Git checkout (a detached `HEAD` is fine) or the running installation is not inside it. A `.gts` source checks each dependency out at the commit the snapshot recorded, and is refused, naming every repository, when a remote no longer holds one. A `private`, `writable` entry clones the branch computed from the project's name, then its `fallback_branch`, then the remote's own active branch. Dispatches on file extension:

- `.cgs` source (clone mode): clones the workspace's dependencies. The optional `-output-path` controls `CGSPATH`; `CGSHOME` is derived as `CGSPATH/<project-name>`. When omitted, `CGSPATH` defaults to `../...` On success, ends in `READY`.
- `.gts` source (restore mode): clones the same dependencies and puts each on the commit the snapshot recorded, so the rebuilt tree carries the snapshot's State name. On success, ends in `READY`.

- Direct CLI authoring: requires `-project` and at least one repeatable `-repo`. It is mutually exclusive with a source file. The CLI collects values only and calls `ComplexGitSyncClient.configure()` (the `.cgs` writer, which has no command of its own); that reusable Python API delegates parsing, normalization, and validation to `cgs_format.py` and produces the same canonical `CgsDocument` as an equivalent `.cgs` file.

All initialization paths end in a `READY` tree or fail explicitly.

`initialise` re-clones your dependencies. Only the root repository at `CGSHOME` is kept as it is. Every repository below it whose directory already holds files is **deleted and cloned again**. The old `.git` goes with it, so nothing in that directory can be recovered once the run has passed that point.

Before deleting anything, `initialise` inspects each destination and stops the whole run when one of them holds work that exists nowhere else: uncommitted changes, commits that have not been pushed to the branch's upstream, or a branch with no upstream at all. It names every repository that blocked it, in one message, and deletes none of them. No flag overrides this. The message says what to do: commit and push, or move those directories aside yourself.

A destination that is not a Git checkout at all — what a clone interrupted halfway leaves behind — is still cleared without any flag. That is the case the deletion was written for, and it still works.

Before a `.cgs` source is confirmed ready, every repository with children (root, or any nested repository that itself has further nested children) is safely pulled parent-first and has its `.gitignore` updated with the relative path of each immediate child — nested repositories are plain independent clones rather than gitlinks, so without this a parent's `git status`/`git add` would otherwise see a child's working tree as ordinary untracked content. This is logged as the `GT-GITIGNORE` phase, after `GT-CLONE`. If the safe pull for one of these repositories fails, `initialise` raises immediately and nothing is written; run `pull -force` on that repository, then `initialise` again. By default nothing is staged, committed, or pushed by this step — it only writes the file and prints what changed. Pass `-commit-gitignore` to explicitly approve staging (`.gitignore` alone), committing (with a message listing exactly which children were added), and pushing each changed repository. The commit identity defaults to local git config; `-git-user-name`/`-git-user-email` override it and persist the override to `CGSHOME/.cgitsync/master.toml` via `MasterConfig`, so later invocations on the same workspace pick it up without repeating the flags.

The root's own `.gitignore` additionally always gains `.cgitsync/` (the generated runtime-state directory — snapshots, ledger, run logs, never part of the `.cgs`/`.gts` project spec; the run logs never leave the machine, are never pushed with a memory, and only the 200 most recent are kept) and its `<project-name>.lgr` register file, whether or not the tree has any nested repositories at all — that state is written under the root regardless of tree shape, so it is excluded the same idempotent way, via the same underlying write `sync_gitignore()` already uses for child mount points.

`-force-protocol {ssh,https}` is an expert option, typically used systematically in CI rather than for everyday development. It overrides `access_protocol` in memory for every repository this run clones — including ones discovered later from a nested `.cgs` living in a different, separately-cloned repository — so no `.cgs` file is ever read differently or written to. Forcing `https` is the usual CI case: it removes the dependency on an SSH key or agent being present on the runner, a dependency that otherwise differs between a `push` run and a `pull_request` run (GitHub Actions withholds repository secrets from a pull request opened from a fork). Omitted (the default), each repository follows whatever protocol its own `.cgs` entry declares, exactly as without the flag.

5.3.2 `bootstrap`

```
$ pixi run cgitsync bootstrap <spec.cgs|snapshot.gts> [project_name] [--
   cgs-path DIR]
    [--force-protocol {ssh,https}]
```

The *standalone install*: clones the full repository tree from a `.cgs` or `.gts` source (root included) into an isolated `CGSHOME`, for running `cgitsync` from its own standalone clone rather than nested inside the project tree it manages — one `ComplexGitSync` clone reused across many projects. It delegates to the same `ComplexGitSyncClient.clone_cgs` pipeline, which picks each repository's branch by the fallback chain and discovers nested `.cgs` files after the parents are cloned, but differs from `initialise` in two ways:

- The workspace is named after the project: the `[project] name` the `.cgs` or `.gts` declares. The optional `project_name` argument replaces it, for instance to keep two copies of one project apart.
- `CGSHOME` does not default to `../..` relative to the current directory (that default assumes `ComplexGitSync` is cloned inside the tree it manages — the nested-mode case `initialise` is for). When `-cgs-path` is omitted, `CGSHOME` is instead a fresh `$HOME/.cgs/<name>-<timestamp>` directory — for example `$HOME/.cgs/CGSil1-20261007104123` — so each workspace says which project it holds, and a bootstrapped project can never land inside `ComplexGitSync`'s own clone. With `-cgs-path DIR`, `CGSHOME` is `DIR/<name>`, with no timestamp.

`-force-protocol {ssh,https}` behaves exactly as described under `initialise` above, applying here to every repository the run clones — including the root, which `bootstrap` (unlike `initialise`) also clones from scratch rather than assuming already checked out.

5.3.3 discover

```
$ pixi run cgitsync discover [ROOT] [--write FILE] [--max-depth N]
```

Scans `ROOT` (default: the current directory) for git repositories and drafts a `.cgs` from what is checked out, through the `ComplexGitSyncClient.discover_repos()` facade. This is the entry point for adopting a project that exists on disk but has no `.cgs` describing it yet.

The command is a pure read of the filesystem and of each repository's git config: nothing is cloned, fetched, staged, or modified, and no network call is made. `-write FILE` saves the draft; a relative `FILE` is written inside `ROOT`, the tree it describes, not in the current directory. Without `-write` it only prints what it found, matching the "report first, write only when asked" posture of `-commit-gitignore` and of `submodules report` before `submodules import`.

The walk descends with no depth limit by default, treating every directory that contains a `.git` entry as a repository. Pass `-max-depth N` (`ROOT` itself is depth 0) to bound it. It never descends *into* a `.git` directory: for a submodule the real git directory lives at `<parent>/.git/modules/<name>` while the child's own `.git` is a file, so walking into it would report the same repository twice.

For each repository, `origin`'s URL is read and converted to the canonical `provider:owner/repository` shorthand through the existing `parse_repo_id()` grammar — this command adds no second parser. `relative_path` is taken straight from the walk rather than from a repository name, so a child mounted at `external/Thing` is recorded there and not at `Thing`. `nested_config` is left unset on every entry: the default `auto` already resolves cleanly whether or not a repository carries its own `*.cgs`.

Repositories with no `origin`, or whose remote URL does not resolve to a registered provider, are reported as warnings and left out of the draft. They are never guessed at: a draft that silently invented an address would be worse than one that states what it could not determine. The warning names what to add, not only what was missing — a ready-to-edit entry with the address it could not fill in. Always review the generated file, then `validate` it.

The branch each repository is actually checked out on is drafted twice, for two different jobs. The root repository's own branch becomes `project.default_branch` — the tree's target, read back the same way any other declared branch is ("Branches in a `.cgs`", below). Every repository also keeps its scanned branch as `fallback_branch`, consulted only when the target branch turns out to be absent from the remote. A repository scanned on a *different* branch than the root gets its own explicit `default_branch` too, so it is not silently retargeted at the root's branch on the next `initialise`; one scanned on the same branch as the root needs no per-entry override at all. Re-materialising the drafted tree therefore reproduces the tree that was scanned, branch included — before this, the branch was drafted only as a fallback, which `initialise` never targets first, so a tree scanned entirely on a non-`main` branch quietly re-cloned onto `main` wherever the remote happened to have it.

A repository found *inside* another repository is drafted as that repository's child, not the project root's. Nesting is worked out by comparing paths: every path in a `.cgs` is written from the project root, so a repository whose path lies under another's sits inside it, and the innermost one wins. The report marks such an entry `inside: <path>` and prints the resulting tree, and `build_registry_from_cgs_document()` reads the drafted file back the same way — the holding repository becomes a `PARENT` and keeps its own `.gitignore` entry for the repository it holds, which is what stops a later `add` from restaging that repository as a submodule.

Only what is *checked out at scan time* can be found. A repository cloned without `-recurse-submodules` leaves its submodule paths as empty directories, and those are correctly not reported; recovering them from git metadata instead is the job of `submodules report` and `submodules import`. The same applies when `-max-depth` is given: if it stops the walk while directories are still left to look into, the report says so in a warning rather than presenting a partial answer as a complete one. Without the flag, the walk cannot stop early — there is nothing to stop it at.

5.3.4 `pull`

```
$ pixi run cgitsync pull [spec.cgs|snapshot.gts] [--commit-gitignore]
    [--git-user-name NAME] [--git-user-email EMAIL]
    [--force-protocol {ssh,https}] [--private]
```

Resynchronises the project tree. `.cgs` sources reload the source, discover nested configs, then resynchronise the loaded tree. `.gts` sources load the saved registry as the starting point and then run the same pull workflow. In both cases the operation starts at the project root: `root -> parent -> leaf`. Every repository — root, parent, and leaf alike — is a plain independent clone and receives its own `git pull -ff-only`. Use `checkout <tag> -ref-kind tag` when you want to check out a frozen recorded release state instead of pulling current remotes. If local changes block this safe pull, the CLI suggests `cgitsync autofix`, and names any repository whose commits only exist locally, because `pull -force` refuses while they do.

Before pulling your own branch, `pull` fetches *every* branch of each remote. That is what lets `checkout` join a branch somebody else pushed rather than starting a new one of the same name (see `checkout` below). It also repairs a workspace whose clone left a single-branch fetch refspec behind: a clone downloads one branch, and a workspace cloned by an older build may still carry that narrow refspec.

For a `.cgs` source, `pull` also runs the same `.gitignore` lifecycle sync described under `initialise` — including `-commit-gitignore` and the `-git-user-name`/`-git-user-email` identity override — once the tree-wide pull completes. A `.gts` source runs no discovery, so there is nothing new for the sync to find. `pull -force` does not run this sync at all, and so does not take `-commit-gitignore`, `-git-user-name` or `-git-user-email`; it is a destructive recovery command, not a lifecycle path the sync is wired into.

`-force-protocol {ssh,https}` rewrites every repo's remote to that protocol before pulling, persisting the change (`git remote set-url`, a no-op when the URL already matches) rather

than a one-off override — the switch sticks for every command after this one too, the same way a repo's protocol at clone time sticks for everything downstream of it. Same meaning as `initialise`/`bootstrap`'s `-force-protocol`, applied to an already-cloned tree instead of at clone time — useful when a repo was adopted rather than cloned by `ComplexGitSync` (`initialise` never touches an adopted root's remote) and its remote's protocol does not match what the ambient environment can actually authenticate with. On a failure that looks like an SSH or HTTPS auth problem, the error gains an actionable hint naming `-force-protocol <the other one>` even without the flag.

The rewrite reads the repository's *actual current* remote URL and only swaps its scheme — `ssh` (`user@host:path`) to/from `https`/`http` (`scheme://host/path`) — keeping the host and path exactly as they already are. It does *not* rebuild the URL from the repository's stored identity fields (`gitprovider`/`project_owner_name`/...): those can be missing or stale for a repository loaded from an older `.gts` snapshot (see the "Formal `.gts` snapshot specification" section of the architecture reference), and rebuilding from a wrong or default-GitHub provider would silently aim the pull at the wrong host. Converting the URL that is already configured never has that failure mode. A remote in neither recognised form (a bare local filesystem path, for instance) cannot have its protocol converted; `-force-protocol` then fails with an explicit error naming the repository, rather than guessing.

5.3.5 `pull -force`

```
$ pixi run cgitsync pull --force [spec.cgs|snapshot.gts] [--force-
    protocol {ssh,https}]
     [--private]
```

Destructively resynchronises the same tree. Every repository — root, parent, and leaf alike — is forced to the selected remote branch, parent-first, with `git fetch`, `git checkout -B <branch> FETCH_HEAD`, and `git clean -fd`. Uncommitted changes and untracked files are not discarded: each repository's is set aside first with `git stash push -u`, a warning names it, and `git stash pop` brings the work back. The command never force-pushes, and never leaves a commit on no branch: if any repository holds commits that no remote has, it refuses for the whole tree before touching anything, names the repositories, and asks you to push or merge first. `-force-protocol` behaves exactly as described under `pull` above.

5.3.6 `autofix`

```
$ pixi run cgitsync autofix [spec.cgs|snapshot.gts] [--error TEXT] [--
    repo NAME]
```

Diagnoses and repairs the situation named by the last failing command's error, instead of a person reading raw `git` output and guessing which of `merge`/`pull -force` is safe. With no `-error`, reads the most recent `.cgitsync/logs/*.log`'s failing command (run logs stay local in that folder, so a `push` that folded the memory never moves the log `autofix` needs) — the normal workflow is a command fails, run `cgitsync autofix` next, and nothing gets retyped. `-repo` names which mounted repository to repair when it cannot be guessed from the error text (a chain-shaped repository's name is usually visible in its own remote URL, which `git`'s error text already includes).

Tricky git states, and the safe `cgitsync` command for each. The `SYNC` column `status` prints (§5.3.28, below) names the git-level situation; it cannot say whether the content behind it is safe to reconcile automatically. This table can:

SYNC state	Safe
ahead(+N)	push
behind(-N)	pull
diverged, plain-text content (e.g. .localSpec, .claude)	merg
diverged, chain-shaped content (.memory's lgr/ ledger, and — once built — omniscience's journal)	auto
Anything autofix does not recognise	Refu

autofix tells a plain-text divergence apart from a chain-shaped one before acting on either: fetches, attempts the ordinary merge, and only on a conflict in a repository registered as chain-shaped does it re-sequence both sides' new ledger entries — in the order they were actually recorded, not "whichever side pushed first" — verify the result, and complete a real two-parent merge commit. It never force-pushes and never rewrites history; a repair it cannot verify as correct is refused, leaving the conflict exactly as git left it, rather than writing something that looks resolved and is not.

5.3.7 checkout

```
$ pixi run cgitsync checkout <branch> [--gts <snapshot.gts>] [--ref-kind
    branch|tag]
```

Checks out a branch or tag across the whole tree, parent-first. Loads the READY registry from -gts, then propagates the ref, creates missing local branches, and runs git checkout in every repo. On success the tree remains READY and a new .gts snapshot is written.

A branch that exists on the remote is joined, not recreated. When the repository already holds a remote-tracking ref for the branch — because some earlier pull downloaded it, or another checkout/branch create fetched it on demand — cgitsync creates the local branch *from that ref*, with tracking set. You land on your colleague's commits. When it has no such ref cached, it asks the remote directly (one ls-remote, no full fetch) before deciding: found, it fetches that one branch and joins it the same way; genuinely not found, it creates a new branch where you currently stand, at no extra cost beyond that one lookup.

This is a change of behaviour, and the reason for it is the failure it removes. cgitsync used to create the branch at the current HEAD whenever this workspace had not already cached the remote's ref — so typing a colleague's branch name, on a clone that had never fetched it, gave you a branch of that name holding commits that had nothing to do with theirs, and nothing in the output said so. Depending on some earlier command (a pull, or a differently-built cgitsync during bootstrap) to have fetched every branch first was the actual defect: one command's gap silently became another command's fork.

pull still fetches every branch of each remote before pulling your own, and remains the cheapest way to bring many refs across at once. checkout and branch create no longer depend on it having run first: each fetches the one ref it needs, exactly when it needs it.

Checking out a tag. `checkout <tag> -ref-kind tag` puts every repository that holds the tag on it, detached, and creates no branch. Every writable repository is asked for the tag first — fetching that one tag from its remote when this clone lacks it — and if any still lacks it, the command refuses, naming each, and nothing is checked out. A read-only repository is left where it is: `tag` and `release freeze` only tag what this project may push, so it holds no such tag, and a warning says so. The memory, everything under `.cgitsync/`, stays on its branch as well: it records the tree's history, and a release does not rewind it. To rebuild a release exactly, read-only repositories included, `bootstrap` the `.gts` that release recorded. `checkout <branch>` goes back to work.

Before 4.2.4, this command created a local *branch* named after the tag, and Git then checked out that branch rather than the tag, so the release was not restored. Such a branch may still exist in an older workspace: the tag is checked out regardless, and a warning names the branch so it can be looked at and removed. It is never deleted for you.

5.3.8 `branch create` and `branch list`

```
$ pixi run cgitsync branch create <branch> [--private] [--gts <snapshot.
    gts>]
$ pixi run cgitsync branch list [--per-repo] [--private] [--gts <
    snapshot.gts>]
```

`branch create` creates a branch across the whole `READY` tree without checking it out. `branch list` prints the project's own branches: those of the root repository, local or on origin as of the last fetch, the one the project is on marked with `*`. Each line says which repositories hold the branch they would be on under it, or names those that lack it; a `closed/` branch is listed apart. `-per-repo` prints one line per repository with its own local branches instead. Changes nothing. On either subcommand, `-private` narrows it to the writable configuration repositories.

5.3.9 `fetch`

```
$ pixi run cgitsync fetch [--private] [--gts <snapshot.gts>]
```

Runs `git fetch -prune origin` in every cloned repository of the tree, parent-first. Only the remote-tracking refs change: no local branch, no `HEAD`, no worktree, and no State is written. A branch pushed elsewhere appears in `branch list` afterwards, and one deleted on origin disappears. Prints `fetched` or `skipped` (with the reason: not cloned, no origin, or the fetch failed) for each repository, then a summary; exits non-zero if any fetch failed. `-private` narrows it to the writable configuration repositories.

5.3.10 `branch close`

```
$ pixi run cgitsync branch close <branch> [--private] [--gts <snapshot.
    gts>]
```

Closes a *project* branch: keeps what it alone holds on the project's `ancestors` branch, then renames it to `closed/<branch>` tree-wide, leaf-first, locally and on the remote. Each repository closes the branch it follows under that project branch: the same name in a project repository, `<project>_<branch>` in a private/local one (so `branch close tmpPyPi` closes `ComplexGitSync_tmpPyPi` in `.memory`), and nothing in a private/distant one, which never follows the project and is reported as skipped. Requires `READY`; the tree remains `READY` on success. Renames only: nothing is deleted, so the commits stay exactly as reachable as before, under a name that says what happened to them. Per repository, the branch is pushed under its closed name first, never forced; the old remote name is removed only once the remote is seen to hold the closed one; only then

is the local branch renamed. A branch that exists only on the remote, never checked out here (run `cgitsync fetch` first to see it), is closed there from its remote-tracking ref. A branch the remote never had is renamed locally only.

Refuses, before touching any repository, in two cases: closing the project's own default branch (every repository with no branch of its own falls back to it), and closing a branch that any repository in scope is currently checked out on, which it names, and a branch that lacks commits its remote holds, which would be lost (pull or merge it first). This command closes a branch, it does not move the tree off one; check it out elsewhere first. A repository with no such branch is skipped, not an error.

`-private` selects the writable configuration repositories instead of the project's own, the same as `branch create`.

Closing is the safe point. Before anything is renamed, every commit the branch alone holds — one no other branch, local or on origin, reaches — is kept on the project's permanent `ancestors` branch: `ancestors` in a project repository, `<project>_ancestors` in a private/local one. It starts as an empty commit and gains one merge per branch it keeps; the merge keeps `ancestors`' own (empty) tree and takes the branch as its second parent, so its whole history stays reachable. Nothing is rewritten and nothing is forced: `ancestors` only ever gains a commit. Each move is recorded in the ledger (the entry's `relocations`: what moved, from which address, to which, and its hash there), and the ledger is checked to still verify. A step that fails refuses with nothing renamed. Afterwards the closed branch can be deleted by anything, `cgitsync` or plain Git or the provider's web page, without losing a commit. `ancestors` itself is never closed or deleted. Run on a branch that is already closed — one closed before `ancestors` existed — `branch close` keeps and records what it alone holds and renames nothing.

5.3.11 `branch check`

```
$ pixi run cgitsync branch check <branch> [--private] [--gts <snapshot.
    gts>]
```

Says, per repository, what deleting `<branch>` would lose: the commits only it reaches, and the ledger entries and State files only it holds (by content, so a copy on another branch counts as kept). Each repository is `safe` (nothing would be lost), `recorded` (`ancestors` keeps it and the ledger records the move) or `needs ancestor`. `<branch>` may be a live branch, `closed/<branch>`, or a closed branch by its old name. It first fetches every branch of origin, so the answer counts what origin holds today; that moves remote-tracking refs and nothing else.

5.3.12 `branch delete`

```
$ pixi run cgitsync branch delete <branch> [--private] [--gts <snapshot.
    gts>]
```

Deletes a closed branch, tree-wide. A branch that is not closed is refused: close it first. The ledger is read first: a move a close already recorded, and that still resolves, is reused and never recorded twice, so a branch closed by `branch close` records nothing new here. Anything not yet kept — a branch closed before `ancestors` existed — is kept and recorded exactly as a close does. Then the ledger is verified, and only then is the branch deleted, on origin first and then locally, leaf-first; a local branch is removed only while it still points at the commit that was checked. A step that fails refuses with nothing deleted.

Developer trees only. `branch close` and `branch delete` are refused, before any repository is touched, in a user's tree: one whose `.cgs` holds no private repository. Both remove the branch on the remote for everyone, and a plain `install.cgs` is such a tree. `branch check` and

`branch list` still work there. There is no flag to override it: a maintainer closes the branch on the provider's page. This is a guard against a mistake, not a lock. The tree's kind comes from the `.cgs` its user holds, and `git push -delete` needs no `cgitsync`; only the provider's branch protection stops someone set on it.

5.3.13 add

```
$ pixi run cgitsync add [PATH ...] [--gts <snapshot.gts>] [--dry-run]
```

With no `PATH` arguments (the default), stages all changes across the tree (`git add -all`), leaf-first. Requires `READY`; tree stays `READY`. With `-dry-run`, `cgitsync` prints the leaf-first execution plan and does not mutate repositories.

With one or more `PATH` arguments, each is resolved (relative to `CGSHOME`, or absolute) to the one repo in the tree that owns it and staged there individually (`git add - <path>`), leaving every other repo untouched. A path outside every repo in the tree raises an error immediately, before anything is staged.

5.3.14 rm

```
$ pixi run cgitsync rm PATH [PATH ...] [--gts <snapshot.gts>] [--dry-run
   ]
```

Removes one or more tracked files, each resolved the same way as `add`'s targeted form: to the one repo in the tree that owns it, where it is deleted from disk and the removal staged (`git rm - <path>`). A plain tracked file only — a path that resolves to a directory, or that does not exist, raises an error rather than partially applying, and a path outside every repo in the tree raises before anything is removed. Requires `READY`; tree stays `READY`. With `-dry-run`, `cgitsync` prints the plan without mutating repositories.

Distinct from and unrelated to `submodules import`' internal use of `git rm -cached` (index-only, dropping a gitlink while keeping the child's working tree) — `rm` deletes the file from disk.

5.3.15 commit

```
$ pixi run cgitsync commit <message> [--gts <snapshot.gts>] [--no-stage]
    [--dry-run]
```

Commits changes across the tree, leaf-first. Loads the READY registry from `-gts`. Repositories with no staged changes after the optional `git add -all` step (suppressed by `-no-stage`) are silently skipped. Requires a `READY` tree; the tree remains `READY` on success. Commit preflight now emits warnings for actionable workspace issues such as dirty worktrees, ahead branches, or stale recorded `commit_sha` values. With `-dry-run`, `cgitsync` prints the planned stage/commit sequence without performing git writes.

Two refusals, only in a tree that adopted DevSpec. Both run before anything is staged, and either one refuses the whole commit, naming what it broke; nothing is committed anywhere. A tree whose root does not hold `.agent/.distant/dev-sync/AgentConduct.md` commits with no check at all.

- **The message**, when the root also holds a `pyproject.toml` naming the project. It must start with `<project-name>-<version>` (for example `cgitsync-6.2.0`: one dash, no space and no `v`), the version read from the root's `pixi.toml` like a release's (from `pyproject.toml` only when `pixi.toml` declares none). It must be three lines at most, and carry no backtick, no `$(` and no agent-credit trailer.

- **An archived planning ticket needs its orchestrator's record**, when an `.agent/.local` mount holds a `DevTickets/archive`. A commit that would add a planning ticket stamped on or after 2026-10-09 to `DevTickets/archive/` is refused unless a `self-history` record names that ticket and was written by an orchestrator, a role other than the worker's (see `self-history` below). Only the tickets the commit adds are checked; `pixi run check-tickets` checks the whole archive.

5.3.16 `push`

```
$ pixi run cgitsync push [--gts <snapshot.gts>] [--dry-run] [--force-
    protocol {ssh,https}]
```

Pushes all repositories to their configured remotes, leaf-first. Loads the READY registry from `-gts`. Each repo is pushed to `entry.remote_name` (defaulting to `"origin"`) on its currently resolved branch — never on a tag: when the State records a tag there, the branch Git has checked out is pushed instead. Refreshes the stored hash in `GitTree`. Requires a `READY` tree; the tree remains `READY` on success. Push preflight blocks detached HEADs, missing remotes, unresolved merges, and behind/diverged branches; it warns when the local branch is ahead or when the loaded snapshot records an older `commit_sha`. With `-dry-run`, `cgitsync` prints the push plan without mutating any repository. `-force-protocol` behaves exactly as described under `pull` above.

Folds this project's own memory first, when one is mounted. Before pushing anything else, `push` folds whatever has accumulated in `.cgitsync` into `.cgitsync/.memory` and sends it — the same work `cgitsync memory push` does by hand, run automatically the moment a command was already about to reach a remote. This happens regardless of `-private`: a memory, once adopted, settles exactly as much as an ordinary push. Nothing happens when the tree declares no memory at all. A memory that is declared but not yet adopted, or one that cannot reach its own remote for any reason, prints a warning and does not block the push — the memory's own trouble is never a reason to fail somebody else's publish. `-dry-run` lists `cgitsync memory push` first in its plan under the same condition the real run would perform it.

5.3.17 `tag`

```
$ pixi run cgitsync tag <name> [--private] [--gts <snapshot.gts>]
```

Creates the tag `<name>` in every repository this project may write, leaf-first, and pushes it. The worktrees must be clean. Tagging moves no `HEAD`: every repository stays on its branch, and the next `commit` and `push` carry on there. `tag` writes no State.

5.3.18 `release`: `freeze`, `list`, `load`

```
$ pixi run cgitsync release freeze <message> [--force-tag TAG] [--dry-
    run]
     [--force-protocol {ssh,https}] [--gts <snapshot.gts>]
$ pixi run cgitsync release list [--json]
$ pixi run cgitsync release load <version | number | tag> [--workspace
    NAME [--cgs-path DIR]]
```

A release is the tree as it was, every repository at an exact commit, under a name anyone can find again. `release` has three subcommands, and replaces `freeze-release`, which no longer exists: typing it names `release freeze` instead.

`release freeze` is the release workflow from a READY tree. It chains public operations in order: `add -> commit -> pull -> push`, then it creates and pushes the release tag and writes the `.gts` State. It is the one freeze procedure: there is no force variant. To release from a branch that has fallen behind, run `pull -force` first (it refuses while a commit exists that no remote holds, so commit and push before it). The message is used for the release commit, and it titles the release's tag: the first line of it follows the tag's name in the tag's subject, so a Git provider's tag page says what the release is.

The tag is always `<project-name>-<suffix>`, the project name being the one the tree's root records. The suffix is, in this order: the `TAG` of `-force-tag` (the prefix is added for you); the version the root repository's `pixi.toml` declares, as written, a SemVer or a build version alike (`[workspace]` first, then `[project]`); or, with no version there, the next number after the project's highest numbered release (`1`, then `2`). The project's name is in the tag because one repository can belong to several projects, and each project's releases must keep their own names in it. A tag any repository already holds is refused before anything moves, naming the repository and, when its tag says so, the project that released it: to release again, bump the version in `pixi.toml`, or choose a name with `-force-tag`. `-dry-run` prints the tag, where its suffix came from, and the planned steps; nothing is committed, tagged or pushed.

The root repository's tag carries the release. Every other repository gets a plain tag; the root's is annotated. Its message is the subject line, then one TOML document that reads like a `.gts` file: a `[cgitsync_release]` table (the project, the tag, the message in full, the version, the `cgitsync` version) followed by the release's State, restricted to the project's own repositories, in the `.gts`'s own tables. A release tagged before 6.1 keeps the State as one escaped string in that table, and still loads. A private repository, read-only or writable, is never written into it: the tag may be public. That is what lets any user find and load the release, including in a tree whose memory never leaves the disk.

The ledger entry the release writes carries a `release` row: the tag (`git_tag`), the `project`, the project's version (`project:version`, when `pixi.toml` declares one), and the installed `cgitsync`'s own version and build (`semver`, `artefact:src`). The keys are additive, so every ledger written before them still verifies.

The `pull` step is skipped (not attempted) when the current branch has no upstream configured yet — a branch just created and never pushed. There is nothing to pull in that case; `push` (already upstream-aware, adding `-u` the way plain `git push -u` would) publishes the branch on its own. `-force-protocol` is forwarded to the `pull` and `push` steps, behaving exactly as described under `pull`; the remote rewrite persists, so the tag push picks it up too.

The freeze step stages and commits, creates and pushes the tag — and the branch too, when this step itself made a commit, so the remote branch is never behind its release — refreshes each commit, and writes the `.gts` State, leaf-first, the root last. Its preflight checks remote availability, branch alignment, detached HEADs and unresolved merges. Dirty worktrees and stale recorded `commit_sha` values are reported as warnings because the freeze will stage, commit, and record them. The State includes a deterministic `freeze_manifest` section recording the freeze invariants and the release's name.

`release list` prints every release of the project, newest first: its tag, the project's version, when it was recorded, who tagged it, and where it was read. `ledger` is this workspace's own record; `tag` is the root repository's tag, fetched from its remote (only this project's tags, and nothing else moves); `ledger+tag` is both. So a release another user made is listed too. A tag named like a release but carrying no State is listed as `tag only`. `-json` prints the same rows for a script. It moves no branch and no file, and writes nothing to a remote; it only brings the project's release tags from the root's remote. `BY` is the tagger named by the root's tag, empty for a release only the ledger knows.

`release load` puts the tree back at a release. The name is the whole tag, or what follows the project's name: `release load 1.2.0` loads `<project>-1.2.0`. The State comes from this disk, from the memory, or from the root repository's tag. Without `-workspace` the release goes into the workspace the command resolves (the `cgshome=` line it prints), and nowhere else. A repository the release records and the tree lacks is cloned, at its recorded commit, as `bootstrap` would clone it; a commit its remote no longer holds is refused by name before anything is cloned. Every recorded repository, the cloned ones included, is then left *detached* at its recorded commit, fetching the commit if needed; no branch moves. A repository the tree holds and the release does not record is left as it is and named. The memory (everything under `.cgitsync/`) stays on its branch, as with `checkout <tag>`: a release does not rewind history. If Git cannot move one repository, every one already moved is put back before the refusal. It is refused, by name and before anything moves, when a repository has uncommitted changes to tracked files, or for a `tag only` release, which `checkout <tag> -ref-kind tag` can still look at. With `-workspace NAME` the release is loaded into a new workspace instead, exactly as `bootstrap` would build it from the State, with each repository on its branch at the recorded commit, and the current workspace is left alone; the command prints the `export CGSHOME=...` line `bootstrap` prints, with the timestamped path it built.

Leaving a loaded release. A tree loaded in place, or put on a tag with `checkout <tag>`, is detached: `status` prints `cgitsync_branch=detached`, and `release load` ends by naming the branch the tree was on. There are two ways back onto a branch.

- `checkout <branch>` goes back to work and leaves the release behind: every repository returns to its branch (a private repository to `<project>_<branch>`), the ones the load cloned included. The tag still names the release, and `release load` reaches it again.
- `branch create <name>` then `checkout <name>` works *from* the release, for a fix to a released version. Each repository's new branch starts where it is, at the release commit or at a later commit made while detached, so nothing is lost. `push` then publishes the new branch.

`cgitsync commit` refuses on a detached tree, so a commit made there was made with plain `git`. Such a commit is on no branch, and `checkout` refuses to leave it: it names each repository and its unnamed commits, moves nothing, and points to the second way. A load into a new workspace needs neither, since its repositories are on their branches.

After a release, the tree is still on its branch. The tag names the release; nothing checks it out. The next `add`, `commit` and `push` work on the branch as before, and the State the release recorded names that branch and the exact commit of every repository. Before 4.2.4, the release command recorded the tag as every repository's branch, so the next `push` pushed the tag again and sent nothing, while reporting `origin/<name> (+N)`. A workspace in that state is put right when its next command loads the State, with a warning; check with `status` and push any commit it shows ahead.

Folds this project's own memory first, same as `push`. `release freeze` (and `tag`, which folds the same way on its own combined commit-tag-push) crosses the `.cgitsync` → `.cgitsync/.memory` frontier before doing anything else, under the same conditions `push` does. A release folds twice in one run — once via its `push` step, once via the freeze step — which is harmless: a fold with nothing new pending commits nothing and only re-sends the branch.

5.3.19 `submodules report` and `submodules import`

```
$ pixi run cgitsync submodules report REPO_ROOT [--recursive]
$ pixi run cgitsync submodules import REPO_ROOT [--recursive]
```

Reads `REPO_ROOT/.gitmodules`; `report` lists, and `import` converts, every declared git submodule to a plain ComplexGitSync nested clone. This is the whole job: turning gitlinks into plain clones on disk. It does not author a `.cgs` — `.gitmodules` never records the root's own identity, and a checkout worth importing already has a `.cgs` (hand-authored) or can get one from `discover` (§5 "`discover`"), before or after importing.

`submodules report` is the *dry run*: it prints each submodule's name, path, URL, and branch, without touching the repository.

`-recursive` (same name and meaning as `git submodule update`'s own flag) also converts any submodule that itself has its own checked-out `.gitmodules`, at any depth — root first, not leaf first: converting a submodule stages changes in its own working tree, and converting a deeper level first would make the shallower level's own preflight check (is the submodule's working tree clean?) see that staged dirt and reject its own conversion; converting the shallower level first avoids this, since `git rm -cached <path>` only ever touches the parent's own index, never the submodule's working tree. A submodule path that was never checked out (`git submodule update -init` not run for it) has no `.git` to read a nested `.gitmodules` from and is invisible either way — the same ceiling `discover` already has for itself. Without `-recursive`, behavior is unchanged: exactly `REPO_ROOT/.gitmodules`, one level.

The report lists every level together. Each entry's `path` is counted from `REPO_ROOT`, and `declared in` names the `.gitmodules` file it was read from. A submodule's own `.gitmodules` path means nothing on its own once more than one repository is involved — in `cawaqsviz`, two of the three submodules are called `docs/...` by their own repository.

`submodules import`:

- Verifies every submodule's working tree is clean (`git status -porcelain` must be empty apart from that repository's own `.gitignore`); rejects dirty trees immediately. The `.gitignore` exemption exists because `cgitsync` writes that one file itself in every repository holding a child, so it is routinely dirty in exactly the tree being converted; no conversion can harm it, since `git rm -cached` never touches the child's working tree.

- Runs `git rm -cached <path>` per submodule, dropping the gitlink from the index while leaving the child's working tree and `.git` directory intact (no re-clone, no local history lost).

- Removes the converted stanza from `.gitmodules` (deletes the file entirely when all stanzas are converted) and stages the change.

- Appends `<path>` to the parent's `.gitignore` using the same helper that the `.gitignore` lifecycle sync uses.

After importing, review and commit the staged changes manually:

```
$ git -C REPO_ROOT status          # deleted .gitmodules, modified .
    gitignore
$ git -C REPO_ROOT commit -m "chore: retire git submodules"
```

Note: ComplexGitSync stores no credentials and has no private-repo authentication story. If a submodule's upstream is private and the ambient environment lacks access, the subsequent `cgitsync initialise` or `pull` will fail at the clone step. The `submodules import` step itself (which only touches the local index and working tree) will still succeed.

5.3.20 submodules init

```
$ pixi run cgitsync submodules init REPO_ROOT [--cgs FILE]
      [--max-depth N] [--dry-run] [--force] [--force-protocol {ssh,https
         }]
```

Adopts a submodule-based checkout in one command: `discover` the repositories under `REPO_ROOT`, write the drafted `.cgs`, `initialise` the tree in place, then convert every submodule with `submodules import -recursive`. `REPO_ROOT` is a project cloned and `git submodule update -init -recursive`'d by hand; the command performs no clone of the root itself.

The conversion must come last, and this command is the reason why. `initialise` adopts the root in place but deletes and re-clones every *other* repository straight from its remote, and those remotes still declare submodules — so a conversion run first is undone for every repository except the root. Nor can a second pass repair that: `submodules import -recursive` follows the submodule graph declared by the root's own `.gitmodules`, which the first pass removed, so no deeper level stays reachable. Running the two commands in the other order by hand produces a half-converted tree that looks finished.

Three conditions are checked before anything is written, so a run that cannot work fails while the checkout is still untouched:

- the discovery resolved at least one repository (skipped when `-cgs` supplies a file, which is then the authority);
- the project name matches the directory name — `CGSHOME` is resolved as `<parent>/<project-name>`, so a mismatch would send `initialise` to a sibling directory that does not exist;
- `REPO_ROOT` has a `.gitmodules`. Without one there is nothing to convert, while the clone step would still delete and re-clone every non-root repository, discarding any uncommitted work in them. `-force` overrides this one.

`-dry-run` prints the discovery and the submodules that would be converted without writing a `.cgs`, cloning, or touching any repository. `-cgs FILE` uses an existing `.cgs` instead of writing one (or writes the draft there, when `FILE` does not exist). `-max-depth N` bounds the discovery step, which is otherwise unbounded — most trees never need it. `-force-protocol` applies to the clone step only; discovery reads each repository's configured remote as it is.

The command stops at a `READY` tree with the conversion staged but not committed — it touches every repository that held a submodule, and some of those may belong to other people — and prints the `branch create/checkout/add/commit` sequence to run next.

5.3.21 memory

```
$ pixi run cgitsync memory status [--search-dir DIR]
$ pixi run cgitsync memory list   [--branch NAME] [--search-dir DIR]
$ pixi run cgitsync memory show <state> [--full] [--search-dir DIR]
$ pixi run cgitsync memory explore [--branch NAME] [--timeline] [--
    search-dir DIR]
```

Looks at what this workspace remembers. A **State** is one recorded snapshot of the tree, named by its own content; the **ledger** holds one entry per operation, saying when that State was seen and by which tool versions.

A workspace whose `.cgs` declares no memory — every install from `install.cgs` — gets one made for it, locally, the first time a command records something: a repository at `.cgitsync/.memory` with no remote. `memory push` folds and commits into it and never sends it anywhere, even if a remote is added by hand; `memory status` says it is local and prints the `.cgs` line that publishes

it. Declaring that entry and running `memory adopt` is the opt-in; publishing a memory is a developer's privilege.

A user's tree or a developer's. `status` prints `profile=user` or `profile=dev` on its summary line, and `status -json` carries the same word as `profile`. A tree whose `.cgs` holds no private repository is a user's, its memory never leaves the disk, and `branch close` and `branch delete` are refused in it. A tree holding at least one private repository, read-only or not, is a developer's, and its memory is meant to be sent to the repository its `.cgs` declares. When a developer's `.cgs` declares none, the work is still recorded locally, and the first command that records something asks, in a terminal only, for the provider (default `github`), the owner (the owner of most of the tree's private repositories, else the project's own) and the name (default `.memory`); it shows the command that creates the repository and, on a yes, runs `memory setup`. A no is asked once. Without a terminal — `-json`, CI, a Python caller — after a no, or after a step failed, it only warns that the work has no memory back-up and no global ledger record, and names the command that fixes it; `memory status` says the same.

```
$ pixi run cgitsync memory setup [--provider P] [--owner O] [--name N]
    [--cgs FILE]
```

`memory setup` runs `repo create`, `memory mount` and `memory adopt` (below) as one command, in that order, and stops at the first that fails, saying which and what was left done. In a terminal it asks the three questions first, with the defaults filled in; elsewhere it uses the flags and defaults as given.

- `status` — how many States, how long the chain is, when it was last written, which of `verify check`'s answers this memory is in, and the toolchain the records carry. The first entry's versions are shown beside the latest when they differ, so a chain spanning an upgrade says where the upgrade fell.
- `list` — every State, newest recording first, with the commands that recorded it. A State with no timestamp is one no entry records: history from before the ledger, or a file that arrived some other way.
- `show <state>` — the State's integrity schema and the root hash of its tree (`integrity_schema`, `gittree_root`); the environment that State's own commits ran under, as a bare reference; that State's own topology right after, drawn exactly the way `view-tree` draws the live one (`format_view_tree`) but as it was at that State, not as it is now; then every entry naming it and what each `commit` wrote: the message, the repository, and whether the commit has been published. The argument is the State's content hash or any unambiguous prefix of it; nobody retypes sixty-four characters. Pasted back verbatim from wherever it was printed also works: the bare prefix, the `state=<hash>` label `explore -timeline` prints it under, or the full `state(<hash>)` form a ledger entry uses. `-full` prints whole messages instead of first lines, and the remote and ref each publication went to.
- `show env=<ref>` — one Environment record, in full: machine, tools, credentials and manifest digests, drawn as the same kind of tree `view-tree` draws a repo tree with. This is the detail a State's own bare `environment=env(<hash>)` reference never shows, since a State answers "what was this tree", not "what ran it". Takes the full id, a bare prefix, or the `env=<hash>` form its own argument is spelled with.
- `explore` — a memory a person can read without a hash. The default view is one row per commit this memory recorded as published, newest push first, on the branch checked out here — what a colleague pulling this branch would see. `-timeline` reads the ledger straight through instead: one row per entry, in the order it happened, so `checkout`/`merge`/

`push` appear beside `commit` rather than being dropped — each row also prints the State it recorded, so a prefix to hand `show` is never more than a scroll away. `-branch NAME` asks for a different chapter of the memory. One not checked out here is read from Git, as a timeline of its entries: from its branch, its closed name, or the copy `ancestors` keeps once the branch is deleted, and the answer says which. One this clone holds nowhere names `memory clone -branch NAME` rather than answering for the wrong branch. `list` and `as-of` take the same option, and `show` looks in every chapter for a State that is not on disk; it also names the repositories whose commit, in that State, only `ancestors` still reaches.

- `as-of <time>` — what was this tree at that time? The State the ledger recorded at or before it, not the nearest and not the latest, with the `show` command to type next. A time is read as UTC unless it carries an offset (`2026-09-30T17:00+02:00`); a bare date means the end of that day. A time before the first entry answers "nothing recorded yet". When the chain does not verify, or its clock moved backwards (`verify check`'s `time-inconsistent`), the answer is printed with `answer_reliable=false` and a warning, never as if the chain were clean.

A commit marked `unpushed` exists on this machine and nowhere else. That distinction is the reason the messages are kept at all: while the repository is present `git log` is the better answer, and this record is for when it is not — a deleted branch, an archived project, a fork that went away. The ledger entry that recorded a set of messages carries their fingerprint, so editing one afterwards is a finding `verify check` reports.

`status`, `list`, `show` and `explore` are read-only; nothing they do sends a memory anywhere.

Keeping a memory when the disk does not. A memory can be a repository of its own, mounted at `.cgitsync/.memory` exactly as `.localSpec` is mounted: one repository holding every project's memory, on a branch per project. `.cgitsync` itself stays the workspace's own live state — States, the ledger, logs — so the mount one level inside it is checked out and merged like any other private repository, never blocked by activity that is not its own. A developer gets there with one command, run at any time: `memory setup` (§5.3.21). The five commands below are the same path step by step, run once per project; `tutorials/05_memory.md` walks both.

```
$ pixi run cgitsync repo create github:you/.memory
$ pixi run cgitsync memory mount [--cgs FILE]
$ pixi run cgitsync memory adopt
$ pixi run cgitsync memory push
$ pixi run cgitsync memory branch --project-branch main
```

- `repo create` — creates the repository by running the provider's own tool (`gh`, `glab`, `tea`), which you have already signed in to. ComplexGitSync stores no credential and sends none. A repository that already exists is an ordinary success; a tool that is missing or signed out prints the command to run and exits `2`.

- `memory mount` — adds one entry to a `.cgs` that already exists, leaving the rest of the file, comments included, exactly as it was. Adding it twice changes nothing.

- `memory adopt` — creates `.cgitsync/.memory` fresh and turns it into that repository, on this project branch's memory branch. It starts empty: the first `memory push` is what folds in whatever `.cgitsync` already held. When the workspace already has the local memory ComplexGitSync made for it, that repository and every commit in it are kept: adopting adds the remote and, if the repository already holds this branch's base, joins it with one merge commit. If the two cannot be merged cleanly, nothing is adopted and

the local memory is left exactly as it was; `-reboot` or `cgitsync autofix` are the ways on. `-reboot` adopts the same repository identity but skips carrying forward whatever the fallback branch already holds, so the mount's content starts fresh instead (§5.3.21).

- `memory branch` — creates the memory branch another project branch will need, and pushes it. Needed once, for a memory born on a feature branch: merging `memory-dev` into `main` asks for a branch that has never existed. A merge that finds one missing names this command rather than creating the branch itself, because a branch that is not there is as likely to be a typing mistake as a new branch.

Migrating a memory mounted before `memory migrate` existed. A memory adopted before this command existed sits directly at `.cgitsync`, sharing it with the workspace's own live state. `cgitsync memory migrate [-cgs FILE]` moves it onto the layout above — `.git` and every tracked file travel down into `.cgitsync/.memory`, history unchanged; whatever was never tracked stays exactly where it already was, which is where the new layout wants it anyway — and updates the `.cgs` entry that declares the mount to match. A fresh `memory adopt` never needs this: it mounts at the new layout directly.

```
$ pixi run cgitsync memory init     # the .cgs entry to add, and the
    branch
$ pixi run cgitsync memory clone    # bring a memory onto a new machine
$ pixi run cgitsync memory push     # commit what it gained, and push it
```

`init` proposes and stops: ComplexGitSync speaks Git and nothing else, so it prints the command that creates the repository and waits. Nothing is ever pushed automatically — a machine with no network keeps a complete, verifiable memory and sends it later.

Cloning a memory refuses to write over a memory that is already here, because a memory nobody has pushed is the only copy of itself.

Starting a memory's history over. A project's shape changes — repositories added, removed, restructured — and a memory built for the old shape stops being a clean answer to "what does this project look like." `cgitsync memory reboot` closes the current chapter and opens an empty one, without losing the old one:

```
$ pixi run cgitsync memory reboot
```

In order: whatever `.cgitsync` is holding pending is folded in and pushed under the branch's current name, exactly as `memory push` already does; the tree's current shape is exported — from the loaded `.gts`, never a hand-authored file — to a permanent, versioned `.cgitsync/.memory/.cgs/<project>` (`N` incrementing once per reboot, never overwritten, never reused); the branch is archived, renamed both locally and on origin to `<branch>.archived-<date>`, the old name removed from origin only *after* the new one already holds the same commits, so nothing the memory ever recorded is unreachable for even a moment; and a fresh branch takes the original name, so nothing about how the memory is mounted changes. Its tracked content is cleared except `.cgs/` — the permanent, versioned record above, which a reboot must not erase — and one fresh State is written, committed, and *pushed* immediately, so the branch is live, with a real `HEAD` `cgitsync status` can read, and is never missing from origin: the old name was already gone from there by the time the fresh branch is built, so a bootstrap on another machine that ran in between would otherwise have found no branch under this project's name at all, and fallen back to one that never held its `.cgs`. Nothing is ever force-pushed or deleted. `cgitsync memory adopt -reboot` is the same fresh start for a mount being adopted for the very first time. Appending — the ordinary `memory adopt`, and the only thing `memory push` ever does — stays the default either way.

Keeping a branch's memory when its project branch is merged. A memory is a hash chain: one numbered file per ledger entry, plus `HEAD`. Two memories that grew apart hold *different* entries under the *same* numbers, so merging them file by file either collides or interleaves them into a chain that no longer verifies — and a memory started afresh by `memory reboot` shares no commit with the one it replaced at all. So a memory is never merged file by file. `memory merge` keeps one side whole and the other as history:

```
$ pixi run cgitsync memory merge <branch> [--into TARGET] (--ours | --
    theirs)
```

`<branch>` and `TARGET` are **project** branches (the target defaults to the one checked out); the memories merged are the branches derived from them. The words are Git's: merging `<branch>` into `TARGET`, `-ours` keeps the memory of `TARGET` and `-theirs` keeps the memory of `<branch>`. Exactly one is required, because which chain continues is never guessed.

It writes one commit whose files are the kept side's, with the target's branch as first parent and the other as second, so the whole history of both is reachable and `verify check` reads one intact chain. What is pending in `.cgitsync` is folded and committed first, so nothing recorded is lost; the result is pushed without force, and a push the remote would refuse is reported as such, saying the merge is already done here. Nothing is rewritten and nothing is spliced. Merging a memory that is already part of the target does nothing, whichever side is asked for, and `-theirs` on a target that is simply behind moves it forward without a new commit.

What a memory carries off your machine. One path: the tree's own root, with `$HOME` substituted. Every other path it records — each repository's, and any path in a command line it logged — is written against the tree as `$CGSTREE/...`, so neither your directory layout nor your user name travels with it.

5.3.22 `self-history`

```
$ pixi run cgitsync self-history add \
    --ticket NAME --goal TEXT --action TEXT \
    --worker-role ROLE --worker-vendor NAME --worker-model NAME \
    --orchestrator-role ROLE --orchestrator-vendor NAME --orchestrator-
       model NAME \
    --spec-respect-score N --spec-respect-basis measured|asserted --spec
       -respect-reasoning TEXT \
    --gating-score N --gating-basis measured|asserted --gating-reasoning
        TEXT \
    --quality-score N --quality-basis measured|asserted --quality-
       reasoning TEXT \
    [--state-before STATE] [--state-after STATE] \
    [--lint-passed|--lint-failed] [--tests-passed|--tests-failed] \
    [--pushed] [--pushed-reason TEXT] [--search-dir DIR]
```

Records one piece of agent work: which ticket it served, the goal and the action taken in plain English (at most three lines each), the two agents involved — a **worker** that implemented the ticket and an independent **orchestrator** that quoted the work, each with a role from `.localSpec/AGENT.md`'s roster, a vendor and a model version — and a three-part conformity score (spec respect, `.PUBLIC/ .PRIVATE` gating, quality of production), each part marked `measured` or `asserted` rather than blending the two. In a tree that adopted DevSpec, this record is what lets a finished planning ticket be committed: `commit` refuses to add an archived ticket that no orchestrator's record names (see `commit` above).

The score is out of **100**: spec respect is scored from 0 to 33, gating from 0 to 33 and quality from 0 to 34, and the total is their plain sum. The orchestrator starts each part at its maximum and takes off points for each shortfall it can name, saying which in that part's reasoning;

that method is guidance, and the tool checks only that a score is within its range (a score outside it is refused with a clear message) and that the total is the sum. Every score is shown with its maximum, for example `spec_respect=26/33 (asserted) gating=32/33 (measured) quality=26/34 (asserted) total=84/100`. The optional `-conformity-explanation` is a short account, in the orchestrator's own words, of how the total came about; it is stored with the record and printed under the score.

Two facts the command fills in itself, because this project can check them directly rather than trust an agent to type them: which signed agent-contract record was in force (by its own hash, from `.agent/.distant/dev-sync/agent-contracts/current`), and this workspace's own `errors=` count from the same view `status` prints. Everything else is the orchestrator's own account. A missing signed contract is not an error — the field is simply empty.

`-state-before`/`-state-after` are **verified against the ledger**, not merely shape-checked: each, if given, must name a State some ledger entry actually recorded, still on disk, still hashing to its own name — the same three questions `verify check` itself asks. A citation that fails any of them is refused, not recorded as though it were fact. `-state-after` defaults to the ledger's own most recent entry when not given.

The record is written to `.cgitsync/.self-history/`, content-addressed the same way a State or an Environment record is — the pending half. Once self-history has been adopted (below), `cgitsync memory push` folds it into `.cgitsync/.memory/.self-history/` and pushes it, *before* folding and pushing `.memory` itself, so the parent's own push always records a leaf that already exists.

```
$ pixi run cgitsync self-history adopt [--owner NAME] [--branch NAME]
   [--search-dir DIR]
```

Self-history rides `.memory`'s own lifecycle rather than needing a lifecycle of its own. `self-history adopt` is the *one* place this is ever decided: it writes `config-memory.cgs` once, for a brand-new project or as the explicit retrofit for a `.memory` adopted *before* self-history existed. From then on, `cgitsync memory adopt` follows that decision automatically, silently, whenever a project's own memory branch inherits it from a fallback base branch that already carries it — the same mechanism that already lets a new project's memory share history with another. A project that has never adopted self-history has no such file anywhere, so this sees no new behaviour at all. Unlike that automatic follow, asking for `self-history adopt` explicitly raises when the remote is not there, rather than quietly doing nothing — asking for it means wanting to know why it did not work.

`cgitsync memory clone` brings self-history back too, the moment it was adopted upstream; `cgitsync memory reboot` leaves it entirely alone, on purpose — a reboot starts `.memory`'s own history over, and the record of what happened, including across the reboot, is exactly the thing that should survive it.

```
$ pixi run cgitsync self-history list [--search-dir DIR]
```

Every self-history record this workspace holds, folded and pending, oldest first — consultation only. A workspace that has never adopted self-history, or has adopted it but written nothing yet, prints that plainly rather than erroring.

5.3.23 `verify check` and `verify repair`

```
$ pixi run cgitsync verify check [--search-dir DIR] [--json]
$ pixi run cgitsync verify repair [--search-dir DIR] [--json]
```

`verify check` says which of five things is true of this workspace's recorded history. One of them, never a blur of two:

Answer	Exit	Means
`verified`	0	A chain was read and every link held.
`no-history`	0	Nothing has been recorded here yet. Not a failure — a new workspace is not a broken one.
`legacy`	1	History exists, in the single-file `.lgr` register, which carries no chain. Readable, not verifiable: an edit to it leaves no trace.
`corrupt`	1	A chain was read and it does not hold.
`time-inconsistent`	1	The chain held, but its own timestamps move backwards somewhere. Your history is intact; the clock that stamped it was not.

`legacy` exits non-zero on purpose. The question is "is this history intact?", and "I cannot tell" is not a yes: a build that gates on `verify check` must not pass because the evidence is in a format nobody can check. Every workspace created before the hash-chained ledger is written is in exactly that state.

`time-inconsistent` is deliberately not `corrupt`. Every link checked out — nothing was rewritten — so the two answers send you to look at different things. A clock corrected mid-session, a restored virtual machine, or a machine that disagreed about the hour all read like this, and so does an entry somebody backdated: a date cannot be moved backwards without contradicting the chain around it. It still exits non-zero, because something is wrong even though your history is not.

When there *is* a chain, what is checked: every entry's `prev` must match the previous entry's `entry_hash` (`BROKEN_LINK`), every entry's own `entry_hash` must match its recomputed canonical hash (`BAD_ENTRY_HASH`), `seq` numbers must be gap-free and unique (`SEQ_GAP`/`SEQ_DUPLICATE`), and the cached `HEAD` must agree with the recomputed head (`HEAD_STALE`).

`CGSHOME` is located the same way as `status`: from `-search-dir`, else `$CGSHOME`, else the current working directory's ancestors.

`verify repair` runs the same check, then corrects a stale `HEAD` cache to match the recomputed true head. **Entries themselves are never rewritten or deleted** by either subcommand — a broken chain is reported, not silently healed, since a ledger that can be edited back to looking clean provides no evidence of anything.

The States on disk are checked too. Beyond the chain itself, `verify check` cross-references what the entries say against what is there: an entry naming a State that is missing (`MISSING_STATE`), a stored snapshot whose content no longer hashes to the name it is filed under (`STATE_DIGEST_MISMATCH`), and a State on disk that no entry records (`ORPHAN_STATE`). The second is what naming a State by its content bought: a stored snapshot can no longer be edited quietly.

A State is hashed at three levels: each repository (`repo_hash`), the tree of repositories (`merkle_root`), and the State itself. So an edited snapshot is reported bottom-up — `REPO_HASH_MISMATCH` naming the repository by its path, then `GITTREE_ROOT_MISMATCH`, then `STATE_DIGEST_MISMATCH` — and you see which repository changed, not only that something did. A State written before this scheme (`integrity_schema = 1`) is refused rather than measured: run `memory reboot` to start a new memory under it.

Where the history lives. `.cgitsync/lgr/`, one file per entry, appended to by every command that writes a State. Each entry records the versions that produced it — cgitsync, git, pixi, and the data backends dvc and git-lfs — inside the entry hash, so an edited version string is as detectable as an edited command. The older single-file `.lgr` register is still read, and is no longer written.

5.3.24 `env show` and `env check`

```
$ pixi run cgitsync env show [--search-dir DIR]
$ pixi run cgitsync env check [--search-dir DIR] [--cgs FILE]
```

`env show` observes the environment that can make the current tree usable. It reports the machine architecture, Pixi platform, operating system, kernel and libc; raw and parsed versions for Python and the tools `ComplexGitSync` drives; provider authentication as a fact; and the paths and digests of recognised manifests in the tree. A missing tool is printed as `none`, never as an empty value. Tokens, keys, user names, key paths, and paths outside `$CGSTREE` are never part of an Environment record.

`env check` compares that observation with requirements from the loaded tree's source `.cgs`. Pass `-cgs FILE` to check an explicit authoring document instead. Missing tools and versions older than the declared minimum are drift: the command names each mismatch and exits non-zero, which makes it suitable for continuous integration. Extra observed tools are reported as information and do not make the check fail.

The check is separate from tree operations on purpose. `initialise` and `pull` may warn about drift, but a mismatch never blocks them. Both environment commands are read-only and install nothing.

The optional authoring declaration names the repository where the environment is installed and requirements that cannot be inferred from a lock file:

```
environment_root = "my-project"

[environment]
tools = { git = "2.43", pixi = "0.66" }
compilers = ["cc"]
system_libraries = ["libssl"]
services = ["postgresql"]
manifests = ["Cargo.lock"]
```

The root must name a configured repository (or `root`). Manifest contents remain in Git; the Environment record stores only tree-relative paths and digests.

5.3.25 `view-tree`

```
$ pixi run cgitsync view-tree [spec.cgs|snapshot.gts] [--depth N] [--
    collapse REPO]
```

Renders a deterministic terminal tree that focuses on repository topology and operational state. Each line reads:

```
name (node_type) [sync_state] @sha br=<branch> [fb=<fallback>]
```

`br=` is printed for *every* repository, including the common case where the branch is simply the tree's default, so that reading `view-tree` tells you the branch each repository targets without opening the `.cgs`. `fb=` appears only when the declared fallback branch differs from that target, since a fallback equal to the target says nothing. Use `-depth` to limit visible levels and repeat `-collapse` to hide selected subtrees.

On a tree with private mounts, `br=` is where you see pinning at work: the unprivate repositories carry the branch you moved the tree to, and each private mount still shows its own.

Branch Topology Propagation Rules (T35):

1. **Reference branch**: The root repository's current branch is the canonical reference for all repos.

2. **Leaf-to-root inheritance**: Branch targeting flows root-first via `propagate_global_branch`, verified on-disk by `ComplexGitSyncClient.validate_topology()` (Python API only) without issuing any git writes.

3. **Allowed divergence**: Repositories in a frozen (`TAG`) state produce a `tag_divergence` diagnostic that does *not* make the topology incoherent.

4. **Incoherent states** (blocking): `misaligned_branch` (different branch from root) and `detached_head` (detached HEAD without a tag reference).

5.3.26 What `cgitsync` promises a script

Exit codes. Three, meaning the same thing in every command:

Code	Means
`0`	The command did what was asked.
`1`	It ran, and the answer is no — a merge conflict, a tree that is not `READY`, a verification that found something.
`2`	It could not run — bad arguments, no workspace, a missing or unreadable file.

The distinction that matters to a script is between "I asked and the answer is no" and "I could not ask". `validate` is the one command that judges a document rather than acting on one, so an invalid document is its answer (`1`); every other command exits `2` on the same document because it could not run at all.

An expected failure prints one line on stderr and no traceback. A traceback is therefore a defect report, not a usage error — which is the point of mapping the expected ones.

`-json`. `status`, `verify check` and `verify repair` accept it. Each prints one JSON object on stdout and nothing else; every human-facing line goes to stderr instead, and the exit code is unchanged, so a caller may use either signal. A failure prints one object too, carrying `"status": "error"` and the exit code, rather than leaving a pipe with nothing to parse. `-help`, `-version` and a command line that fails to parse are deliberately not JSON: the flag itself may be what failed.

The object carries `schema_version`. The promise attached to it is **additive only**: new fields may appear, existing fields do not change meaning and do not vanish.

What is stable, and what is not. This is the contract the version number is measured against: a change that breaks a line marked stable is a new major version.

Surface	Promise
Command names and documented flags	Stable within a major version.
Exit codes	Stable within a major version.
`-json` output	**Additive only**, as above.
`.cgs` and `.gts` grammar	Versioned in the file, and the version is read on load.
Python modules under `src/ComplexGitSync/`	**Not a public interface.** `ComplexGitSyncClient` is the CLI's own implementation; import it and a refactor may break you, with no deprecation owed. The CLI is the product.
`verify`	**Experimental.** The ledger it reads is still changing, so its output and findings may change.

5.3.27 Where a command works: the default workspace

Every command that is not given an explicit path finds its workspace from three inputs, in this order: `-search-dir`, then `$CGSHOME`, then the ancestors of the current directory. When none of them lands on a `.cgitsync` directory there is a fourth answer: an empty workspace of ComplexGitSync's own, under `$HOME/.cgs` (or `$CGSPATH` when that is set).

That workspace is a real directory holding a valid `.gts` with no repositories in it. An empty tree is not an error — it is a project that has not started — so `status` answers `no living project yet`, names the three commands that start a project, and exits `0`.

- **Created once, then reused.** Its path is recorded in `$HOME/.cgs/default`; later runs read that pointer instead of minting a second empty workspace.
- **It never chooses for you.** When other workspaces exist under the root they are listed, each with the `export CGSHOME=...` line that would select it, and none is selected automatically.
- **It never overrides `-search-dir`.** A directory you named that holds no workspace is an error, not an invitation to work somewhere else.

The `use_case` field. Every discovery line also says which of the two ways of running is in force: `nested` when the ComplexGitSync being executed lives inside the workspace it manages, `standalone` otherwise. It is observed and printed, never obeyed — nothing behaves differently because of it. It exists so that a user who believes they are in one case while standing in the other is told.

5.3.28 `status`

```
$ pixi run cgitsync status [--gts FILE]
```

Summarises tree readiness and per-repository sync state. The table reports the scope, local branch, upstream branch, local worktree state, ahead/behind tracking counts, current HEAD, and the commit recorded in the loaded `.gts`.

The `cgitsync_branch` field. The `summary` line carries `cgitsync_branch=<branch>`: the branch the project is on, which is the branch its root repository is on. It is what `cgitsync checkout` sets (`branch create` makes a branch without moving onto it), and what every other repository follows.

The table shows a branch per repository and they are not all the same on purpose — a `private/local` repository keeps this project's settings on a branch named after the project, so a project on `apoub` shows `ComplexGitSync_apoub` there. The field is the one place that says which of those branches is the project's own.

Shown	Means
`<branch>`	The branch the root repository is on.
`detached`	The root is parked on a commit rather than a branch. `cgitsync checkout <branch>` puts the tree back.
`unknown`	There is no branch to report: no project has been loaded, or Git could not be asked.

The field is printed in every state. A field that disappears when the answer is awkward is harder to read, by eye or by script, than one that says it does not know.

The `SCOPE` column. It says, in three words, what kind of repository each row is and whether you may write to it:

Shown	Means	In the `.cgs`
`project`	this project's own; write freely	no `private`
`private/local`	shared, and yours to write	`private, writable`
`private/distant`	shared, read-only	`private` alone

private means the repository is shared with other projects — a configuration repository. **local** then means this project may write to it, normally because it sits on a branch named after your project; **distant** means you only read it. A repository nested inside a private one is shown the same way, because it is just as shared.

`commit` and `push` reach the `project` rows; `-private` reaches the `private/local` rows; nothing writes to a `private/distant` row. When any private row is present, the table prints a legend saying so.

The `SYNC` column. It says how the branch a repository is on stands against the branch it tracks — the one named in `UPSTREAM_BRANCH`, which reads `-` when there is none. There are six values:

Shown	Means
`synced`	Level with the upstream.
`ahead(+N)`	`N` commits here that the remote does not have. `push` sends them.
`behind(-N)`	`N` commits on the remote that are not here. `pull` fetches them.
`diverged(+N/-M)`	Both, from a common ancestor. `autofix` (§5.3.6) diagnoses which of `merge` or a chain-aware splice is safe; `pull -force` refuses to run while this workspace's own `N` commits exist nowhere else.
`no-upstream`	This branch was never pushed, so there is nothing to compare it to. Normal for a branch you just made, and for a `private/local` repository that only `push -private` ever sends.
`unknown`	The branch names an upstream that does not resolve. `pull` or `push` repairs it; if it persists, the remote is unreachable or the ref was deleted.

The last two are worth telling apart. `no-upstream` is a normal state of a branch nobody has pushed yet; `unknown` is a fault. They used to share the one word `unknown`, which made the column not worth reading — an ordinary new branch looked exactly like a broken one.

`unmeasured` in the summary line. The summary counts the `no-upstream` and `unknown` rows as `unmeasured`, separately from `ahead` and `behind`. Those rows were previously folded into `ahead=0 behind=0`, which reported a measurement nobody had taken. A repository nobody could measure is not a repository that is level, and the summary no longer says the second when it means the first.

The memory mount reading `dirty` mid-`push`, or `HEAD` ending in `*` right after, is normal. If the tree mounts its own memory at `.cgitsync/.memory` (§5.3.21), that row is an ordinary private/local repository the rest of the time — clean before and after every command except `memory push` itself, which folds `.cgitsync`'s own pending content into it before committing. One thing about it is still worth knowing:

- **`HEAD` ends in `*` right after `memory push`.** The `*` means the loaded `.gts` recorded a different commit than the one actually there now. `memory push` moves the memory's real commit directly and does not write a new `.gts` itself, so the State most recently written still names the commit from before the push. The very next command that writes a State reads the memory's true `HEAD` and the two agree again — it never needs a second `memory push` to catch up, only an ordinary next command.

Not worth chasing to a clean reading on purpose: a memory that never showed this for a moment would not be recording what `memory push` just did.

5.3.29 `merge`

```
$ pixi run cgitsync merge <branch> [--into TARGET] [--private] [--ff-
    only|--no-ff] [--dry-run] [--resolve]
```

Merges a branch across the tree, leaf-first. The argument is always the **project's** branch; each repository works out what that means for itself. A project-owned repository merges that branch. A private/local repository merges the branch derived from it (`<base>_<branch>`), because that is where its settings for that project branch live — it has no branch of the project's own name at all.

`-into`: merging into a branch you are not on. Without it, `merge` merges into whatever is checked out, so reaching another branch means running `checkout` first. `-into TARGET` names the target and does both halves in one command.

That is not a convenience when the tree being merged **contains the ComplexGitSync you are running** — the developer checkout, which installs itself editable. A separate `checkout` replaces that build, and the merge typed after it runs under the older one, against a workspace the newer one wrote. A single command finishes under the build it started with, because Python has already imported its modules.

Every repository is checked before any is touched, so a conflict or a missing target leaves the whole tree where it was: still on the source branch, nothing checked out and nothing merged. A fast-forward is reported as one, which is usually why a repository looks untouched afterwards. `-into` never creates its target: a branch that is not there is as likely to be a typing mistake as a new branch, so it is named and the command stops. It cannot be combined with `-resolve`, which promises the opposite about a conflict.

`checkout` warns when it is about to install a different build of the tool, and still performs the checkout — looking at an older branch is legitimate, being surprised by it is not.

A `-private` or `-all -into` call only ever touches part of the tree, and the rest is meant to still be somewhere else until it too is asked to move. So a second scoped call — `-private` after the plain form, or `-all` after either — finishes what the first one left behind, rather than refusing those repositories for not having moved yet.

`-private` merges the writable configuration repositories instead of the project's own. Read-only ones are never merged.

The memory is kept, never merged. Whatever the scope, the memory (`.cgitsync/.memory`) is not merged file by file (§5.3.21): the tree-wide `merge`, `merge -into` and `pull -private` act on it as `memory merge -ours`. The target's memory stays exactly as it was, so `verify check` on it does not change, and the source's memory is recorded as history, so `branch delete` of the source then finds its entries held by another branch. The plan and the report say so in those words: *kept*, never *merged*. A memory the branch's own memory already contains is *up to date* and never blocks the other repositories — it no longer turns a rebooted memory into a conflict that names no file.

Branches with nothing in common. A repository whose two branches share no commit is not a conflict: Git will not merge unrelated histories, and there is no file to resolve. The tree is refused by name, `-dry-run` included, and `-resolve` is not offered:

```
merge refused; no repository was merged: conf: 'demo_feature' and 'demo'
    share no commit; Git will not merge unrelated histories
```

A conflict in any repository leaves no repository merged.

Every repository in scope is checked before any is merged, using a read-only test that touches neither worktree nor index. A tree-wide merge that stopped halfway would leave a workspace no `.gts` describes and no command undoes, so it is prevented rather than reported. `-dry-run` prints the plan — and, importantly, the branch each repository would actually merge, and whether it would merge at all.

The refusal names the branch and the files. When `cgitsync` refuses, it names every blocked repository and, under each, the branch it was merging and every file that conflicts:

```
merge refused; no repository was merged: ComplexGitSync: merging 'my-
   feature' conflicts in tests/unit/test_documents.py
```

A file that both branches changed is only listed when it genuinely conflicts. Binary files count: a tracked PDF changed on both sides conflicts just as a source file does, and is reported the same way. `-dry-run` shows the same list without merging.

`-resolve`: fix the conflict instead of reading about it. `-resolve` merges one repository at a time and stops at the first that conflicts, then opens that repository in your merge tool.

`-resolve` gives up the guarantee above.

Repositories merged before the conflict stay merged, so the tree can be left partly merged. That is the price of reaching a conflicted worktree at all, and it is why this is opt-in: the command prints the warning before it writes anything, then names what it merged, where it stopped, and which repositories it never reached.

Your own `merge.tool` is used whenever you have configured one. Otherwise VS Code is suggested when it is available, passed for that one call — your Git configuration is never written. With no merge tool available at all, the command prints the command to run by hand and stops, rather than failing.

A memory is never merged file by file, here or in any other mode: it is kept whole and the other side recorded as history, and it is never a stop. A repository whose branches share no commit stops the run by name; nothing is attempted there, since Git refuses it.

`-all-conflicts`: go on past a text conflict. Used with `-resolve`, once your merge tool leaves no conflict in a repository it commits that merge (`Merge branch '<source>'`), lists it with the others as merged, and goes on. It stops, naming the repository, on histories with no common commit, on a binary conflict (it names the file, says that Git left this branch's version and on which branch the other one is), on a file left unresolved, and when no merge tool is available, in which case it prints the command to run by hand. It never resolves, regenerates or stages a file in silence, and it always ends. Tutorial 7 teaches merging a tree end to end.

Check out the target first. `merge` brings a branch *into* the one the tree is on, exactly as `git merge` does. Running `cgitsync merge multi-branch` while the tree is still on `multi-branch` would merge it into itself — which Git reports as success while doing nothing — so `cgitsync` refuses and says to check out the target branch first.

Landing a feature branch is therefore two commands:

```
$ pixi run cgitsync merge multi-branch
$ pixi run cgitsync merge --private multi-branch
```

5.3.30 `pull -private`

```
$ pixi run cgitsync pull --private
```

The other direction. While a project feature branch is open, work lands on a private/local repository's base branch and its derived branch does not see it. This fetches and merges the base branch into each one, so a settings branch does not drift behind the project's. A repository already on its base branch has nothing to take. The memory is kept and not merged here either (as above), and a repository whose base shares no commit with its branch is refused by name.

5.4 Document Formats

`.cgs` means **ComplexGitSync** specification, `.gts` means **GitTreeState** snapshot, and `.lgr` means **Local Git Register**.

5.4.1 Local Authoring Spec (`.cgs`)

A TOML file that describes the repository tree to manage. It is the only hand-authored input; it is never modified at runtime. The normal form has two top-level keys:

```
project = "CGSil1"
repos = [
  "gitlab:CGS_test/CGSil1",
  "gitlab:CGS_test/CGSil2",
  "github:flipoyo/CGSih1",
]
```

Repository identifiers have the form `provider:owner/repository`. Nested GitLab namespaces are supported; the final path segment is the repository name. The document pipeline is:

```
PARSE -> NORMALIZE -> VALIDATE -> canonical CgsDocument
```

The canonical providers and hosts are:

Provider	Host	Remote generation
`github`	`github.com`	SSH and HTTPS
`gitlab`	`gitlab.com`	SSH and HTTPS
`codeberg`	`codeberg.org`	SSH and HTTPS
`custom`	explicit URL required	SSH and HTTPS

Repository-ID parsing is provider-independent and deterministic. For example:

```
codeberg:GX4G/GX4G
```

normalizes to `gitprovider = codeberg`, `project_owner_name = GX4G`, and `repo_name = GX4G`. A custom identifier must use an inline table so the host is explicit:

```
{ repository = "custom:team/tool", gitprovider_url = "https://git.
    example.com" }
```

No host is inferred for `custom`. The configured access protocol then selects `git@host:owner/repository.gi` or `https://host/owner/repository.git`.

The textual grammar and its only parser, `parse_repo_id()`, live in `cgs_format.py`. The provider model in `git_repo.py` receives the already-separated provider, owner, and repository fields and composes remotes; it does not parse authoring shorthand.

The `.cgs` format pipeline is deterministic and offline. A syntactically valid repository may therefore reference a host, owner, branch, or tag that is not currently reachable. Remote existence and reference availability are checked only by explicit Git operations during runtime resolution and execution.

Normalization supplies `default_branch = main` (see "Branches in a `.cgs`" below for the full chain), makes `fallback_branch = main` (a private, writable entry: its own `default_branch`), selects `access_protocol = ssh`, enables `nested_config = auto`, and uses the repository name as `relative_path`. If exactly one repository name matches `project`, its path is inferred as `..`

Advanced inline tables are used only where a value differs from those defaults:

```
project = "demo"
repos = [
  "github:owner/demo",
  { repository = "github:owner/docs", relative_path = "documentation",
      access_protocol = "https" },
]
```

The legacy advanced `[project]` and `[[repos]]` tables remain accepted for project-wide branch settings, custom provider URLs, tags, and other explicit overrides. Unknown providers, malformed identifiers, duplicates, and missing `project` or `repos` are rejected before a `GitTree` is built.

Two checked-in files document this boundary. Copy `examples/template.cgs` for normal authoring. Its equivalent `examples/normalized_template.cgs` expands every inferred field and is provided as a developer reference for the canonical `CgsDocument` shape. The two files are regression-tested for semantic equivalence.

The repository also keeps three checked-in, didactically ordered tutorials under `tutorials/` — easiest first, see `tutorials/README.md` for the guided order — exercising this authoring surface end to end: `tutorials/01_first_multi_repo_workspace.md` walks a fully exercised local sandbox topology (`CGSil1`) through the complete CLI lifecycle; `tutorials/02_onboarding_a_real_build_tre` hand-authors a `.cgs` for the larger, real-world `cawaqs` tree and the point where `cgitsync` hands off to the project's existing `make`-based build workflow; and `tutorials/03_adopting_a_real_project.md` covers the real-world `cawaqsviz` project (not synthetic), which has no `.cgs` of its own and still tracks three repositories on two levels as git submodules, as one verified procedure: clone, check out the submodules, adopt the whole tree with `submodules init` (`discover` + `initialise` + `submodules import`, in that order), then `branch create/checkout/add/commit/push`.

`examples/cawaqsviz.cgs`, the hand-authored alternative mentioned in `tutorials/03_adopting_a_real_p` above, describes `https://gitlab.com/cawaqs/gviz/cawaqsviz` and exercises several patterns above at once: a nested GitLab subgroup path (`cawaqs/gviz/cawaqsviz`, three segments rather than a plain `owner/repository` pair), an explicit `relative_path = "."` on the root entry rather than relying on the project-name auto-mount rule, two GitHub children mounted at non-default relative paths (`external/HydrologicalTwinAlphaSeries` and `docs/CWV_user_guide`). Neither child has a `.cgs` of its own, but no override is needed for that: the default `auto` nested discovery resolves a repository with zero `*.cgs` matches as a normal leaf. This file previously shipped with a nonexistent repository identifier and an invalid `default_branch`; it was corrected and given real end-to-end test coverage (`tests/integration/test_cgsi_topology.py`, and one

verified run against the live repositories). See `tutorials/03_adopting_a_real_project.md` for the full worked example, and `bootstrap` above for running `ComplexGitSync` against it without cloning `ComplexGitSync` inside the tree it manages.

Generated specifications use the same concise representation. The `GitTree.to_cgs()` convenience method delegates model projection to `cgs_format.py`; neither `GitTree` nor `GitRepo` formats TOML or implements the authoring grammar. Serialization omits every default that normalization can deterministically reconstruct and retains inline tables only for exceptional settings such as a non-default branch, path, protocol, tag, custom host, or discovery rule.

The supported round trip is semantic:

```
.cgs -> CgsDocument -> GitTree -> CgsDocument -> .cgs
```

Parsing and normalizing the generated file must reproduce the initial canonical document. Whitespace, comments, and TOML table layout need not be byte-for-byte identical.

5.4.2 Branches in a `.cgs`

Two different questions, both decided per repository, and easy to conflate because both are spelled "branch": **which branch does this repository target**, and **what does `cgitsync` clone if the target turns out not to exist on the remote**.

Field	Where it is written	Default
`repos[].branch`	one repository entry	— (unset)
`repos[].tag`	one repository entry	— (unset)
`repos[].default_branch`	one repository entry	the project's
`project.default_branch`	the `project` table	`main`
`repos[].fallback_branch`	one repository entry	`main`*
`repos[].private`	one repository entry	`false`
`repos[].writable`	one repository entry	`false`

*A private, writable repository falls back to its own `default_branch` instead, so a configuration repository shared by several projects never lands on another project's `main`.

The target is decided by the first four, read as one chain, a tag winning outright over any branch:

```
tag -> branch -> default_branch -> project.default_branch -> main
```

`project.default_branch` is the tree's branch: every repository inherits it unless the repository says otherwise. `repos[].default_branch` is the branch that one repository targets. This chain governs the **root** repository exactly as it governs every other entry — a `branch`, `tag` or `default_branch` declared on the root targets that ref, the same as anywhere else in the tree. (An earlier version of `cgitsync` silently ignored these three fields on the root entry specifically, cloning `project.default_branch` regardless of what the root itself declared; `DiscoverRoundTrip` fixed this, so a `.cgs` written against that version and relying on the old silence should be checked.)

The fallback is a separate question, decided afterwards, only for repositories the chain above already resolved: `repos[].fallback_branch` is consulted only when the *target* branch turns out to be absent from the remote — `cgitsync` clones the fallback instead and records that it did so, rather than failing. A fallback is never itself a target: setting only `fallback_branch` on a repository, with no `branch`, `tag` or `default_branch` anywhere in its chain, still targets whatever `project.default_branch` says (`main` by default) and merely falls back to the declared branch if that target is missing on the remote — `discover -write` drafts both fields for exactly this reason (see `discover`, above): a tree scanned on a branch that differs from the root's own also

needs an explicit `default_branch`, not only a `fallback_branch`, or re-materialising it retargets that repository at the root's branch instead of reproducing what was scanned.

A repository that declares no `fallback_branch` falls back to `main`, no longer to its own `default_branch`: a fallback equal to the target is no second chance, and a missing branch would fail the clone instead (`No cloneable branch found ...expected one of ['lMOLO', 'lMOLO']`). Writing `fallback_branch = "main"` on an entry is therefore no longer needed.

Each link in the target chain is silent. Nothing warns you that a decision was made, so **state `project.default_branch` in every `.cgs` you write**, even when the answer is `main`. A reader of the file can then see the branch the tree lands on without knowing what `cgitsync`'s built-in default happens to be. Every `.cgs` shipped with ComplexGitSync does this, and a test enforces it.

What `private` means

A repository is `private` when it is **shared with other projects** and must stay on its own branch.

Branch moves stop at a private repository. Tags do not.

`cgitsync branch create <name>` and `cgitsync checkout <name>` skip a private repository and leave it on its declared `default_branch`, because moving it would drag every other project that mounts it onto your branch.

Read-only and writable configuration repositories

A private repository is a **configuration repository**: shared with other projects rather than owned by this one. `private = true` alone means **read-only** — `cgitsync` will not write to it. Add `writable = true` for the case where the repository lives outside your project but the part you use is yours, normally because it sits on a branch named after your project that nobody else reads:

```
{ repository = "github:you/.sharedSpec", default_branch = "main",
   private = true },
{ repository = "github:you/.myNotes", default_branch = "MyProject",
   private = true, writable = true },
```

Read-only is the default because the expensive mistake is writing to something shared by accident, never the reverse. `writable = true` on an unprivate repository is rejected at validation: an unprivate repository is this project's own and is always writable.

This is a **safety rail, not a permission system**. It stops a tree-wide sweep from writing where you did not mean; it cannot stop anyone running `git` in that directory by hand, and it is no substitute for branch protection on the remote.

A private/local repository gets a branch per project branch. A private/distant repository is private to its owner: you read it, only that owner writes to it, and nothing you do moves it — so a branch move leaves it exactly where it is. A private/local repository is the opposite case: it holds settings that configure *your* project and you commit to it, so it needs somewhere to record them per project branch.

`<project name>` on `main`, `<project name>_<branch>` on any other branch

The base is the **project's** name. `main` takes no suffix, because the project's main line's settings branch is simply the project's name — which is what every existing tree already has,

so nothing has to move. The separator is an underscore because hyphens already occur inside branch names.

A user never types the derived name. `cgitsync checkout <branch>` puts the project's own repositories on that branch and each private/local one on the branch this rule names, creating it when it is missing.

Without this, one settings branch serves every branch of the project, and documentation written for an unfinished feature is live on the default branch describing code that is not there.

`-private` everywhere. Every command that touches Git takes it: ten commands, `pull` (with or without `-force`), `fetch`, `checkout`, `branch` (on `create`, `list` and `close`), `add`, `rm`, `commit`, `merge`, `push` and `tag`. `release freeze` does not take it: it is a whole-tree release. It narrows the command to the writable configuration repositories alone, so they can be worked on without reaching for plain `git`. Read-only configuration repositories are never written to, with or without it. Four of those commands also take `-all`, described next.

`-all`: both halves, one command. Running every write twice — once plainly, once with `-private` — is the right default when the two changes are different changes. When they are one change, `-all` does both halves in a single pass, under a single commit message. Four commands take it: `add`, `commit`, `push` and `merge`.

Form	Reaches
`cgitsync <cmd>`	this project's own repositories
`cgitsync <cmd> -private`	the writable configuration repositories
`cgitsync <cmd> -all`	both of the above, in one pass

The plain form has not moved, so nothing an existing script runs changes meaning. `-all` and `-private` cannot be combined: `-all` already includes everything `-private` selects. No form of either writes to a read-only configuration repository — "all" means every repository `cgitsync` may write to, never every repository in the tree.

The output still names the two halves apart, so giving up the typing does not mean giving up seeing which repository was written as what. On a tree with no writable configuration repository, `-all` does the project half and reports the other as empty; that is the common case, not a fault. An explicit `-private` on the same tree still stops, because there the user asked for something that is not there.

`merge -all` checks every repository before merging any.

That is why `-all` is one pass and never two. Two sequential merges mean two preflights: the project half would merge, the configuration half would then refuse, and the tree would be left half-merged — exactly the state `merge`'s whole-scope check exists to prevent. `-dry-run` previews the same combined scope, and `-resolve` keeps its own warning and its deliberate partial progress.

A pin covers everything inside it. A configuration repository often holds repositories of its own, reached through its nested `.cgs`. Those are just as shared as the repository holding them, so they inherit its state: **a repository inside a read-only configuration repository is read-only, even when its own entry says nothing about it**. One inside a writable configuration repository is writable, and `-private` reaches it.

A nested entry may restrict itself further — writing `private = true` on its own line, with no `writable`, makes it read-only inside a writable parent. It cannot go the other way: `writable = true` inside a read-only configuration repository is ignored, because a repository can never be more open than the one that contains it.

Which repositories each command touches

Command	Reaches	Why
`add`, `commit`, `push`	this project's own	they write this project's history
the same, `-private`	writable config repos	shared, but on your own branch
`branch create`, `checkout`	this project's own	a branch must not spread
`tag`, `release freeze`	everything writable	they create *and push* a tag
`pull`, `status`	everything	reading a shared repo is the point

Preflight follows the same scope. Each command checks only the repositories it is about to touch. A configuration repository that is behind its upstream, or sitting on its own branch, never blocks a `commit` of this project's own work — that repository is not part of the commit. Under `-private` it is in scope, and its branch is then checked against the branch it is *private* to, not against the one the project is on.

Which commands accept `-private`. Exactly these: `pull`, `fetch`, `checkout`, `branch`, `add`, `rm`, `commit`, `merge`, `push` and `tag` — and nothing else.

`-private` is exclusive, not additive: it selects *only* the writable configuration repositories, so a shared repository gets its own command and its own commit message rather than being swept into this project's. Asking for it in a tree with no writable configuration repository is an error naming the read-only ones, not a silent no-op. Every scoped command reports what it left alone:

```
scope=project skipped=5 configuration repo(s) (.claude, .localSpec with
   --private)
```

`tag` and `release freeze` stop at read-only repositories because they push. Nothing is lost: a `.gts` snapshot records every repository's exact `commit_sha`, read-only ones included, so an exact tree is rebuilt from the snapshot rather than from tags, which is what `release load` does.

In practice shared config mounts are usually dot-named (`.agentSpec`, `.localSpec`), and `cgitsync discover` drafts a dot-named repository with `private = true` for that reason. The rule is *shared*, not *hidden*, though: a plainly named repository that two projects mount should be private too.

The branch a repository is private to decides who sees your commits. A mount private to a branch named after your own project is yours to push freely. A mount private to `main` is read by every project that mounts it, so a commit there is published the moment it is pushed — no branch, no pull request, no review. Check which of the two you are looking at before committing to a private mount.

The two meanings of "force"

`pull -force` is destructive, but it loses nothing: uncommitted and untracked work is stashed (`git stash pop` restores it), and it refuses, for the whole tree, while a commit exists that no remote holds, so it never drops a commit. `-force-protocol` is not: it only switches a remote between `ssh` and `https`. The two are unrelated despite the shared word.

5.4.3 Repository Placement and Nested Discovery

For every non-root entry, `relative_path` is relative to the parent repository represented by the current `.cgs`; it is not relative to the process working directory. At the top level that parent is the project root. While expanding a nested config, it is the repository containing that config. The default is the repository name. A unique repository matching the project name uses `.` because it represents the current root.

The `nested_config` option controls whether traversal continues after the repository is available:

- `auto` (default) loads the sole root-level `*.cgs` file, if one exists; finding none resolves the repository as a normal leaf; multiple matches are rejected as ambiguous.

  A consequence worth knowing before you publish a repository: **a repository that other people may mount inside their tree must keep exactly one `.cgs` at its root**, because `auto` has no way to choose between two. The tree's own root is exempt — it is never scanned for its own `.cgs`, since that would recurse — so a root holding two `.cgs` files works fine until the day someone mounts it. The error names both files and how to resolve it.

- `disabled` does not inspect the repository for nested config.

- A relative path ending in `.cgs` loads that exact file inside the repository, failing if it does not exist. It may not escape the repository root.

For the integration topology, `CGSil2.cgs` contains:

```
{ repository = "github:flipoyo/CGSih1", relative_path = "../CGSih1" }
```

Because the config describes the tree below `CGSil2`, the path resolves from that directory:

```
<CGSHOME>/CGSil2/../CGSih1 -> <CGSHOME>/CGSih1
```

It therefore references the existing sibling declared by `CGSil1.cgs`, instead of cloning another `CGSih1` under `CGSil2`. Discovery recognizes the already-registered absolute path and keeps the canonical entry — this guard runs before `auto` would ever get a chance to reopen `CGSih1.cgs` through the cross-reference, so no `nested_config` override is needed here.

5.4.4 Git Tree State Snapshot (`.gts`)

A TOML file generated automatically by bootstrapping and release operations. It must never be hand-edited. It captures the exact commit SHAs and lifecycle state of the tree at a point in time.

Top-level sections:

- `[document]` — `format_version`, `schema_version`, `generated_at`, `command_origin`, `hash_algorithm`, `snapshot_hash`.
- `[project]` — project name and `root_absolute_path`.
- `[tree_state]` — `lifecycle_state`, `is_ready`, `registry_complete`.
- `[[repo_state]]` — one entry per repo; includes `commit_sha`, `sync_state`, and ref information.

Why both version fields and hash metadata exist:

- `format_version` identifies the broad document family and keeps long-range compatibility intent explicit.
- `schema_version` identifies the exact field-level contract used by validation and canonical snapshot serialization.
- `snapshot_hash` (with `hash_algorithm = "sha256"`) provides deterministic workspace identity: identical tree state produces identical digest even if volatile metadata (`generated_at`, `command_origin`) differs.

How this is used at runtime:

- During snapshot generation, ComplexGitSync writes `schema_version`, `hash_algorithm`, and canonical `snapshot_hash`.
- During load/validation, `snapshot_hash` is recomputed from the canonical payload and must match.
- The project-local `.lgr` register uses this canonical hash to deduplicate equivalent snapshots.

5.4.5 Local Git Register and Sync Ledger (`.lgr`)

A TOML file maintained per project at `<project-root>/<Project_name>.lgr`. It is updated automatically whenever a `.gts` snapshot is written.

Sections:

- `[register]` — current snapshot pointer (`current_snapshot_id`, `current_snapshot_hash`, `current_snapshot_path`).
- `[[snapshots]]` — list of stable `gts-XXXXXX` identifiers, one per unique workspace state, deduplicated by SHA-256 hash.
- `[[ledger]]` — append-only list of synchronisation events forming a DAG via `parent_sync_ids`.

Ledger event schema:

```
[[ledger]]
sync_id         = "lgr-000001"
parent_sync_ids = []
operation       = "clone"
timestamp       = "2026-05-20T19:48:50.159Z"
actor           = "username"
workspace_hash  = "5f7c8a2d..."   % 64-character SHA-256 hex digest
gts_snapshot_id = "gts-000001"
affected_repos  = ["demo", "dep-a"]
```

The `workspace_hash` field equals the canonical SHA-256 digest (`document.snapshot_hash`) of the associated `.gts` file, creating a direct link between each ledger event and the immutable workspace state it records. Events are parents-before-children in DAG order.

5.5 Worked Examples: From `CGSil1` to `cawaqsviz`

`ComplexGitSync` ships a tutorial under `tutorials/` for each level below (`tutorials/README.md` lists them in order). Run through them in order: each one adds exactly one new idea on top of the last, so together they form a guided path from the simplest possible `.cgs` to a real project with no `.cgs` of its own at all.

Levels 2 and 3 have their `.cgs` checked in under `examples/`. Level 1 does not: `CGSil1.cgs` lives in the `CGSil1` repository itself, which is what the transcript below clones.

Every command below is a Pixi task. `cgitsync` is not a globally installed executable — always run `pixi run cgitsync ...`, never a bare `cgitsync ...` (see the §5 "CLI Overview" note above).

Level	Topology	Entry point	New idea introduced
1	`CGSil1`	hand-authored `.cgs`	minimal spec, name-matching auto-mount
2	`cawaqs`	hand-authored `.cgs`	the same minimal-field style as Level 1, at 19-repo scale
3a	`cawaqsviz`	hand-authored `.cgs`	explicit overrides on a real, irregular tree
3b	`cawaqsviz`	`discover`	drafting a `.cgs` from a checkout, no authoring at all
3c	`cawaqsviz`	`submodules import`	migrating an existing git-submodules project

Levels 3a–3c are not three different projects: they are three different *starting points* for the same real project, `cawaqsviz`, chosen so this section can demonstrate every route `ComplexGitSync` offers for arriving at a working `.cgs` — write it by hand, derive it from a checkout, or migrate it from an existing submodules setup — without inventing a fourth toy topology to do it. It is the most advanced level not because `cawaqsviz` is a larger tree than `cawaqs` (it is much smaller, three repos against nineteen) but because, unlike either earlier level, it has no `.cgs` anywhere upstream to copy from.

5.5.1 Level 1 — `CGSil1`: the minimal synthetic topology

`CGSil1` is the easiest way into `ComplexGitSync`: a 4-repository, mixed-provider tree with every field left at its default. Its full `.cgs` and the rationale for each line are in §5 "Local Authoring Spec" above; `tutorials/01_first_multi_repo_workspace.md` installs it and `tutorials/02_working_with_a` works on it, with the expected output of each step. What follows is the command sequence itself.

```
# 1. Fetch the root repo and cgitsync itself (nested mode: cgitsync is
#    cloned inside the tree it will manage).
$ git clone https://gitlab.com/CGS_test/CGSil1
$ cd CGSil1 && git clone https://github.com/flipoyo/ComplexGitSync
$ cd ComplexGitSync

# 2. Check the spec offline, no network beyond parsing.
$ pixi run cgitsync validate ../CGSil1.cgs

# 3. Preview the topology before touching disk.
$ pixi run cgitsync view-tree ../CGSil1.cgs

# 4. Clone every declared repo and reach READY.
$ pixi run cgitsync initialise ../CGSil1.cgs

# 5. The ordinary git cycle, run tree-wide from here on --- no path
#    argument needed, the .gts snapshot is discovered automatically.
$ pixi run cgitsync pull
$ pixi run cgitsync add
$ pixi run cgitsync commit "my commit message"
$ pixi run cgitsync push

# 6. Release: stage, commit, pull, push, tag and record the State.
   CGSil1 declares
#    no version in a pixi.toml, so the release is numbered: CGSil1-1.
$ pixi run cgitsync release freeze "release v1.1.0"

# 7. Return to a release at any later point, from any workspace.
$ pixi run cgitsync release list
```

```
$ pixi run cgitsync release load 1
```

The only `.cgs` idea this level relies on is the **name-matching auto-mount**: because `CGSil1`'s repository identifier's last segment equals `project = "CGSil1"`, the root is inferred at `relative_path = "."` with no override written anywhere. That convenience is exactly what stops being safe once a project's identifier and display name diverge — which is where Level 3a picks up.

5.5.2 Level 2 — `cawaqs`: the same minimal style, at scale

`cawaqs` scales Level 1's minimal, no-override authoring style up to a real 19-repository build tree (root + 17 physics libraries across five GitLab groups + one required build-tooling repo), instead of a synthetic 4-repo one. Every entry's on-disk layout already matches the default `relative_path = <repo_name>`, so not a single `relative_path` override is written anywhere in `examples/cawaqs.cgs` — only the shared branch-fallback convention. Full narrative in `tutorials/03_onboarding_a_real_build_tree.md`.

```
$ pixi run cgitsync validate examples/cawaqs.cgs
$ pixi run cgitsync bootstrap examples/cawaqs.cgs cawaqs-smoke-test
$ pixi run cgitsync view-tree --search-dir <path bootstrap printed>

# Move the whole 19-repo tree to a shared branch in one call, falling
# back to main per-repo wherever that branch does not exist.
$ pixi run cgitsync checkout my-feature-branch
```

Reaching `READY` here only replaces the *repository-fetching and branch-selection* half of `cawaqs`'s own `make_Cawaqs.sh`/`make_Cawaqs_from_branches.sh` scripts. `ComplexGitSync` does not compile anything: the remaining `make -f Makefile` build step is still the user's job, and only succeeds from this tree with `LIB_HYDROSYSTEM_PATH` left unset or pointed at the bootstrapped root — the alternative shared, decoupled install layout those scripts also support is out of scope for `ComplexGitSync`'s containment model.

`discover` (Level 3b below) is not a route into `cawaqs.cgs`: the 17 library repositories never coexist inside one directory tree until *after* a `.cgs` already lists them, so there is no single checkout for a filesystem walk to scan. Authoring this one from documented developer knowledge, the way Level 3a does for `cawaqsviz`, is the only available route.

5.5.3 Level 3 — `cawaqsviz`: three ways to the same `.cgs`

`cawaqsviz` (`https://gitlab.com/cawaqs/gviz/cawaqsviz`) is a real GitLab project, nested three path segments deep, with two GitHub children that were, until recently, plain git submodules. One of those children, `HydrologicalTwinAlphaSeries`, holds a submodule of its own at `docs/hydrological_twin`: four repositories on two levels, which is why every command below uses `-recursive`. It has no `.cgs` of its own anywhere upstream. That makes it a good vehicle for showing all three ways `ComplexGitSync` can produce a working `.cgs` for a project it does not already control the source of, ordered here by how much the operator has to already know about the tree before running the command. Full narrative, as one combined ten-step procedure rather than three independent routes, in `tutorials/04_adopting_a_real_project.md`.

3a — Write it by hand

The most explicit, and most error-prone, route: read the project's own `.gitmodules` and topology, then author `examples/cawaqsviz.cgs` directly. Its corrected content and the exact mistakes the first draft made (wrong nested-subgroup identifier, missing `relative_path` overrides) are already spelled out in §5 "Local Authoring Spec"; this level is about running it, not rederiving it:

```
$ pixi run cgitsync validate examples/cawaqsviz.cgs
$ pixi run cgitsync bootstrap examples/cawaqsviz.cgs cawaqsviz-demo
$ pixi run cgitsync view-tree --search-dir <path bootstrap printed>
```

`bootstrap` is used instead of `initialise` here because, unlike `CGSil1` above, this example is not meant to have `ComplexGitSync` cloned inside the tree it manages — see `bootstrap` in §5 for why the two commands pick different default locations.

3b — Derive it with `discover` (autodiscover mode)

The opposite route: start from nothing but a checkout, and let `ComplexGitSync` read the filesystem instead of writing the `.cgs` by hand. This is the route to prefer whenever a checkout is already available, since it cannot repeat the hand-authoring mistakes of 3a — `relative_path` falls out of the walk directly instead of being typed.

```
# A plain clone leaves submodule paths empty; initialise them first so
# discover has something to find at those paths. --recursive reaches the
# second level too: without it, hydrological_twin stays an empty
# directory and discover cannot see a repository that is not there.
$ git clone https://gitlab.com/cawaqs/gviz/cawaqsviz cawaqsviz
$ cd cawaqsviz && git submodule update --init --recursive

# Dry run: report what discover sees, write nothing.
$ pixi run cgitsync discover . --max-depth 3

# Satisfied with the report: draft the .cgs.
$ pixi run cgitsync discover . --write cawaqsviz-discovered.cgs
$ pixi run cgitsync validate cawaqsviz-discovered.cgs
```

The draft reconstructs 3a's hand-corrected file field-for-field: the root at `relative_path = "."` and every other repository at its real submodule path (`external/HydrologicalTwinAlphaSeries`, `docs/CWV_user_guide`, `external/HydrologicalTwinAlphaSeries/docs/hydrological_twin`) rather than their bare repo names. That last path lies inside `HydrologicalTwinAlphaSeries`, so the report marks it `inside: external/HydrologicalTwinAlphaSeries` and the file is read back with `HydrologicalTwinAlphaSeries` as its parent. No repository here has a `.cgs` of its own, and `discover` writes no `nested_config` override for that: the default `"auto"` already resolves a repository with zero nested `*.cgs` matches cleanly. See `discover` in §5 for exactly what is and is not reported (no network calls, unresolvable remotes are warned about and left out, never guessed at).

3c — Migrate it with `submodules import`

The historical route: before either `.cgs` above existed, `cawaqsviz` tracked its children as real git submodules. This mode starts from that state and converts it, rather than describing a tree that is assumed to already be plain clones.

```
# Dry run against a real cawaqsviz checkout that still has submodules:
# reports name, path, URL, branch and declaring .gitmodules per
    submodule;
# changes nothing. --recursive reaches HydrologicalTwinAlphaSeries' own
# submodule; without it, only the root's two are converted.
$ pixi run cgitsync submodules report /path/to/cawaqsviz --recursive

# Convert: drop the gitlinks, retire .gitmodules, extend .gitignore.
$ pixi run cgitsync submodules import /path/to/cawaqsviz --recursive
```

```
# Review and commit the resulting working-tree changes as usual. Two
# repositories have changes here, one per converted level.
$ cd /path/to/cawaqsviz && git status
$ git commit -m "chore: retire git submodules in favour of
   ComplexGitSync"
```

This is the whole job: turning gitlinks into plain clones on disk. It does not also write a `.cgs` — `.gitmodules` never records the root's own identity, and a checkout worth converting already has a `.cgs` (3a) or can get one from `discover` (3b), run before or after `submodules import`. The full before/after picture (index, `.gitmodules`, `.gitignore`) and the automated test that proves it against fixture repositories are in `tutorials/04_adopting_a_real_project.md` §4 — this level is the condensed command sequence, not a restatement of that walkthrough. `submodules import` and `discover` answer different questions about the same tree: `discover` reports what is *checked out*, `submodules import` reads what git *declares* in `.gitmodules` and acts on it — a submodule path that was never initialised is invisible to 3b but not to 3c.

A shared caveat across all three

`ComplexGitSync` stores no credentials and has no private-repository authentication story (§5 "`submodules report` and `submodules import`" and "`discover`"). All three routes above assume every repository in the tree is reachable anonymously or through credentials the ambient environment (`ssh-agent`, an HTTPS credential helper) already holds.

5.6 Configuration and Environment

`ComplexGitSync` uses project files for topology and does not require global topology configuration. Snapshot discovery can use `CGSHOME`; when it is not set, the `initialise(.cgs)` output-path default is `CGSPATH=../..` (nested mode: `ComplexGitSync` cloned inside the tree it manages); later commands can also walk upward from the current directory to find `.cgitsync/state/`. `bootstrap(.cgs)` uses a different default instead — `CGSHOME=$HOME/.cgs/<project name>-<timestamp>`, created automatically — since it is meant for a `ComplexGitSync` clone that is not nested inside any project tree; see `bootstrap` above. The logging and runtime-state subsystems use the user state directory by default:

- logs: `/.local/state/ComplexGitSync/logs/`
- latest runtime-state mapping: `/.local/state/ComplexGitSync/runtime/`

If `XDG_STATE_HOME` is set, these directories are rooted there instead.

5.7 Error Reference

`ConfigValidationError` Raised when `project` or `repos` is missing, a repository identifier is malformed, or a canonical field is invalid (for example an unknown `gitprovider`).

`GitSyncError` Raised when clone-time Git operations fail, for example because the remote branch cannot be resolved or the target directory is already occupied.

Exit code 1 General CLI failure (parse error, validation error).

Exit code 2 Command exists but is not yet implemented.

5.8 Troubleshooting

"`gitprovider_url` is required for custom provider" Add a `gitprovider_url` field to the offending `[[repos]]` entry in your `.cgs` file.

`validate` exits non-zero despite a correct-looking file Check that `project` is present and every `repos` entry uses `provider:owner/repository`. For advanced `[[repos]]` blocks without a `repository` shorthand, include `project_owner_name` and `project_name`.

`Clone destination already exists and is not empty` Move the existing directory aside yourself, or point `bootstrap` at a fresh `-cgs-path`.

`No cloneable branch found for <repo>` Confirm that the remote exposes either the repo's `default_branch` or its `fallback_branch`.

PyYAML import error when calling `from_yaml`/`to_yaml` Add PyYAML in the Pixi environment: `pixi add pyyaml`.